

Business Entity Resolution System

Architecture and Design Document

Project	Amazon ML Challenge – Business Entity Resolution
Repository	Manureddy148/Ai
Component	code/business_entity_resolution/src/er_pipeline.py
Document version	1.0
Status	Draft for review

This document describes the architecture of an end-to-end entity-resolution pipeline that links noisy business records from three independent sources without any external data: text normalisation, memory-bounded candidate generation, pairwise feature engineering, a gradient-boosted matching model, and a precision-oriented decision layer tuned for the challenge's macro- $F_{0.5}$ metric. It is written for engineers who need to run, audit, extend or re-implement the system.

Contents

1 Executive Summary	5
1.1 Purpose and scope	5
1.2 The problem	5
1.3 The solution	5
1.4 Key design principles	6
1.5 What is delivered	7
1.6 Document map	8
2 Problem Statement and Evaluation Metric	8
2.1 Formal task definition	9
2.2 File formats	9
2.3 The evaluation metric	10
2.4 Public and private leaderboards	12
2.5 Constraints	12
2.6 Implications of the metric for each pipeline stage	13
3 Data Understanding and Noise Taxonomy	15
3.1 Sources and What Each Contributes	15
3.2 One Entity, Three Records	16
3.3 Noise Taxonomy	16
3.4 Country-Specific Address Anatomy	18
3.5 The Open-Set Country Requirement	19
3.6 Profiling the Data: What profile() Logs	20
4 System Architecture Overview	22
4.1 Architectural style	22
4.2 Module map	22
4.3 Data contracts between stages	24
4.4 Control flow of main()	24
4.5 Runtime environment and dependencies	25
4.6 Logging design	26
4.7 Configuration surface	28
5 Text Normalisation Layer	30
5.1 Role of Normalisation in a Precision-Weighted Pipeline	30
5.2 Character Tier: basic()	30
5.3 Token Tier: expand() and the Four Vocabularies	31
5.4 Name Columns: norm_name(), core_name() and name_parts()	33
5.5 Address Columns: norm_addr(), _postal and _addr_nums()	33
5.6 prep(): Output Columns and Worked Examples	34
6 Candidate Generation (Blocking)	37
6.1 Why blocking is needed	37
6.2 Representation: character n-gram TF-IDF	37
6.3 The four views	37
6.4 Partitioning by country and the open-set fallback	39
6.5 Bounded top-k retrieval: topk_sparse()	39
6.6 Memory budget and time complexity	40
6.7 Cosines for every surviving pair: pair_cos()	41
6.8 Diagnostics and how to act on them	41

7 Pair Feature Engineering	42
7.1 Role of the feature layer	42
7.2 TF-IDF cosines	43
7.3 rapidfuzz name similarities	44
7.4 Alias and acronym features	45
7.5 Address features	45
7.6 Lengths and source flag	45
7.7 Context features	46
7.8 Leakage, categorical handling and value encoding	47
8 Matching Model: LightGBM Classifier	48
8.1 Why gradient-boosted trees for pairwise entity resolution	48
8.2 Hyperparameters	48
8.3 Label construction and class balance	49
8.4 Grouped cross-validation and out-of-fold assembly	50
8.5 Early stopping, best iteration and the refit	50
8.6 Reading the gain importances	51
8.7 Model interface and how to swap the model	51
8.8 Quantities recorded from the run	52
9 Decision Layer: From Probabilities to Match Lists	53
9.1 Role of the decision layer	53
9.2 decode_table(): assignment and group maxima	
9.3 decide(): an absolute and a relative bar	
9.4 Metric arithmetic behind the rule	55
9.5 Out-of-fold selection of (thr, ratio)	56
9.6 to_lists(), ordering and the writer	
10 Validation Strategy and Tooling	58
10.1 Validation protocol	58
10.2 Diagnostics printed by main()	59
10.3 The scorer: utils/score.py	60
10.4 Producing a train-OOF prediction file (suggested addition)	61
10.5 The validator: utils/validate_submission.py	61
10.6 Error-analysis playbook	62
10.7 Pre-submission checklist	63
11 Outputs, Packaging and Reproducibility	64
11.1 The output files	64
11.2 candidate_pairs.tsv: the set the model scores	65
11.3 Repository layout and the submission package	65
11.4 The Kaggle notebook	66
11.5 Determinism	68
11.6 Dependency pinning	68
11.7 Reproducibility checklist	68
11.8 Run-time expectations	69
12 Performance, Complexity and Scalability	71
12.1 Notation and cost model	71
12.2 Asymptotic complexity per stage	71
12.3 Memory model	73
12.4 Scaling knobs	73
12.5 Where time goes and how to profile it	74
12.6 Parallelism and optimisation opportunities	76

12.7 Kaggle limits and capacity	77
13 Fair Play, Licensing and Constraint Compliance	78
13.1 Compliance matrix	78
13.2 No external lookup	79
13.3 Licence and model-size constraints	80
13.4 Output-format constraints by construction	81
13.5 Country as an open set	81
13.6 Auditability of blocking	81
14 Limitations, Risks and Roadmap	83
14.1 Status of the evidence	83
14.2 Limitations	83
14.3 Risk register	85
14.4 Roadmap	86
15 Appendices	89
15.1 Appendix A: CLI and configuration reference	89
15.2 Appendix B: Log-line reference	92
15.3 Appendix C: File format reference	94
15.4 Appendix D: Glossary	95
15.5 Appendix E: Function index	97

1 Executive Summary

This document describes the architecture of a record-linkage system for the Amazon ML Challenge on Business Entity Resolution. The system links every Source 1 business record to the Source 2 and Source 3 records that describe the same business, or declares it a singleton, using only the provided files. It is one Python module, `er_pipeline.py`, with the stages load, normalise, block, pair features, LightGBM matcher, (threshold, ratio) decode and TSV output. Every design decision follows from the metric: macro-averaged $F_{0.5}$ per Source 1 entity, where a singleton scores 1.0 only when the predicted list is empty.

1.1 Purpose and scope

The document serves two audiences. For challenge reviewers it is the technical companion to the methodology write-up: it explains, at the level of functions, regular expressions and parameter values, how the pipeline turns the input files into `matching_results.tsv` and `candidate_pairs.tsv`, and why each stage is shaped as it is. For maintainers it is the reference for the data contracts between stages, the memory budget, the validation protocol and the known limitations. Every claim is tied to the code in `er_pipeline.py`. The pipeline has not yet been executed on the challenge data; any quantity that only a run can produce is marked *to be filled from the run log*.

1.2 The problem

Three independent sources describe the same population of businesses without a shared identifier. Each record carries an `entity_id` whose prefix (S1-, S2-, S3-) names its source, a free-text `business_name`, a free-text `business_address` and a `country` label. Source 1 is the deduplicated reference: a Source 1 entity may correspond to zero, one or many records in Sources 2 and 3, and because no two Source 1 records denote the same business, the pipeline assumes that a Source 2 or 3 record belongs to at most one Source 1 entity (Chapter 2). The fields carry the noise typical of business directories: abbreviated or expanded legal forms, trade names introduced by markers such as *dba*, ampersands versus *and*, transposed tokens, typos, abbreviated street types, missing postal codes and landmark-based addresses. Training data cover the United States and India with a ground-truth file; the test data add France, never seen in training, whose records must all be resolved (see Chapter 3).

The scoring rule shapes the solution more than any property of the data. For each Source 1 entity the organisers compute $F_{0.5} = 1.25 \cdot P \cdot R / (0.25 \cdot P + R)$, with P and R the precision and recall of the predicted list against the true list, and average over all Source 1 entities. With $\beta = 0.5$ precision is weighted twice as heavily as recall: one wrongly merged record costs more than one missed record. Singletons are included and scored in a binary way: an entity with no true match earns 1.0 if the predicted list is empty and 0.0 otherwise. A system must therefore be conservative on every entity and treat "no match" as a first-class prediction (see Chapter 2).

1.3 The solution

The system is a supervised record-linkage pipeline with the seven stages of Figure 1.1. **Load** (`load_split()`) reads the three tab-separated source files of a split as strings, concatenates Sources 2 and 3 into one frame, and parses the ground truth into a dictionary from Source 1 id to a set of matched ids. **Normalise** (`prep()`) derives for every record an ASCII-folded, abbreviation-expanded name (`name_n`), a core name without legal forms (`name_c`), alias variants split on markers such as *dba* (`name_alts`), a stop-word-free address (`addr_c`), the

postal code, the digit runs and the leading house number. **Block** (`block()`) fits TF-IDF character 2–4-gram vectorisers on the split's own texts for three views (core name, name plus address, address) and, within each country, retrieves the top-k Source 2/3 neighbours per view ($k = 25, 25$ and 10) with the chunked sparse product in `topk_sparse()`; a fourth view restricted to a shared postal code adds up to 10 name-cosine neighbours. Each view keeps only pairs with cosine above 0.05; the de-duplicated union of the four views is the candidate table, which is exactly the set the matcher scores and which is written, as one id list per Source 1 entity, to `candidate_pairs.tsv`.

Features (`features()`) computes per candidate pair the view cosines, rapidfuzz and Jaro-Winkler similarities, token Jaccard, first-token, initials, alias and acronym agreement, address similarities, ternary postal-code and house-number agreement, street-number overlap and conflict, lengths, a source flag, and contextual features that compare the pair with the other candidates of the same Source 1 entity and of the same Source 2/3 record (gap to the best value in the group, so that the best candidate has gap 0; rank by `cos_full`; candidate count). **Match** trains a LightGBM binary classifier on the labelled training pairs with 5-fold GroupKFold grouped by Source 1 entity and early stopping on the held-out fold, yielding out-of-fold probabilities, and then refits a final model on all pairs for 1.1 times the mean best iteration of the folds. **Decode** (`decode_table()`, `decide()`) assigns each Source 2/3 record to the Source 1 entity that scores it highest and keeps a pair only when its probability reaches an absolute threshold *and* a fixed fraction of the best probability in its (Source 1, source) group; (thr, ratio) is chosen by exhaustive sweep on the out-of-fold macro $F_{0.5}$ from `macro_f05()`, singletons included. **Write** emits one row per test Source 1 entity in each TSV with de-duplicated id lists.

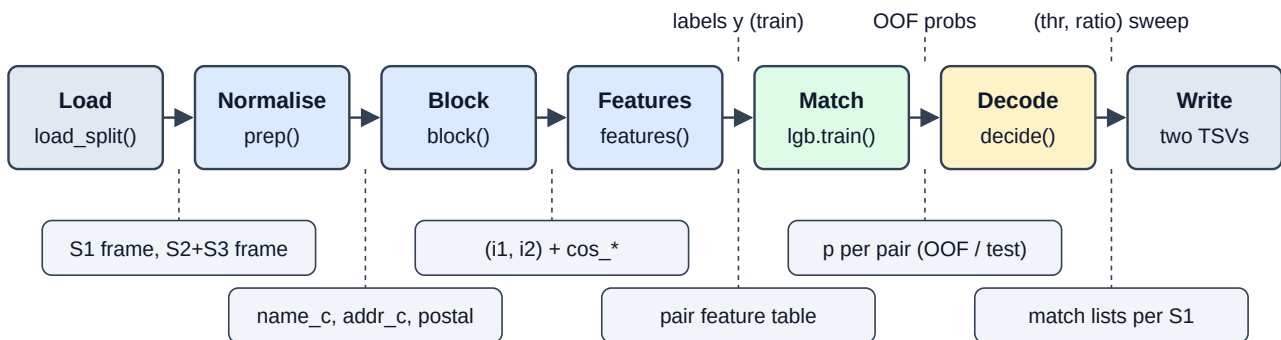


Figure 1.1 — High-level pipeline: the seven stages of `main()` and the artefact each stage hands to the next. The same `prep()` → `block()` → `features()` path runs on the training and the test split; labels, out-of-fold probabilities and the (thr, ratio) sweep exist only on the training side.

1.4 Key design principles

Six principles run through every stage; Table 1.1 records where each is realised in the code.

- **Precision first.** Because $F_{0.5}$ penalises a false merge more than a missed link, the decision layer never uses a naive 0.5 cut-off: it combines an absolute threshold, a relative ratio to the strongest competitor in the same (Source 1, source) group and a one-Source-1-per-record constraint, and selects the operating point directly on the official metric.
- **Singleton awareness.** `f05()` reproduces the organisers' rule exactly, and the (thr, ratio) sweep averages over every training entity, so the cost of a spurious link on a singleton is part of the objective being optimised.
- **Open-set country.** Country only partitions blocking; a label absent from Sources 2 and 3 falls back to the whole index, the normalisation tables include French legal forms and address idioms, and the vectorisers are refit per split, so France is handled without any code path that names it.

- **Memory-bounded computation.** Top-k retrieval densifies one chunk of Source 1 rows at a time, sized from the block width to a budget of 2×10^7 cells, and pair cosines are computed in slices of 200,000 pairs, so peak memory does not grow with a country block or the candidate table.
- **No external data.** The only inputs are the provided TSVs; there is no gazetteer, registry, geocoder, pretrained model or web access, and the matcher is an MIT-licensed LightGBM ensemble far below the 8-billion-parameter limit.
- **Train/test parity.** The training and the test split pass through the same `prep()`, `block()` and `features()` calls with the same parameters, so the feature distribution the matcher was fitted on is the one it scores at inference, and the operating point chosen on out-of-fold probabilities transfers without adjustment. The same candidate table feeds both the matcher and `candidate_pairs.tsv`, so the rule that every matched id must also be a candidate holds by construction rather than by a post-hoc check.

Table 1.1 — Design principle and where it is realised in `er_pipeline.py`

Principle	Mechanism	Location in code
Precision first	Keep a pair only if $p \geq \text{thr}$ and $p \geq \text{ratio} \cdot \text{best}$; each S2/S3 record assigned to its highest-scoring S1; (thr, ratio) chosen by exhaustive sweep on out-of-fold macro $F_{0.5}$	<code>decide()</code> , <code>decode_table()</code> (<code>drop_duplicates("i2")</code>), sweep loop in <code>main()</code>
Singleton aware	Per-entity score is 1.0 iff truth and prediction are both empty; macro average runs over all S1 ids, not only matched ones; empty rows written for unmatched entities	<code>f05()</code> , <code>macro_f05()</code> with keys <code>tr1.entity_id</code> , TSV writer in <code>main()</code>
Open-set country	Blocking groups on <code>country_n</code> ; an S1 label with no S2/S3 counterpart compares against all S2/S3 rows; French forms in the normalisation tables; vectorisers fit per split	<code>block()</code> (other_countries fallback), <code>LEGAL</code> , <code>NAME_ABBR</code> , <code>ADDR_ABBR</code> , <code>ADDR_STOP</code>
Memory bounded	Chunk size = $\max(16, 2 \times 10^7 // B)$; argpartition slice copied; float32 TF-IDF; pair cosines in 200k-pair slices	<code>topk_sparse()</code> , <code>pair_cos()</code> , <code>TfidfVectorizer(dtype=np.float32)</code>
No external data	Reads only <code>*_source{1,2,3}.tsv</code> and <code>train_ground_truth.tsv</code> ; no network calls except an optional package install; model is LightGBM (MIT)	<code>find_data_dir()</code> , <code>read_tsv()</code> , <code>load_split()</code> , <code>params</code> in <code>main()</code>
Train/test parity	Identical <code>normalise</code> → <code>block</code> → <code>features</code> path for both splits; matches are a subset of candidates by construction; id lists de-duplicated with <code>dict.fromkeys</code>	<code>prep()</code> , <code>block()</code> , <code>features()</code> , <code>to_lists()</code> , writer in <code>main()</code>

Design note. The sweep includes `ratio = 0`, which reduces `decide()` to a flat threshold, so the relative rule is adopted only when it improves the out-of-fold macro $F_{0.5}$.

1.5 What is delivered

The package follows the structure the organisers require. `code/business_entity_resolution/src/er_pipeline.py` is the complete pipeline; `requirements.txt` lists its six dependencies (NumPy, pandas, SciPy, scikit-learn, LightGBM, rapidfuzz) with minimum versions, and `README.md` gives the run command. `utils/validate_submission.py`, a standard-library script, checks both output files against every formatting rule of the statement, and `utils/score.py` computes macro $F_{0.5}$ on a labelled split. `notebooks/kaggle_run.ipynb` locates the dataset in a Kaggle session (an attached input dataset or a downloaded archive), runs the script with an explicit `--data-dir`, validates the outputs and packages the submission zip; when the script is run without `--data-dir`, `find_data_dir()` searches `/kaggle/input`, the working directory, its parent and `dataset/` for `train_source1.tsv`.

`Documentation_template.md` holds the methodology write-up, and this document sits under `docs/architecture/`. The `output/` folder receives the two TSVs once the pipeline has been run (see Chapter 11).

Risk. Every quantitative claim is conditional on a run that has not yet taken place. Blocking recall, out-of-fold macro $F_{0.5}$, the selected (thr, ratio) and per-country match rates remain placeholders until the run log exists (Chapters 10 and 12).

1.6 Document map

Chapter 2

Problem Statement and Evaluation Metric: task, file contracts, $F_{0.5}$ and the singleton rule.

Chapter 3

Data Understanding and Noise Taxonomy: schema, country split, name and address noise.

Chapter 4

System Architecture Overview: stage graph, inter-stage frames, control flow of `main()`.

Chapter 5

Text Normalisation Layer: `basic()`, alias splitting, lookup tables, postal and number extraction.

Chapter 6

Candidate Generation (Blocking): TF-IDF views, postal view, `topk_sparse()`, recall diagnostics.

Chapter 7

Pair Feature Engineering: every column of `features()`, including the contextual features.

Chapter 8

Matching Model: LightGBM Classifier: parameters, GroupKFold protocol, early stopping, refit.

Chapter 9

Decision Layer: From Probabilities to Match Lists: `decode_table()`, `decide()`, the (thr, ratio) sweep.

Chapter 10

Validation Strategy and Tooling: log diagnostics, `score.py`, `validate_submission.py`, run-log table.

Chapter 11

Outputs, Packaging and Reproducibility: TSV writer, submission zip, Kaggle notebook.

Chapter 12

Performance, Complexity and Scalability: time and memory profile per stage.

Chapter 13

Fair Play, Licensing and Constraint Compliance: no-external-data audit, licences, model size.

Chapter 14

Limitations, Risks and Roadmap: pairwise decoding, single operating point, French transfer.

Chapter 15

Appendices: feature list, normalisation tables, regular expressions, command-line interface.

2 Problem Statement and Evaluation Metric

This chapter states the task as the challenge defines it, fixes the file formats the pipeline must read and write, and derives the evaluation metric from first principles. The metric is a macro-average of the per-entity $F_{0.5}$ score over every Source 1 entity, with an all-or-nothing rule for entities that have no true match. Precision weighting, equal weight per entity and the singleton rule each shape one or more stages of the pipeline described in Chapters 5 to 11; the closing table maps those consequences to the functions in `er_pipeline.py` that implement them.

2.1 Formal task definition

Three independent data sources describe overlapping sets of real-world businesses. Each record carries a free-text business name, a free-text address and a country label, but no shared key. **Source 1** (S1) is designated the *deduplicated reference source*: every S1 record stands for a distinct real-world entity. **Source 2** (S2) and **Source 3** (S3) are the sources to be linked. The task is, for every S1 record, to return the set of S2 and S3 records that describe the same business. That set may be empty (the S1 entity is a *singleton*), contain one record, or contain many records, possibly from both S2 and S3. Neither S2-to-S3 nor S1-to-S1 links are requested; a self-match to an S1 identifier is a validation error.

Formally, let E be the set of S1 entities and O the union of S2 and S3 records. The ground truth is a function $g: E \rightarrow 2^O$ assigning each S1 entity a (possibly empty) subset of O ; a submission is another function $h: E \rightarrow 2^O$ on the same domain, with every element of E appearing exactly once. `load_split()` reads `g` from `train_ground_truth.tsv` into a dictionary from S1 identifier to a Python set, discarding empty tokens so that a blank list becomes the empty set.

One structural assumption follows from S1 being deduplicated: because two S1 records never denote the same business, an S2 or S3 record belongs to the ground-truth set of at most one S1 entity. The decision layer exploits this in `decode_table()`, which keeps each S2/S3 record only in the pair where it scores highest (see Chapter 9).

The country label is an *open set*. Training records are labelled `US` or `India`; test records additionally carry `France`, which never occurs in training. The rules forbid hard-coding, filtering or one-hot encoding the pipeline to the two training values and require a row for every test S1 entity, French ones included. `block()` partitions candidates by country but falls back to the full S2/S3 index for a label without counterpart (Chapter 6).

2.2 File formats

All files are tab-separated because both the address column and the identifier-list columns legitimately contain commas. The pipeline reads every file through `read_tsv()` with `sep="\t"`, `dtype=str`, `keep_default_na=False` and `quoting=3` (no quote processing), so that a quote character inside a name or an empty address survives loading unchanged. Table 2.1 lists the seven input files and the two output files; the column names are contractual, since `utils/validate_submission.py` splits the header line on tabs and requires it to equal `["source1_entity_id", list_col]` exactly.

Table 2.1 — Input and output files with exact column names

File	Split	Columns (in order)	Notes
------	-------	--------------------	-------

File	Split	Columns (in order)	Notes
train_source1.tsv, test_source1.tsv	train, test	entity_id, business_name, business_address, country	Reference source. Identifiers prefixed S1-. One row per real-world entity.
train_source2.tsv, test_source2.tsv	train, test	same four columns	Identifiers prefixed S2-. Concatenated with S3 by <code>load_split()</code> .
train_source3.tsv, test_source3.tsv	train, test	same four columns	Identifiers prefixed S3-. Source is recoverable from the prefix (<code>src</code> column in <code>prep()</code>).
train_ground_truth.tsv	train only	source1_entity_id, matched_entity_ids	Second column is a comma-separated list of S2/S3 identifiers, empty for singletons. No test counterpart is distributed.
output/matching_results.tsv	test (produced)	source1_entity_id, matched_entity_ids	The only file scored. Exactly one row per test S1 entity; empty second column for singletons; no duplicates within a list; only S2/S3 identifiers that exist in the test set.
output/candidate_pairs.tsv	test (produced)	source1_entity_id, candidate_entity_ids	Not scored. Must be the exact set the matcher scored; every identifier in <code>matching_results.tsv</code> must also appear here.

Both outputs are written by the same loop in `main()`, which iterates over `te1.entity_id` in file order and de-duplicates each list with `dict.fromkeys()`. The candidate file comes from the `(i1, i2)` table that `features()` received, so the required subset relation holds by construction (Chapter 11).

2.3 The evaluation metric

2.3.1 From F_β to $F_{0.5}$

For one S1 entity with predicted set $h(e)$ and true set $g(e)$, **precision** P is the fraction of predicted identifiers that are correct and **recall** R is the fraction of true identifiers that were predicted. The general F_β score is the weighted harmonic mean of the two:

$$F_\beta = (1 + \beta^2) \cdot P \cdot R / (\beta^2 \cdot P + R), \text{ equivalently } 1 / F_\beta = (1 / (1 + \beta^2)) \cdot (1 / P) + (\beta^2 / (1 + \beta^2)) \cdot (1 / R).$$

The parameter β expresses how many times more important recall is than precision. Setting $\beta = 0.5$ gives $\beta^2 = 0.25$ and $1 + \beta^2 = 1.25$, hence the formula printed in the problem statement:

$$F_{0.5} = 1.25 \cdot P \cdot R / (0.25 \cdot P + R).$$

In the harmonic-mean form the reciprocal of precision carries weight 0.8 and the reciprocal of recall weight 0.2. Along the diagonal $P = R$, $\partial F_{0.5} / \partial P = 0.8$ and $\partial F_{0.5} / \partial R = 0.2$, so a unit gain in precision is worth four times a unit gain in recall at a balanced operating point; the statement's "2×" phrasing refers to $\beta = 1/2$.

Substituting $P = tp / m$ and $R = tp / n$, where tp is the number of correct identifiers, $m = |h(e)|$ the number predicted and $n = |g(e)|$ the number true, the per-entity score collapses to a form that makes the asymmetry concrete:

$$F_{0.5}(e) = 1.25 \cdot tp / (m + 0.25 \cdot n).$$

Every wrong identifier adds a full unit to the denominator; every missed true identifier adds only a quarter. This is the single most important fact about the metric for the design of the decision layer.

2.3.2 Macro-averaging over S1 entities

The leaderboard score is the arithmetic mean of $F_{0.5}(e)$ over all S1 entities in the evaluation set. This is a *macro* average: an entity with twelve true matches and an entity with one contribute equally, and an entity with true matches none of which blocking paired with it contributes exactly zero however good the matcher is elsewhere (a singleton with no candidates, by contrast, scores 1.0 automatically). The pipeline reproduces the definition verbatim: `f05()` implements the per-entity score with its special cases, and `macro_f05()` averages it over the full key set `tr1.entity_id`, never over the subset that received predictions. The `stdlib scorer utils/score.py` implements the same function for independent checking (Chapter 10).

Micro-level statistics such as total true positives are therefore only diagnostics: ten extra true pairs on one heavily linked entity raise the score far less than ten zero-scoring entities turned into correct singletons.

2.3.3 The singleton rule

When $g(e)$ is empty, precision and recall are undefined, and the challenge fixes the score by rule: an empty prediction earns 1.0 and any non-empty prediction earns 0.0. Two further degenerate cases score 0.0: an empty prediction when the truth is non-empty, and a non-empty prediction with no correct identifier. `f05()` checks these three cases in that order before evaluating the formula. The rule has no middle ground: on a singleton, one spurious identifier costs the entire point, and on a single-match entity one missing identifier costs the entire point too. Whether to emit the first identifier is therefore a symmetric decision; the precision bias comes from the second and later identifiers, since adding one wrong identifier to a correct single match drops the score from 1.000 to 0.556, whereas missing a second true identifier drops it only to 0.833. The optimal threshold is consequently a property of the data (singleton fraction, true-set sizes, classifier calibration), which is why `main()` sweeps `thr` over 0.20 to 0.95 in steps of 0.025 and the relative ratio over $\{0, 0.6, 0.7, 0.8, 0.9, 1.0\}$ on out-of-fold probabilities instead of assuming 0.5.

2.3.4 Worked scenarios

Table 2.2 evaluates the per-entity score for seven prediction outcomes, including the problem statement's example (two correct identifiers plus one wrong, 0.714). Scenarios 3 and 4 are the comparison to remember: one of two true matches with nothing wrong (0.833) beats both true matches with one false merge (0.714).

Table 2.2 — Per-entity $F_{0.5}$ for representative prediction scenarios

#	Scenario	Truth $g(e)$	Prediction $h(e)$	tp / m / n	P	R	$F_{0.5}(e)$
1	Correct singleton	$\{\}$	$\{\}$	— / 0 / 0	—	—	1.000 (rule)
2	False merge on singleton	$\{\}$	$\{S2-A\}$	0 / 1 / 0	—	—	0.000 (rule)
3	One of two found, nothing wrong	$\{S2-A, S3-B\}$	$\{S2-A\}$	1 / 1 / 2	1.000	0.500	0.833
4	Both found plus one wrong (statement example)	$\{S2-00047, S3-00812\}$	$\{S2-00047, S2-00193, S3-00812\}$	2 / 3 / 2	0.667	1.000	0.714
5	All predictions wrong	$\{S2-A, S3-B\}$	$\{S2-C\}$	0 / 1 / 2	0.000	0.000	0.000 (rule)
6	Empty prediction, truth non-empty	$\{S2-A\}$	$\{\}$	0 / 0 / 1	—	0.000	0.000 (rule)

#	Scenario	Truth $g(e)$	Prediction $h(e)$	$tp / m / n$	P	R	$F_{0.5}(e)$
7	Single match found plus one wrong	{S2-A}	{S2-A, S3-C}	1 / 2 / 1	0.500	1.000	0.556

Figure 2.1 plots the per-entity score against precision for three fixed recall values. The curves are concave in P and their vertical spacing grows only slowly, the geometric statement that precision dominates: moving right along any curve gains more than moving up between curves.

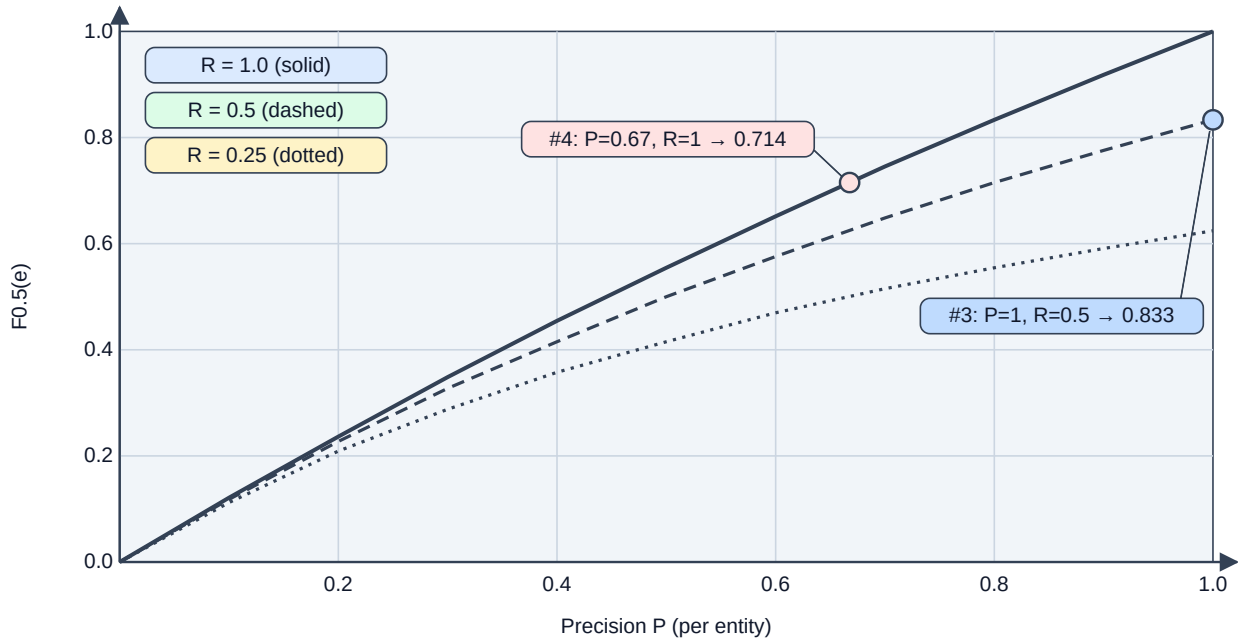


Figure 2.1 — Per-entity $F_{0.5}$ as a function of precision for recall fixed at 1.0, 0.5 and 0.25. The marked points are scenarios 4 and 3 of Table 2.2: perfect precision at half recall outscores perfect recall at two-thirds precision.

2.4 Public and private leaderboards

A submission always covers the full test set. During the challenge the public leaderboard scores an undisclosed subset of the test S1 entities; after the deadline the private leaderboard scores the remaining entities and alone determines the final ranking. Two consequences follow. First, the public score is a noisy estimate of the private one, so decisions are driven by the out-of-fold macro $F_{0.5}$ computed in `main()` on the training split, not by leaderboard probing. Second, the country mix of the two subsets is unknown, so the French share of the private set may differ from the public one; the per-country match-rate diagnostic printed at the end of `main()` exists to detect a French match rate that departs sharply from the US and Indian rates before submission (Chapter 10).

2.5 Constraints

No external lookup

External databases, entity-resolution APIs, government registries, geocoding services and internet-derived augmentation are prohibited on pain of disqualification. The pipeline reads only the seven TSV files located by `find_data_dir()`; its sole network action is the `pip install rapidfuzz` fallback on `ImportError`, which installs a library, not data (Chapter 13).

Model licence and size

The final model must carry an MIT or Apache-2.0 licence and have at most 8 billion parameters. The matcher is a LightGBM ensemble (MIT licence) whose parameters are split thresholds and leaf values: each cross-validation fold trains at most 2,000 rounds of 63-leaf trees (`num_leaves=63`) with early stopping after 100 rounds without improvement, and the final model is refitted for 1.1 times the mean best iteration of the folds. Even at the cap this is a few hundred thousand values, orders of magnitude below the limit.

Format validity

A submission that fails validation is not scored. The rules require exactly one row per test S1 entity, no duplicate rows, no duplicate identifiers within a list, S2/S3 prefixes only, and identifiers that exist in the test set. `utils/validate_submission.py` mirrors these checks locally and additionally rejects quote characters, empty or whitespace-padded list tokens, rows with other than two tab-separated columns, and, when `--candidate` is given, any matched identifier absent from the candidate list. It prints `PASS` and exits 0 only when no issue is found.

Open-set country

The country column must be treated as an unbounded set of string labels; France appears only at test time.

Reproducibility

The final package must regenerate both output files from the raw data using only the code under `code/business_entity_resolution/` with pinned dependencies.

2.6 Implications of the metric for each pipeline stage

Table 2.3 records, for every pipeline stage, the property of the metric or rules that shapes it and the mechanism in the code that responds; it is the bridge to the design chapters that follow.

Table 2.3 — Consequences of the metric and rules for each pipeline stage

Stage (chapter)	Metric or rule property	Consequence and implementing mechanism
Normalisation (5)	Open-set country; precision weighting	Language-neutral folding in <code>basic()</code> and legal-form lists that already include French forms in <code>LEGAL</code> , so French names are comparable without training data. Postal and house numbers are extracted in <code>prep()</code> as explicit disagreement evidence, which the model needs to reject near-duplicates rather than merely to find matches.
Blocking (6)	Macro-average: an entity with no blocked true pair scores 0	Recall is measured per S1 entity, not per pair: <code>main()</code> logs the number of S1 entities with at least one true candidate and the blocking-oracle score. Four unioned views in <code>block()</code> raise entity-level recall; the 0.05 cosine floor limits noise because every candidate is a potential false merge.
Pair features (7)	False merges cost more than misses	Conflict signals (<code>postal_state</code> , <code>house_state</code> , <code>num_conflict</code>) and contextual gap and rank features in <code>features()</code> let the model distinguish the best candidate from a look-alike, which is where false merges originate.
Matcher (8)	Licence and size limits; honest probabilities needed for thresholding	LightGBM (MIT, far below 8B parameters) trained with GroupKFold grouped by S1 so out-of-fold probabilities come from unseen entities.
Decision (9)	Singleton rule; each wrong identifier adds a full unit to the denominator	<code>decode_table()</code> assigns each S2/S3 record to a single S1; <code>decide()</code> applies an absolute threshold and a relative ratio; both parameters are chosen by maximising <code>macro_f05()</code> over all entities, singletons included.
Validation (10)	Macro-average; public/private split	Out-of-fold macro $F_{0.5}$ is the selection criterion; per-country and per-size breakdowns from <code>utils/score.py</code> reveal whether gains come from singletons or from multi-match entities.

Stage (chapter)	Metric or rule property	Consequence and implementing mechanism
Outputs (11)	Format rejection means score 0; matches must be a subset of candidates	One row per test entity in file order, de-duplicated lists, both files generated from the same candidate table, validated with <code>utils/validate_submission.py</code> before upload.

Risk. Data-dependent quantities (singleton fraction, selected threshold, out-of-fold score) appear in Chapters 10 and 12 as "to be filled from the run log"; the pipeline has not yet been executed on the released data and no such number is estimated in this chapter.

3 Data Understanding and Noise Taxonomy

This chapter describes the input data as the pipeline sees it: three tab-separated sources whose records share no key, an `entity_id` prefix that is the only source indicator, and free-text names and addresses in which the same business is spelled, abbreviated, truncated and reordered differently in every source. The noise patterns of the problem statement are expanded into a systematic taxonomy that pairs each pattern with the mechanism in `er_pipeline.py` that absorbs it. The chapter also sets out the address anatomy of the three countries, the open-set country rule, and the profiling output that must be inspected before any modelling decision is trusted.

3.1 Sources and What Each Contributes

3.1.1 The three sources

Every split (train and test) consists of three files, `*_source1.tsv`, `*_source2.tsv` and `*_source3.tsv`, with identical columns: `entity_id`, `business_name`, `business_address` and `country`. The problem statement designates Source 1 (S1) as the *deduplicated reference source*: each S1 record stands for exactly one real-world business, and no two S1 records stand for the same business. Sources 2 and 3 (S2, S3) are independent feeds carrying partial, noisy fragments of the same businesses; the task is to attach each S2/S3 record to the S1 entity it describes, or to leave an S1 entity as a singleton when neither feed mentions it. Because S1 is deduplicated, an S2/S3 record can belong to at most one S1 entity, a fact the decision layer exploits (see Chapter 9), whereas an S1 entity can own zero, one or many S2/S3 records.

The training split adds `train_ground_truth.tsv`, which maps each S1 `source1_entity_id` to a comma-separated `matched_entity_ids` list that is empty for singletons. `load_split()` parses it into a dictionary from S1 id to a set of matched ids; the empty string yields an empty set, so singletons are represented explicitly. The same function concatenates S2 and S3 into a single frame, because blocking and matching treat the two feeds as one candidate pool and recover the source distinction from the id prefix.

3.1.2 `entity_id` prefix semantics

There is no source column. A record's origin is encoded in the first two characters of its `entity_id`: S1-, S2- or S3-. `prep()` materialises this as the `src` column (`entity_id.str[:2]`), from which `features()` derives the binary feature `is_s3` and `decode_table()` forms its per- (S1, source) groups. The prefix also carries two hard output constraints of Chapter 2: a matched id must begin with S2- or S3- and must exist in the test set. Because the pipeline only emits ids read from the test S2/S3 files, both hold by construction (see Chapter 11).

3.1.3 Loading discipline

The files are tab-separated because addresses and id lists contain commas. `read_tsv()` reads with `sep="\t"`, `dtype=str` (postal codes must keep leading zeros), `keep_default_na=False` (an empty address is an empty string, not NaN) and `quoting=3` (`csv.QUOTE_NONE`, so a stray double quote inside a name does not swallow the rest of the line). Reading without these settings corrupts the data silently.

3.2 One Entity, Three Records

Figure 3.1 traces one fictitious but representative Indian business through the three sources and through `prep()`. Each record carries a different subset of the noise patterns of Section 3.3; the right-hand column shows the fields the pipeline compares. After normalisation all three records share the same core name, while their addresses still differ in coverage: S2 lacks the PIN code, S3 lacks the street and house number. No single field is reliable on its own, which is why blocking uses three views (see Chapter 6) and why the address features encode "unknown" as a distinct state rather than as disagreement (see Chapter 7).

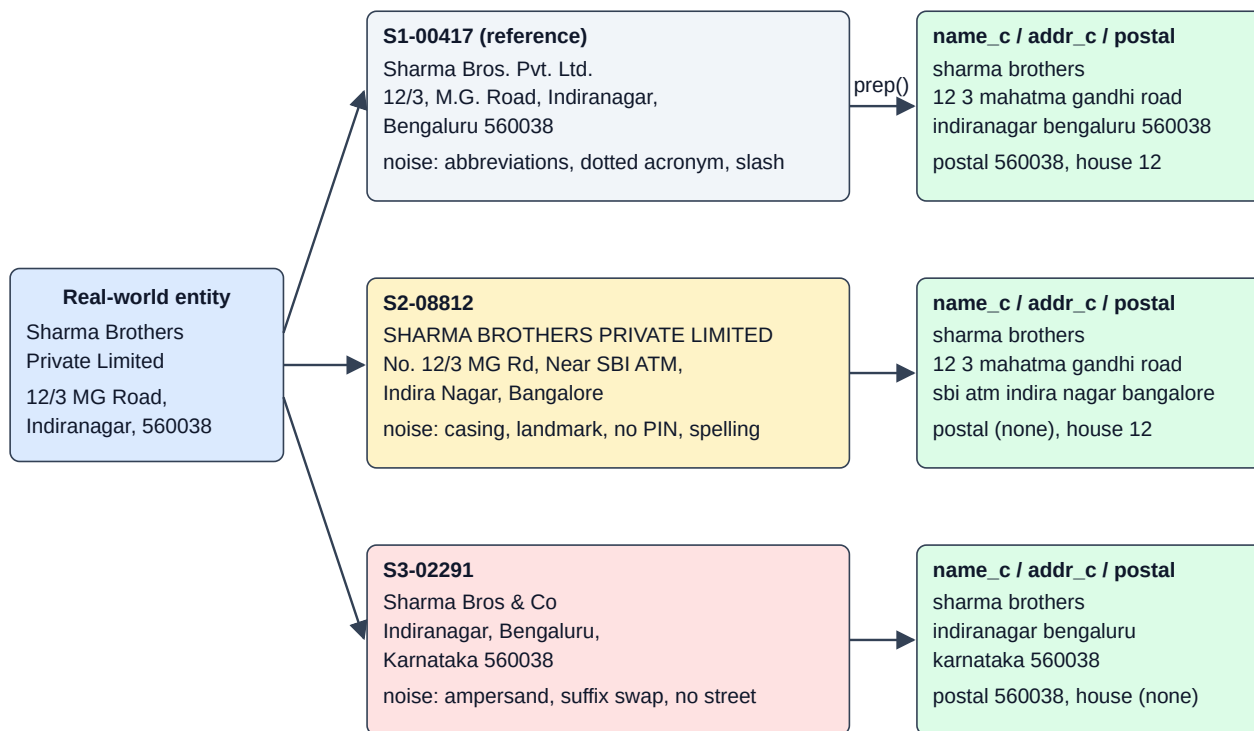


Figure 3.1 — One real-world entity fanning out into S1, S2 and S3 records, with the noise carried by each record and the fields produced by `prep()`. All three records converge on the core name `sharma brothers`; the addresses converge only partially, and the missing PIN (S2) and missing house number (S3) become explicit "unknown" states.

3.3 Noise Taxonomy

The problem statement lists the noise families informally. Table 3.1 restates them as an engineering taxonomy: each row names a pattern, gives a pair of surface forms that must be recognised as the same business, and identifies the mechanism that makes the pair comparable. Three layers recur. *Normalisation* (Chapter 5) rewrites both strings so that the difference disappears before comparison; it is the cheapest and most precise layer, but covers only what `NAME_ABBR`, `LEGAL`, `ADDR_ABBR` and `ADDR_STOP` enumerate. *Representation* (Chapter 6) uses character 2-4-grams inside word boundaries (`analyzer="char_wb"`), which degrade gracefully under typos and reordering that no table anticipates. *Features* (Chapter 7) expose the residual difference to the classifier as a number, so that the model rather than a rule decides how much a given disagreement should cost. The precision-weighted metric favours this layering: normalisation removes noise without manufacturing agreement, while the feature layer lets the model refuse a pair whenever evidence conflicts.

Table 3.1 — Noise taxonomy: pattern, example pair and absorbing mechanism. N = name field, A = address field.

#	Field	Noise pattern	Example pair	Absorbing mechanism
---	-------	---------------	--------------	---------------------

#	Field	Noise pattern	Example pair	Absorbing mechanism
1	N	Legal-form abbreviation	Acme Corp / Acme Corporation; Pvt Ltd / Private Limited	NAME_ABBR expansion in <code>norm_name()</code> , then removal of the expanded token by <code>core_name()</code> via LEGAL
2	N	Legal suffix inconsistency or omission	Acme Inc / Acme, Incorporated / Acme	<code>core_name()</code> strips every LEGAL token (falls back to the full name when nothing remains); <code>n_full_ratio</code> still compares the suffixed forms so the model can weigh a suffix clash
3	N	DBA / trade name inside one field	Acme Holdings LLC dba Acme Pizza / Acme Pizza	<code>_ALIAS</code> split on the raw string (dba, d/b/a, t/a, a.k.a., formerly known as); <code>name_parts()</code> yields variants; <code>n_alias_best</code> , <code>has_alias</code>
4	N	Punctuation and symbol variants	A&B Traders / A and B Traders; Cafe@Home	<code>basic()</code> maps & to and, @ to at, and replaces every non <code>[a-z0-9]</code> character with a space
5	N	Dotted acronyms	P.V.T. Ltd / Pvt Ltd; U.S.A. / USA	<code>_dotted</code> regex collapses runs of single letter plus dot before punctuation stripping, so the acronym survives as one token
6	N	Word-order transposition	Traders Sharma / Sharma Traders	order-free <code>n_tsort</code> (token sort) and <code>n_tset</code> (token set) ratios; <code>n_jacc</code> ; char n-gram cosine is order-insensitive
7	N	Typos and dropped letters	Shrama Brothers / Sharma Brothers	char 2-4-gram TF-IDF cosine in blocking; <code>n_ratio</code> (Indel), <code>n_jw</code> (Jaro-Winkler) as features
8	N	Casing	SHARMA BROTHERS / Sharma Brothers	<code>basic()</code> lowercases after NFKD folding
9	N	Diacritics and transliteration	Société Générale / Societe Generale; Lakshmi / Laxmi	NFKD decomposition and ASCII filter in <code>basic()</code> handle accents exactly; phonetic respellings are only partially absorbed by char n-grams and <code>n_jw</code>
10	N	Acronym versus expansion	SBI / State Bank of India	<code>n_acronym</code> (single token equal to the initials of the other core name; of already removed by <code>core_name()</code>); <code>n_init_eq</code>
11	N	Extra or missing descriptive tokens	Acme Pizza / Acme Pizza and Grill	<code>n_tset</code> and <code>n_partial</code> score the shared part at 100; <code>len_l</code> , <code>len_r</code> let the model discount short strings
12	A	Street-type abbreviation	12 Main St / 12 Main Street; Bd / Boulevard	ADDR_ABBR expansion in <code>norm_addr()</code> (rd, st, ave, blvd, bd, dr, ln, hwy, pkwy, ct, pl, sq)
13	A	Directional and unit abbreviations	100 N Main St Ste 4 / 100 North Main Street Suite 4	ADDR_ABBR (n s e w ne nw se sw, ste, apt, fl, flr, bldg)
14	A	Missing components (no PIN, no state, no street)	Indiranagar, Bengaluru 560038 / 12/3 MG Road, Indiranagar	ternary states <code>postal_state</code> and <code>house_state</code> (0 = unknown, never -1); <code>num_overlap</code> = -1 when either side has no digits; <code>a_partial</code> scores the shared substring
15	A	Landmark-based references	Near SBI ATM, MG Road / MG Road	ADDR_STOP removes near, opposite, behind, beside, next, to (as separate tokens) in <code>addr_c</code> ; residual landmark tokens are tolerated by <code>a_tset</code> ; nr/opp expanded first by ADDR_ABBR
16	A	Municipal numbering formats	No. 12/3 / 12-3 / Plot 12, Door 3	<code>basic()</code> turns / and - into spaces; <code>_digits</code> extracts the run set {12, 3} in <code>_addr_nums()</code> ; no/number are ADDR_STOP words; <code>num_overlap</code> , <code>num_conflict</code> , <code>house_state</code>
17	A	Component reordering	Bengaluru, Indiranagar, MG Road / MG Road, Indiranagar, Bengaluru	<code>a_tset</code> , <code>a_jacc</code> ; the <code>addr</code> and full blocking views are bag-of-n-gram and therefore order-free

#	Field	Noise pattern	Example pair	Absorbing mechanism
18	A	Transliteration and idiom variants	Indira Ngr / Indiranagar; Anna Salai / Anna Road; Gandhi Marg	ADDR_ABBR maps ngr to nagar, marg and salai to road, mg to mahatma gandhi, clny, sec, mkt, stn; the spacing variant indira nagar / indiranagar is left to char n-grams
19	A	ZIP+4 versus ZIP; PIN written with an internal space	Austin TX 78701-2345 / Austin TX 78701; 560 038 / 560038	<code>_postal = \b(\d{5,6}) (?:-\d{4})?\b</code> captures only the 5- or 6-digit stem, so ZIP+4 is absorbed; the last match on the raw address wins. A spaced PIN (560 038) is <i>not</i> matched: <code>postal</code> is empty and <code>postal_state</code> falls to 0 (unknown), which is the safe direction under $F_{0.5}$
20	A	French articles and street types	12 Av. de la Paix / 12 Avenue de la Paix; Bd Hausmann / Boulevard Hausmann; Fbg Saint- Honoré	ADDR_STOP removes de la le du des; ADDR_ABBR expands av, bd, fbg, chem, imp; accents folded by NFKD. A bare r. for rue is not in the table and stays as the token r
21	N/A	Same business, no common tokens at all	trade name on one side, legal name on the other, addresses in different scripts	Not absorbed by any rule; recovered only if the same-postal blocking view finds the pair and the model trusts address evidence. Counted in the blocking recall ceiling (Chapter 6)

Design note. The same token can require different expansions in the two fields. `ste` is `societe` in `NAME_ABBR` (an accent-stripped `Sté`) but `suite` in `ADDR_ABBR`; `st` is expanded to `street` only in addresses, so a name such as `St Mary Hospital` is not rewritten. Keeping two tables rather than one is what makes the expansions safe, and safety matters more than coverage under $F_{0.5}$: a wrong expansion that manufactures agreement is a false-merge generator.

3.4 Country-Specific Address Anatomy

The three countries differ in both component order and numbering conventions. The pipeline does not parse addresses into components; it relies on a few country-agnostic extractions (postal code, digit runs, leading number) and on token-level normalisation. Table 3.2 records the canonical address of each country, the components the data is expected to drop or reorder, and how each extraction behaves on that anatomy.

Table 3.2 — Address anatomy by country and the behaviour of the pipeline's address extractions on each.

Aspect	United States	India	France (test only)
Canonical order	number, street name, street type, [unit], city, state, ZIP	[plot/door/shop number], [building], street or landmark, area (nagar, colony, sector), city, [district], state, PIN	numéro, type de voie, nom de voie, [complément], code postal, commune
Building numbering	Single integer, occasionally with a unit (Ste 200, Apt 4B); 5-digit house numbers exist	Compound forms with slashes or hyphens (12/3, 4-1-23), Plot No., Door No., Shop No., floor prefixes (1st Floor)	Single integer, often with bis/ter suffix (12 bis)
Postal code	5-digit ZIP, optional -NNNN (ZIP+4); <code>_postal</code> keeps the 5-digit stem	6-digit PIN, sometimes written with a space (560 038), which <code>_postal</code> does not match; frequently missing	5-digit code postal whose first two digits are the département; matched by the same <code>\d{5,6}</code> pattern

Aspect	United States	India	France (test only)
Region unit	Two-letter state (TX) or full name; not expanded by the pipeline, except that CT, FL and NE collide with ADDR_ABBR and become court, floor, northeast	State name, often omitted; city spelling variants (Bangalore/Bengaluru, Bombay/Mumbai) are not mapped	Commune name; region rarely present; Cedex suffix possible
Characteristic idioms	N/S/E/W, Hwy, Pkwy, Blvd, Ste	Nr, Opp, Behind, MG Rd, Ngr, Clny, Sec, Mkt, Chowk, Marg, Salai	rue, av, bd, fbg, chem, imp, articles de la le du des, accents
Legal forms in names	Inc, LLC, LLP, LP, Corp, Co, PLLC, DBA	Pvt Ltd, Private Limited, LLP, OPC, & Co (all stripped by LEGAL); Bros is expanded to brothers by NAME_ABBR but kept, since it is part of the trading name	SARL, SAS, SASU, SA, EURL, SCI, SNC, Société, Cie, et
house extraction	Leading digit run = house number; reliable	Leading digit run = first part of the plot number, or a floor number when the floor is written first; moderately reliable	Leading digit run = numéro; bis is lost, which is harmless
nums extraction	{house, unit numbers}; ZIP removed	{plot parts, door, floor, shop}; PIN removed unless it is the leading digit run and the address does not end with it	{numéro}; code postal removed

Two details of `_addr_nums()` follow from this anatomy. First, the postal code is removed from the digit-run set so that two US records in the same ZIP do not appear to share a street number; `num_overlap` and `num_conflict` would otherwise be polluted by the most widely shared number in the data. Second, the removal is suppressed when the postal match is the leading number and does not end the string, because a 5-digit US house number with the ZIP omitted also matches `_postal` and must remain available as the house number. Both rules are precision measures: a spurious numeric agreement is exactly the evidence that pushes a non-match over the threshold.

A related consequence of token-level expansion is that `expand()` has no notion of position. The US state codes CT, FL and NE are rewritten by ADDR_ABBR to `court`, `floor` and `northeast` wherever they occur, and an Indian `Sec 5` becomes `sector 5` even when it abbreviates something else. These rewrites are consistent: both records of a true pair receive the same expansion, so pair similarity is unaffected, and a token shared by every record in a state carries little TF-IDF weight. The cost is confined to the rare non-pair in which an expansion manufactures a shared token.

The Indian anatomy is the hardest. The same building can be written `12/3`, `No. 12/3`, `12-3` or `Door 3`, `Plot 12`; after `basic()` all four yield the digit set {12, 3}, but only the first three yield `house = 12`, so `house_state` can report a false disagreement while `num_overlap` reports full agreement. The classifier sees both signals and learns which to trust (see Chapter 8). City-name transliterations (Bangalore versus Bengaluru) are deliberately not mapped: a gazetteer would be an external resource, and the same-postal blocking view already places both spellings in one block whenever the PIN is present.

3.5 The Open-Set Country Requirement

The training data cover US and India; the test data additionally contain France, which appears in no training record. The statement forbids hard-coding, filtering or one-hot-encoding the country to the training set, and requires every test S1 entity, French ones included, to appear in the output. The pipeline complies in three ways. The `country` value is never a feature: `features()` derives no column from it, so the model cannot learn a rule that France violates. The value is used only as a partition key in `block()`, where `country_n =`

`basic(country)` is compared as a plain string; any label, including an unseen or blank one, forms its own group. Finally, when an S1 group's label does not occur among the S2/S3 records, `block()` compares that group against every S2/S3 record instead of an empty block, so a spelling variant such as `USA` against `US`, or a missing label, degrades to slower blocking rather than to a silently dropped entity.

Risk: unseen-country drift. Compliance with the rule does not remove the statistical problem. The classifier and the decision parameters (`thr`, `ratio`) are fitted on US and Indian pairs only (see Chapters 8 and 9). French names carry longer legal forms and articles, French addresses put the postal code before the commune, and the TF-IDF vocabulary for French text is learned only at test time when the vectorisers are refitted. The similarity distribution of true French pairs may therefore sit above or below what the tuned threshold assumes. Under $F_{0.5}$ the consequence is asymmetric: a threshold too low for France produces false merges on French singletons at a cost of 1.0 each, whereas a threshold too high loses only part of a point per missed match. The only in-pipeline guard is the per-country match rate printed at the end of `main()`; a French rate far above the seen countries' is the signature of over-matching. Chapter 14 discusses a pseudo-unseen-country validation (train on US, validate on India) that would quantify the gap.

3.6 Profiling the Data: What profile() Logs

Before any normalisation, `main()` calls `profile()` once per split. The function is minimal, but every line it prints answers a question that a later design choice depends on. Table 3.3 lists the lines and the decision each informs; the values are to be filled from the run log, since the pipeline has not yet been executed on the released data.

Table 3.3 — Output of `profile()` and how each line is read. Values are to be filled from the run log.

Log line	Meaning	How to read it	Train	Test
S1=... S2+S3=...	Record counts per split	Their product is the pair space; with the per-S1 candidate count it gives the reduction ratio (Chapter 6)	to be filled from the run log	to be filled from the run log
S1 countries: {...}	<code>value_counts()</code> of the raw S1 country label	Confirms the label spelling used for partitioning; a variant such as <code>usa</code> would show up as an extra key	to be filled from the run log	to be filled from the run log
S2/S3 countries: {...}	Same for the concatenated S2/S3 pool	Any S1 key absent here triggers the open-set fallback; France must be present in test for both lines	to be filled from the run log	to be filled from the run log
singletons: ...	Fraction of S1 entities with an empty ground-truth list	The macro $F_{0.5}$ of an all-empty submission; every match the model emits must beat this baseline	to be filled from the run log	n/a (no labels)
mean matches: ... max: ...	Mean and maximum ground-truth list length	The blocking top-k per view must exceed the maximum; the mean bounds the recall a singleton-heavy threshold may sacrifice	to be filled from the run log	n/a (no labels)
head(3) of S1 and S2/S3	First three raw rows of each frame	Verifies that tabs were parsed into four columns and exposes the dominant surface noise at a glance	to be filled from the run log	to be filled from the run log

The singleton fraction is the most consequential line. Because a correctly predicted singleton scores 1.0 and a wrongly matched one scores 0.0, the fraction sets the floor that any model must exceed and defines how much of the macro average is at stake in the threshold choice; this is why the sweep in Chapter 9 scores singletons together with matched entities rather than tuning on matched entities alone. The country dictionaries are checked next: a label present in S1 but absent from S2/S3 is not an error but a signal that the fallback path, with its larger blocks, will run for that group. The maximum list length must be compared with `k_name = 25`, `k_full = 25`,

`k_addr = 10` and `k_postal = 10`: an S1 entity with more true records than the union of the views can return is capped in recall by construction, and the blocking recall ceiling logged later in `main()` (see Chapter 6) is where that cap becomes measurable.

4 System Architecture Overview

This chapter describes the solution as a software system: its architectural style, the modules of `er_pipeline.py` and the responsibility of each function, the data contracts between stages, the control flow of `main()` for the training and test splits, the runtime environment with its dependencies and licences, the logging design, and the configuration surface. Chapters 5 to 11 go deeper into each stage; this chapter fixes the vocabulary and interfaces they rely on.

4.1 Architectural style

The system is a *single-process batch pipeline*: one Python module, `er_pipeline.py`, with one entry point, `main()`, that starts from the raw TSV files and ends with the two submission files. There are no services, no state between runs and no intermediate files. The organising principle is a chain of *pure functions* over two in-memory containers: pandas DataFrames for record-level and pair-level tables, and SciPy compressed-sparse-row (CSR) matrices for the TF-IDF (term frequency–inverse document frequency) representations used in blocking. Each stage takes frames in and returns a new frame out: `prep()` copies its input before adding columns, `block()` and `features()` build fresh frames, and `decode_table()` copies only the three columns it needs. No function mutates module-level state; the only globals are the lookup tables (`NAME_ABBR`, `LEGAL`, `ADDR_ABBR`, `ADDR_STOP`), the compiled regular expressions and the start timestamp `T0` that `log()` reads.

Three considerations tie this style to the evaluation. First, the metric is a macro-average of per-entity $F_{0.5}$ with an all-or-nothing rule for singletons (see Chapter 2), so the decision parameters must be tuned on out-of-fold predictions computed from exactly the features the test path will see; a single linear flow guarantees this because both splits pass through the identical `prep()` → `block()` → `features()` sequence and the column list `feats` is derived once and reused. Second, the corpus is expected to fit in the memory of a Kaggle CPU kernel, so a scheduler or checkpointing layer would add failure modes without capability; the one memory-sensitive operation, the dense chunk inside `topk_sparse()`, is bounded by construction (see Chapter 12). Third, the fair-play rules require a reviewer to verify that no external lookup occurs, and a single file whose only network action is an optional `pip install rapidfuzz` fallback is the easiest artefact to audit (see Chapter 13).

A convention that runs through every stage is *positional row identity*. After `prep()` resets the index, every record is addressed by its integer position; candidate, feature and decode rows carry the pair `(i1, i2)`, where `i1` indexes the S1 frame and `i2` the concatenated S2+S3 frame, and entity identifiers are re-attached only at the edges: by `to_lists()` on output and by the labelling and recall-diagnostic lines of `main()`, which index `tr1.entity_id.values` and `tr2.entity_id.values` by position. This keeps the look-ups in `features()` and the group-by operations unambiguous and lets `GroupKFold` group by `i1` so that all candidates of one S1 entity fall into the same fold (see Chapter 10).

4.2 Module map

Figure 4.1 arranges the functions in the five lanes that correspond to the section headers of the source, with `main()` as orchestrator underneath. Data flow left to right through the lanes; control flows from `main()` upward into each lane.

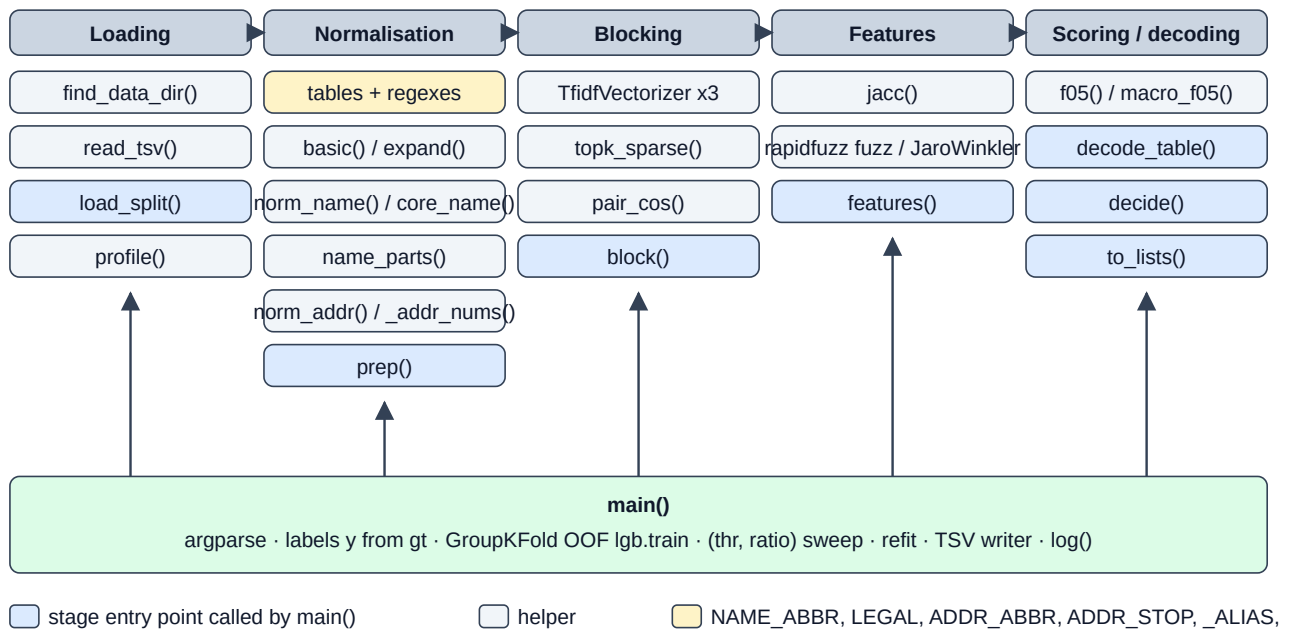


Figure 4.1 — Component map of `er_pipeline.py`: functions grouped by the five source sections, with `main()` orchestrating every lane. Blue boxes are the entry points that `main()` calls directly; grey boxes are helpers used inside a lane.

Loading. `find_data_dir()` searches recursively for `train_source1.tsv` under `/kaggle/input`, the working directory, its parent and `dataset/`, and returns the grandparent of the first hit so that one directory holds `train/` and `test/`. `read_tsv()` reads with `dtype=str`, `keep_default_na=False` and `quoting=3`, so blank fields stay empty strings, a name such as "NA" is not turned into a missing value, and a stray quotation mark cannot swallow the following rows. `load_split()` reads the three source files, concatenates S2 and S3 and, when present, parses the ground truth into a dictionary from S1 identifier to set of matched identifiers. `profile()` logs corpus statistics.

Normalisation. `basic()`, `expand()`, `norm_name()`, `core_name()`, `name_parts()`, `norm_addr()` and `_addr_nums()` are string-level helpers driven by the four lookup tables and the regular expressions `_punct`, `_space`, `_digits`, `_postal`, `_dotted` and `_ALIAS`; `prep()` applies them column-wise (see Chapter 5).

Blocking. `topk_sparse()` returns, for each row of one L2-normalised sparse matrix, the indices and cosine scores of its top-k rows in another, in chunks sized from `max_cells`; `pair_cos()` computes the cosine of listed row pairs in slices; `block()` fits three `TfidfVectorizer` instances and unions four candidate views (see Chapter 6).

Features. `jacc()` is the token Jaccard index; `features()` computes every pairwise and contextual feature over the candidate frame with `rapidfuzz` scorers (see Chapter 7).

Scoring and decoding. `f05()` is the per-entity metric including the singleton rule, `macro_f05()` averages it, and `decode_table()`, `decide()` and `to_lists()` turn probabilities into identifier lists (see Chapter 9). Argument parsing, labelling, LightGBM training under `GroupKFold`, the `(thr, ratio)` sweep, the refit and the TSV writer live in `main()` (see Chapters 8, 10 and 11).

4.3 Data contracts between stages

Table 4.1 states what each stage consumes and adds. Contracts are enforced by column names rather than types; the two collection-valued columns (`name_alts`, a list of alias variants, and `nums`, a `frozenset` of digit runs) are consumed by `features()` and never reach the model.

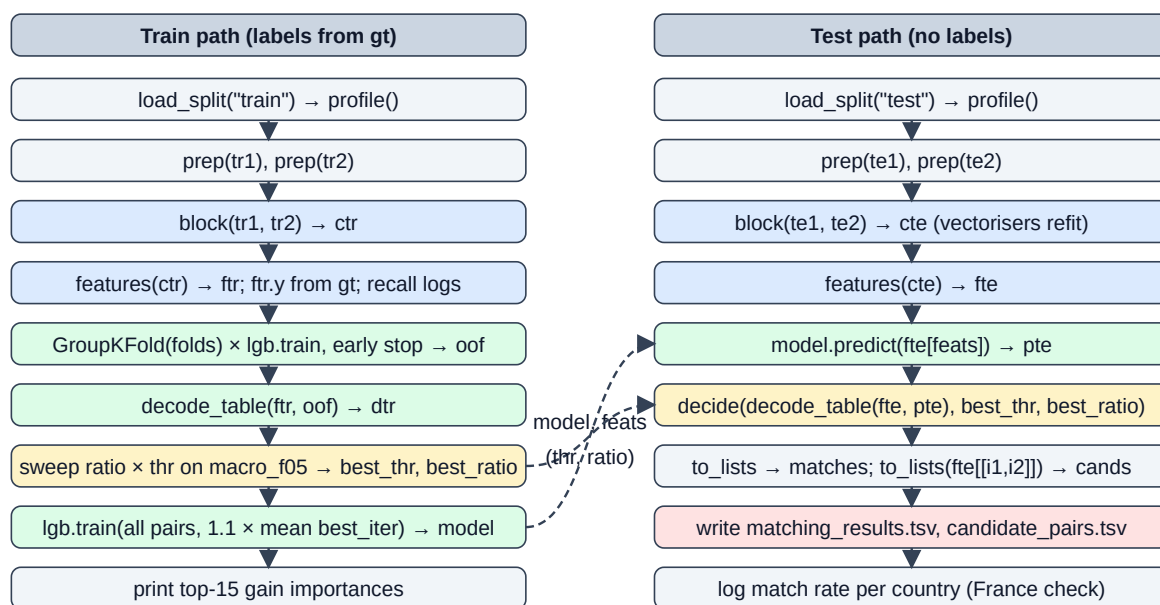
Table 4.1 — Stage-by-stage data contract: what each function requires from its input and what it produces.

Stage / function	Input	Output
Load: <code>read_tsv()</code> , <code>load_split()</code>	The three source TSVs of a split, plus <code>{split}_ground_truth.tsv</code> when it exists (train only)	<code>s1</code> and <code>other</code> (<code>S2</code> followed by <code>S3</code> , <code>ignore_index=True</code>) with columns <code>entity_id</code> , <code>business_name</code> , <code>business_address</code> , <code>country</code> , <code>all_str</code> ; <code>gt</code> : dict <code>S1 id</code> $\rightarrow$ set of ids, or <code>None</code>
Normalise: <code>prep()</code>	The four raw columns	Same rows, index reset to <code>0...n-1</code> , plus <code>name_n</code> , <code>name_c</code> , <code>name_alts</code> (list), <code>addr_n</code> , <code>addr_c</code> , <code>postal</code> , <code>nums</code> (<code>frozenset</code>), <code>house</code> , <code>country_n</code> , <code>src</code> (first two characters of <code>entity_id</code>)
Block: <code>block()</code>	<code>name_c</code> , <code>addr_c</code> , <code>country_n</code> , <code>postal</code> of both frames	Candidate frame <code>cand</code> : <code>i1</code> , <code>i2</code> (positions, deduplicated), <code>cos_name</code> , <code>cos_full</code> , <code>cos_addr</code> (<code>float32</code>)
Features: <code>features()</code>	<code>cand</code> plus the prepared frames (looked up by <code>.loc[i1]</code> , <code>.loc[i2]</code>)	Feature frame <code>f</code> : the five candidate columns plus 35 feature columns, 38 model inputs in total; on <code>train_main()</code> appends the label <code>y</code>
Model: <code>lgb.train()</code> / <code>predict()</code>	<code>f[feats]</code> , where <code>feats</code> = every column except <code>i1</code> , <code>i2</code> , <code>y</code>	Probability vector aligned with the rows of <code>f</code> (oof on train, pte on test)
Decode: <code>decode_table()</code>	<code>f[["i1", "i2", "is_s3"]]</code> and the probability vector	Decode table <code>d</code> : <code>i1</code> , <code>i2</code> , <code>is_s3</code> , <code>p</code> , <code>best</code> ; exactly one row per <code>i2</code> that occurs in <code>f</code> (the highest-probability <code>S1</code> for that record)
Decide: <code>decide()</code>	<code>d</code> , <code>thr</code> , <code>ratio</code>	Row subset of <code>d</code> with <code>p</code> $\geq$ <code>thr</code> and <code>p</code> $\geq$ <code>ratio</code> $\cdot$ <code>best</code>
Lists: <code>to_lists()</code>	Any frame with <code>i1</code> and an id column, plus <code>s1</code> , <code>other</code>	Dict <code>S1 entity_id</code> $\rightarrow$ list of <code>S2/S3</code> ids, with a key for every <code>S1</code> record (empty list = singleton)
Write: <code>main()</code>	The matches and <code>cands</code> dicts	<code>matching_results.tsv</code> , <code>candidate_pairs.tsv</code> : one row per test <code>S1</code> id, comma-joined ids deduplicated by <code>dict.fromkeys()</code>

Three invariants follow. Because `other` concatenates `S2` and `S3`, one `i2` index space covers both sources and the `src` column, from which `is_s3` is derived, tells them apart. Because `feats` is reused verbatim on the test frame, the column order emitted by `features()` is part of the contract. Because `to_lists()` initialises an entry for every `S1` identifier before filling in pairs, an entity with no surviving pair is emitted as an empty row rather than omitted, which the singleton rule and the validator both require.

4.4 Control flow of `main()`

Figure 4.2 shows the two paths `main()` executes in sequence. Both splits are loaded and profiled first, so a missing test file fails before any training time is spent.



Both splits are loaded and profiled before the train path starts; the test path runs after the refit.

Figure 4.2 — Control flow of `main()`. The left column is executed on the training split, the right column on the test split; dashed arrows carry the three artefacts that cross between them (the fitted model, the feature list and the tuned decision parameters).

On the training path the label `y` of a candidate pair is 1 when the S2/S3 identifier belongs to the ground-truth set of the S1 identifier and 0 otherwise; pairs that blocking never produces are neither positives nor negatives and appear only in the recall diagnostics. `GroupKFold(args.folds)` with `groups=ftr.i1` yields out-of-fold (OOF) probabilities: a probability for every training pair from a model that never saw that pair's S1 entity. The decision parameters are chosen by exhaustive search on the OOF probabilities, and the final model is refit on all pairs with 1.1 times the mean early-stopped iteration count.

On the test path no label is read. `block()` refits its vectorisers on the test texts, so the n-gram vocabulary of the unseen country is learned from the test records themselves. Matches and the full candidate frame are converted to identifier lists and written in the same loop, so the matches are a subset of the candidates by construction.

Design note. The frame `fte` that feeds `model.predict()` is the one written as `candidate_pairs.tsv`, so the file documents the exact set the model scored, as the organisers define it.

4.5 Runtime environment and dependencies

The intended host is a Kaggle CPU kernel: a Linux container with a fixed Python 3 image in which the libraries below are normally preinstalled (the `rapidfuzz` import is the only one guarded by an install fallback), the dataset mounted read-only under `/kaggle/input` and outputs written under `/kaggle/working`. The script also runs from a shell with `--data-dir`. No GPU is used; LightGBM parallelises over all cores through `n_jobs=-1`, while the `rapidfuzz` loops in `features()` run in one Python thread and dominate wall-clock time (see Chapter 12). Table 4.2 lists the dependencies as pinned in `requirements.txt`.

Table 4.2 — Runtime dependencies, their role in the pipeline and their licences.

Package (minimum version)	Role in <code>er_pipeline.py</code>	Licence
---------------------------	-------------------------------------	---------

Package (minimum version)	Role in <code>er_pipeline.py</code>	Licence
<code>numpy</code> $\geq$ 1.24	Dense chunk scoring and <code>argpartition</code> in <code>topk_sparse()</code> , ternary state columns, the OOF vector, the threshold grid	BSD-3-Clause
<code>pandas</code> $\geq$ 2.0	All record and pair tables; TSV reading; group-by transforms for the contextual features and for <code>decode_table()</code>	BSD-3-Clause
<code>scipy</code> $\geq$ 1.10	CSR sparse matrices returned by the vectorisers; sparse products (<code>@</code> , <code>.T.tocsr()</code> , <code>.multiply()</code>) in blocking	BSD-3-Clause
<code>scikit-learn</code> $\geq$ 1.3	<code>TfidfVectorizer</code> (<code>char_wb</code> 2–4-grams) and <code>GroupKFold</code>	BSD-3-Clause
<code>lightgbm</code> $\geq$ 4.0	The matching model: <code>lgb.train</code> , <code>lgb.Dataset</code> , <code>lgb.early_stopping</code> , gain importances	MIT
<code>rapidfuzz</code> $\geq$ 3.0	<code>fuzz.ratio</code> , <code>token_sort_ratio</code> , <code>token_set_ratio</code> , <code>partial_ratio</code> , <code>JaroWinkler.similarity</code> in <code>features()</code>	MIT
Python standard library	<code>argparse</code> , <code>os</code> , <code>re</code> , <code>sys</code> , <code>time</code> , <code>unicodedata</code> , <code>glob</code>	PSF

The licence constraint applies to the *final model*, which must be MIT or Apache-2.0 licensed and at most eight billion parameters. The model is a LightGBM gradient-boosted tree ensemble, MIT licensed, of at most 63 leaves per tree and at most 2,200 trees (the refit grows `int(mean(best_iter) × 1.1)` rounds, and each fold's `best_iteration` is capped at 2,000); the BSD-licensed libraries are tooling, not models, and no pretrained weights are loaded. SciPy is not imported by name but enters through the sparse matrices scikit-learn returns, hence its pin in `requirements.txt`. The `rapidfuzz` import falls back to a `pip` install when the package is absent, the only network action the script can take, and it fetches a library, never data (see Chapter 13).

4.6 Logging design

All progress output goes through `log()`, which prefixes each message with the wall-clock seconds elapsed since import (`T0`), one decimal in a seven-character field, and flushes immediately so the Kaggle console shows lines as they happen. The prefix turns the log into a coarse profiler: the difference between consecutive lines is the cost of the stage between them. Multi-line frames use bare `print()`. Table 4.3 lists every line in emission order; the value column is a placeholder until the pipeline is run on the real data.

```
def log(msg):
    print(f"[{time.time() - T0:7.1f}s] {msg}", flush=True)
```

Table 4.3 — Every log line emitted by a full run, in order, with its diagnostic purpose. Values are to be filled from the run log.

Line (template)	Emitted by	Diagnoses	Value
<code>data dir: ...</code>	<code>main()</code>	Which directory auto-detection or <code>--data-dir</code> resolved to	to be filled from the run log
<code>== train: S1=... S2+S3=..., S1 countries: {...}, S2/S3 countries: {...}, singletons: ... mean matches: ... max: ..., then three sample rows per frame</code>	<code>profile("train")</code>	Corpus size, country mix, singleton fraction and match multiplicity that set the top-k budgets and the singleton stakes in the metric	to be filled from the run log

Line (template)	Emitted by	Diagnoses	Value
<code>== test: ... with the same country lines (no singleton line)</code>	<code>profile("test")</code>	Presence and size of the unseen country; source imbalance	to be filled from the run log
<code>blocking: N pairs (x per S1)</code>	<code>block()</code> on train	Candidate volume and reduction ratio	to be filled from the run log
<code>blocking recall ceiling: r (pos=... / true=...)</code>	<code>main()</code>	Fraction of ground-truth links that survive blocking; upper bound of pair recall	to be filled from the run log
<code>S1 entities with >=1 true match blocked: a/b</code>	<code>main()</code>	Entity-level ceiling: every non-blocked entity is a guaranteed 0.0 in the macro average	to be filled from the run log
<code>blocking recall by country: {...}</code>	<code>main()</code>	Whether one country's noise defeats the views	to be filled from the run log
<code>blocking recall by source: {...}</code>	<code>main()</code>	Whether S2 or S3 is harder to recall	to be filled from the run log
<code>fold k: best_iter=... (one per fold)</code>	<code>main()</code>	Early-stopping point per fold; stability of the fit	to be filled from the run log
<code>00F macro F0.5 = s at thr=... ratio=...</code>	<code>main()</code>	The validation score and the selected operating point	to be filled from the run log
<code>macro F0.5 if every candidate were predicted (no model; floor): ...</code>	<code>main()</code>	Score without a matcher; what the model must beat	to be filled from the run log
<code>blocking oracle macro F0.5 (...; ceiling): ...</code>	<code>main()</code>	Score of a perfect matcher on the blocked pairs; gap to OOF is the matcher's loss	to be filled from the run log
<code>Top-15 gain importances (frame)</code>	<code>print()</code>	Which feature families the model relies on	to be filled from the run log
<code>blocking: N pairs (x per S1)</code>	<code>block()</code> on test	Test candidate volume, comparable with train	to be filled from the run log
<code>wrote ../matching_results.tsv, wrote ../candidate_pairs.tsv</code>	<code>main()</code>	Output paths	to be filled from the run log

Line (template)	Emitted by	Diagnoses	Value
test: n/N S1 entities matched; by country: {...}	main()	Match rate per raw country label; sanity check that France is neither starved nor over-matched	to be filled from the run log

The floor, ceiling and OOF lines are read together: the gap between the oracle ceiling and 1.0 is loss only a change in `block()` can recover, and the gap between the ceiling and the OOF score is loss attributable to the matcher and the decision rule. Because $F_{0.5}$ is precision-weighted and singletons score all-or-nothing, the floor is expected to be low even when recall is high.

4.7 Configuration surface

The pipeline exposes three command-line arguments and otherwise reads its configuration from literals in the source. `parse_known_args()` tolerates the `-f` argument Jupyter injects inside a notebook. Table 4.4 enumerates every tunable value with its location and is the reference for the sensitivity discussion in Chapter 14.

Table 4.4 — Configuration surface: command-line arguments and in-source constants.

Item	Where	Value
<code>--data-dir</code>	CLI	Directory containing train/ and test/; auto-detected by <code>find_data_dir()</code> when omitted
<code>--out-dir</code>	CLI	Output directory, default output, created if missing
<code>--folds</code>	CLI	Number of GroupKFold splits, default 5
Top-k per view	<code>block()</code> defaults	<code>k_name=25, k_full=25, k_addr=10, k_postal=10</code>
Cosine cutoff	<code>block()</code>	Pairs kept only if score > 0.05
TF-IDF	<code>block()</code>	<code>analyzer="char_wb", ngram_range=(2, 4), min_df=2, sublinear_tf=True, dtype=float32</code>
Memory bounds	<code>topk_sparse()</code> , <code>pair_cos()</code>	<code>max_cells=2e7 (chunk = max(16, max_cells // B)); chunk=200_000 pairs</code>
LightGBM params	<code>main()</code>	<code>objective="binary", learning_rate=0.05, num_leaves=63, min_child_samples=40, feature_fraction=0.8, bagging_fraction=0.8, bagging_freq=1, lambda_l2=1.0, verbose=-1, n_jobs=-1</code>
Boosting budget	<code>main()</code>	2000 rounds, early stopping patience 100 on the held-out fold; refit with <code>int(mean(best_iter) × 1.1)</code>
Decision grid	<code>main()</code>	<code>ratio ∈ {0.0, 0.6, 0.7, 0.8, 0.9, 1.0}; thr ∈ arange(0.20, 0.96, 0.025)</code> , i.e. 31 values from 0.200 to 0.950; 186 evaluations of <code>macro_f05()</code> ; initial best (0.5, 0.0)
Lookup tables and regexes	Module level	<code>NAME_ABBR</code> (18 entries), <code>LEGAL</code> (26 tokens), <code>ADDR_ABBR</code> (41 entries), <code>ADDR_STOP</code> (16 tokens), <code>_postal</code> , <code>_dotted</code> , <code>_ALIAS</code>

Two values are deliberately *not* configuration: (`best_thr`, `best_ratio`) and the refit round count are learned in every run and never stored, so a reproduction cannot reuse stale parameters. `params` sets no explicit `seed` and no `deterministic` flag: LightGBM falls back to its built-in default seeds for bagging and feature subsampling, which keeps a single-threaded run repeatable, but `n_jobs=-1` lets histogram construction be split across threads in an order that can vary between machines and builds, so repeated runs may differ in the last decimals of the OOF score; Chapter 11 gives the `params` additions that make a run bit-exact.

Risk. Because every constant is a literal in the source, a change to one view's `k` or to the grid range is invisible from the CLI. Any tuning must be recorded in the run log and in Chapter 15, and the ceiling and floor lines of Table 4.3 re-read after each change: a higher `k` can only raise the ceiling and lower the floor, and under the precision-weighted metric it is expected to move the selected `thr` as well, so the sweep must be re-run rather than the previous operating point reused.

5 Text Normalisation Layer

This chapter specifies the normalisation layer implemented by `prep()` in `er_pipeline.py`: the deterministic rewriting of every raw `business_name`, `business_address` and `country` value into the canonical columns that blocking (see Chapter 6) and pair feature engineering (see Chapter 7) consume. `basic()` operates on characters, `expand()` on tokens against four fixed vocabularies (`NAME_ABBR`, `LEGAL`, `ADDR_ABBR`, `ADDR_STOP`), and a final tier extracts structured signals that survive text noise: alias variants, postal code, street numbers and house number. Every rule exists to make two spellings of the same business collide while leaving genuine differences visible, the balance the precision-weighted $F_{0.5}$ metric demands.

5.1 Role of Normalisation in a Precision-Weighted Pipeline

Normalisation is applied identically to Source 1 (S1) and to the concatenated Source 2/3 (S2/S3) frames, on both splits, before any comparison is made. The *core name* without legal forms drives the TF-IDF blocking views and most name similarities; the *full normalised name* keeps the legal form so the matcher can still see that `acme incorporated` and `acme llc` disagree on a token that sometimes separates two businesses. Under $F_{0.5}$ an over-aggressive canonical form raises blocking recall but hides evidence of a false merge, which on a singleton costs a full point (see Chapter 2); the layer therefore canonicalises for recall in the blocking columns and preserves residual detail in parallel columns for precision. All tables are static literals in the source; no geocoder, gazetteer or registry is consulted (see Chapter 13).

5.2 Character Tier: `basic()`

`basic(s)` is the entry point for every text field, including the country label. It applies five operations in a fixed order; the order is load-bearing, as the dotted-acronym rule must run before punctuation is destroyed.

1. **Unicode folding.** `unicodedata.normalize("NFKD", str(s))` decomposes precomposed characters into base letter plus combining mark and maps compatibility forms (full-width Latin, ligatures) to canonical equivalents; `.encode("ascii", "ignore").decode()` discards every non-ASCII code point, so accents vanish (`Société` → `Societe`); the result is lower-cased.
2. **Symbol words.** `&` becomes `and` and `@` becomes `at`, so `A&B Traders` and `A and B Traders` share a token stream.
3. **Dotted acronyms.** `_dotted` collapses `u.s.a` into `usa` and `p.v.t.` into `pvt` (Section 5.2.1).
4. **Punctuation stripping.** `_punct = [^a-z0-9 ]+` replaces every run of characters outside lower-case letters, digits and space with one space, so commas, hyphens, apostrophes, slashes and remaining dots become separators.
5. **Whitespace.** `_space = \s+` collapses runs; the result is stripped.

5.2.1 The `_dotted` regex

```
_dotted = (?<![a-z])(?:[a-z]\.){2,}[a-z]?(?![a-z])
```

(?<![a-z])

Negative look-behind: the match may not start right after a letter, so the tail of an ordinary word is never mistaken for an acronym (`abs.c.d` is left alone).

(?:[a-z]\.){2,}

At least two *letter-dot* units, the signature of a dotted acronym; a single unit such as `no.` or `bro.` does not qualify and falls through to the punctuation rule.

[a-z]?

An optional final letter without its dot, so that `u.s.a` (no trailing dot) is captured whole rather than leaving a dangling `a`.

(?![a-z])

Negative look-ahead: the acronym must end at a non-letter. In `a.b.cdef` the optional letter cannot be consumed, no match occurs, and the string degrades harmlessly to `a b cdef`.

The replacement removes the dots inside the matched span only: `u.s.a` → `usa`, `l.l.c.` → `llc`, `m.g. rd` → `mg rd`. Without this step the punctuation rule would produce `p v t`, three single-letter tokens that never hit the `pvt` → `private` expansion or the `LEGAL` filter and add spurious word boundaries to the n-gram vectors; collapsing them first lets `P.V.T. Ltd.`, `Pvt Ltd` and `Private Limited` converge on one core name.

5.3 Token Tier: `expand()` and the Four Vocabularies

`expand(s, table)` splits the string on spaces and replaces each token by `table.get(t, t)`: an exact whole-token lookup with identity fall-back. It never matches inside a token, which is why the glued form `pvtltd` has its own entry. The two abbreviation tables apply to different fields, so the shared key `ste` resolves to `societe` in names and `suite` in addresses without conflict.

Table 5.1 — NAME_ABBR: whole-token expansions applied by `norm_name()`

Token	Expansion	Group and rationale
corp	corporation	US legal forms. Expansion makes Corp/Corporation, Inc/Incorporated identical in <code>name_n</code> ; the expanded word is then in <code>LEGAL</code> , so it is removed from the core name.
co	company	
inc	incorporated	
ltd	limited	
pvt	private	Indian legal forms; <code>pvtltd</code> covers the frequently glued spelling that whole-token lookup would otherwise miss.
pvtltd	private limited	
intl	international	Generic descriptive abbreviations that appear in one source and are spelled out in another. These words are <i>not</i> in <code>LEGAL</code> , so they stay in the core name and contribute to similarity.
mfg	manufacturing	
svc	services	
svcs	services	
bro	brothers	
assoc	associates	
natl	national	
tech	technologies	
grp	group	
ent	enterprises	
cie	compagnie	
ste	societe	
		French forms for the unseen test country. <code>Cie</code> and <code>Sté</code> (after accent folding <code>ste</code>) are the usual abbreviations of <i>Compagnie</i> and <i>Société</i> ; both expansions are in <code>LEGAL</code> .

Table 5.2 — LEGAL: tokens removed by `core_name()`

Group	Tokens	Rationale
US / common-law forms	corporation company incorporated llc llp lp plc pllc	Sources omit, abbreviate or swap legal forms; removing them stops Acme and Acme Incorporated from differing on more than half of their characters in the edit-distance features.
Indian forms	private limited pvt opc	limited is shared with UK-style names; opc is the one-person company; pvt is defensive, since expansion normally rewrites it first.
Function words	the and of	Optional in practice (Bank of India vs Bank India); dropping of also lets the acronym feature in Chapter 7 compare sbi with state bank india.
Alias residue	dba	Belt-and-braces removal in case a marker survives the raw-string split of Section 5.4.
French forms	sarl sas sasu sa eurl sci snc societe compagnie et	Company types (SARL, SAS, SASU, SA, EURL, SCI, SNC), the words <i>Société</i> and <i>Compagnie</i> , and the conjunction <i>et</i> . France never appears in training, so this list is the only mechanism that makes French core names comparable at test time.

Table 5.3 — ADDR_ABBR: whole-token expansions applied by norm_addr()

Group	Token → expansion	Rationale
Street types	rd → road, st → street, str → street, ave → avenue, av → avenue, blvd → boulevard, dr → drive, ln → lane, hwy → highway, pkwy → parkway, ct → court, pl → place, sq → square	The Rd/Road family named in the problem statement; both spellings must yield identical addr_c tokens.
Unit designators	ste → suite, apt → apartment, fl → floor, flr → floor, bldg → building	Secondary-address components that one source spells out and another abbreviates.
Compass points	n → north, s → south, e → east, w → west, ne → northeast, nw → northwest, se → southeast, sw → southwest	US municipal numbering (1420 N Main St); a single-letter token carries almost no n-gram mass, so expansion is needed for it to count at all.
Indian idioms	nr → near, opp → opposite, mg → mahatma gandhi, ngr → nagar, clny → colony, sec → sector, mkt → market, stn → station, chowk → chowk, marg → road, salai → road	Landmark prefixes (Nr, Opp) are expanded so that ADDR_STOP can drop them; marg (Hindi) and salai (Tamil) are mapped onto road so a transliterated street type agrees with its English variant. chowk is an identity entry that documents a deliberately retained token.
French idioms	bd → boulevard, fbg → faubourg, chem → chemin, imp → impasse	Common French street-type abbreviations, added so that the unseen country is not handled worse than the training countries.

Table 5.4 — ADDR_STOP: tokens removed from addr_n to form addr_c

Tokens	Role	Why they are dropped
near opposite behind beside next to	Landmark relations	A landmark reference (<i>Near SBI ATM</i>) describes position, not identity; the relation word adds shared n-grams to every address in the neighbourhood and inflates cosine between different businesses.
the of and	English function words	Optional, high-frequency, carry no locating information.
no number	Number prefixes	Plot No. 12 and Plot 12 must agree; the digit is kept by _addr_nums(), the prefix is noise.
de la le du des	French articles	Rue de la Paix vs Rue Paix; articles dominate French token counts and would otherwise make every French address resemble every other.

Design note. Whole-token expansion has known side effects: `St. Louis` becomes `street louis`, a possessive `s` in an address becomes `south`, `co-op` becomes `company op`. These rewrites hit both sides of every pair identically, so they create no asymmetric disagreement; the tables trade a little semantic distortion for a large gain in collision rate on the abbreviation patterns the problem statement lists.

5.4 Name Columns: `norm_name()`, `core_name()` and `name_parts()`

`norm_name(s)` is `expand(basic(s), NAME_ABBR)`. `core_name(n)` removes every token in `LEGAL` and, if nothing remains (`The Company`), returns the full normalised name instead: an empty core name would yield an all-zero TF-IDF row in the `name` blocking view, a name-blind row in the `full` view and zero for every name similarity, leaving the address alone to find and confirm a match.

5.4.1 The `_ALIAS` regex and why it runs on the raw string

```
_ALIAS = \b(?:d\s*/\s*b\s*/\s*a | d\.b\.a\.? | dba | doing business as | trading as
          | t\s*/\s*a | a\s*/\s*k\s*/\s*a | a\.k\.a\.? | also known as
          | formerly(?: known as)?)\b      (flag re.I; spaces added for reading)
```

Each alternative is a marker separating a legal name from a trade name inside one field: the `d/b/a` family (optional spaces around the slashes, dotted `d.b.a` with an optional final dot, bare `dba`, `doing business as`), the British `trading as` and `t/a`, the `a/k/a` family with `also known as`, and `formerly` with or without `known as`. The `\b` anchors require a whole word, so `t/a` does not fire inside `Fast/Auto`; `re.I` accepts `DBA`. Bare `aka` is deliberately absent: it occurs as a legitimate name element (personal, place and brand names), and a false split would set `has_alias` and invent a second variant for an entity that has none.

The regex runs on the `raw` field because `basic()` is destructive: it turns `d/b/a` and `t/a` into `d b a` and `t a`, sequences indistinguishable from initials, and `_dotted` collapses `a.k.a.` into `aka`, precisely the bare token the regex refuses to match. Splitting first is the only order in which the marker is still recognisable.

`name_parts(raw)` splits on `_ALIAS`, applies `core_name(norm_name(.))` to each piece, discards empty pieces and falls back to the single core name when nothing survives; the result is `name_alts`. In `prep()` the main column `name_n` is built from the *joined* pieces (`" ".join(_ALIAS.split(s))`): legal and trade names are concatenated with the marker removed, so blocking sees both vocabularies in one string and keeps its recall, while `n_alias_best` in Chapter 7 scores every (variant, variant) pair to reward a record that matches either name alone.

5.5 Address Columns: `norm_addr()`, `_postal` and `_addr_nums()`

`norm_addr(s)` is `expand(basic(s), ADDR_ABBR)` and yields `addr_n`; removing `ADDR_STOP` tokens yields `addr_c`, used by the blocking views and the address similarities.

5.5.1 The `_postal` regex

```
_postal = \b(\d{5,6})(?:-\d{4})?\b
```

The capture group requires a run of exactly five or six digits bounded by word boundaries; an optional `-dddd` suffix is consumed but not captured. `prep()` applies it to the raw address and keeps the *last* match, because postal codes conventionally close an address whereas plot and house numbers open it. A US ZIP or ZIP+4 (`62704-1234`) yields `62704`; the six-digit Indian PIN (`201301`) is captured because `\d{5,6}` is greedy; the five-digit French `code postal` (`75002`), which precedes the city name, is captured by the same rule, so the

unseen country needs no special case. Runs of seven or more digits (phone numbers) never match, because a word boundary cannot fall inside a digit run. A missing code yields the empty string, which the feature layer encodes as an explicit unknown state rather than a disagreement.

5.5.2 `_addr_nums()`: digit runs and the house number

`_addr_nums(a, p)` takes the normalised address and the postal code. It collects every digit run with `_digits = \d+`, removes the last occurrence of the postal code, and returns the frozenset of remaining runs (`nums`) together with the first *remaining* run (`house`); an address whose only digit run is its postal code therefore has an empty `house`. The removal is guarded by one exact exception:

```
if p and p in ns and (ns[0] != p or a.endswith(p)):
    drop the last occurrence of p from ns
```

The postal code is removed when it is not the leading number, or when it is the leading number but the address also ends with it. The case that is *not* removed is a five- or six-digit run that opens the address and does not close it: a US house number such as `12345 Main Street` on a record whose ZIP is missing. `_postal` captures that house number as the postal code, and without the guard it would vanish from `nums`, leaving the record without its most discriminative address token. The guard keeps it as `house` and in `nums`, so `house_state` and `num_overlap` in Chapter 7 still fire. Under $F_{0.5}$ these numeric agreements are the cheapest precision signal available: two records sharing a name but conflicting on a house number are far more often neighbouring branches than one business.

Risk. In the exception case the `postal` column still holds the house number, so `postal_state` compares it against the other record's real ZIP and reports `-1`. Likewise the `+4` suffix of a ZIP+4 stays in `nums` as a separate run (`1234` in the worked example), so a pair where only one side carries the suffix loses some `num_overlap`. Both effects are systematic and visible to the tree model, but they mark where a rule-based extractor is weaker than a country-aware parser (see Chapter 14).

5.6 `prep()`: Output Columns and Worked Examples

`prep(df)` copies the frame, adds the columns of Table 5.5 and resets the index so that the integer positions used by `block()` and `features()` are dense. Country goes through `basic()` only: `US`, `India` and `France` become `us`, `india` and `france`, while a variant such as `U.S.A.` becomes `usa`, a distinct label that `block()` pairs with S2/S3 records carrying the same label or, when none exist, resolves through the open-set fallback against every S2/S3 record (see Chapter 6).

Table 5.5 — Columns produced by `prep()` and their consumers

Column	Definition	Consumed by
<code>name_n</code>	<code>norm_name()</code> of the raw name with alias markers removed and the pieces joined; legal forms retained	<code>n_full_ratio</code>
<code>name_c</code>	<code>core_name(name_n)</code> : legal forms and function words dropped, fallback to <code>name_n</code>	Blocking views <code>name</code> , <code>full</code> ; <code>n_ratio</code> , <code>n_tsort</code> , <code>n_tset</code> , <code>n_partial</code> , <code>n_jw</code> , <code>n_jacc</code> , <code>n_first_eq</code> , <code>n_init_eq</code> , <code>n_acronym</code> , <code>len_l/len_r</code>
<code>name_alts</code>	<code>name_parts()</code> : list of core-normalised alias variants	<code>n_alias_best</code> , <code>has_alias</code>
<code>addr_n</code>	<code>norm_addr()</code> of the raw address	<code>addr_c</code> , <code>_addr_nums()</code>
<code>addr_c</code>	<code>addr_n</code> without <code>ADDR_STOP</code> tokens	Blocking views <code>full</code> , <code>addr</code> ; <code>a_ratio</code> , <code>a_tset</code> , <code>a_partial</code> , <code>a_jacc</code> , <code>alen_l/alen_r</code>

Column	Definition	Consumed by
postal	Last <code>_postal</code> match on the raw address, or ""	Same-postal blocking view; <code>postal_state</code>
nums	Frozenset of digit runs in <code>addr_n</code> minus the postal code (with the exception of 5.5.2)	<code>num_overlap</code> , <code>num_conflict</code>
house	First digit run of <code>addr_n</code> after the postal-code removal, or ""	<code>house_state</code>
country_n	<code>basic(country)</code>	Per-country partition in <code>block()</code> ; recall-by-country log
src	<code>entity_id[:2]</code> (S1, S2 or S3)	<code>is_s3</code> ; (S1, source) groups in <code>decode_table()</code>

5.6.1 Two records traced through every step

The Indian record is `Sharma Bros. P.V.T. Ltd.` at Plot No. 12, Nr. SBI ATM, M.G. Rd, Sec 5, Noida 201301; the US record is `J & R Auto Repair, L.L.C. d/b/a Jim's Garage` at 1420 N. Main St., Ste 200, Springfield, IL 62704-1234. Figures 5.1 and 5.2 trace the name and address paths; Table 5.6 lists the final columns. In the Indian name, `_dotted` rewrites `p.v.t.` but leaves `bros.` and `ltd.` alone because each has a single letter-dot unit; the punctuation rule strips their dots and `NAME_ABBR` expands all three tokens. In the US name the raw-string alias split yields two variants, and the possessive in `Jim's` survives as a stray `s` token, which the token-set similarities of Chapter 7 tolerate.

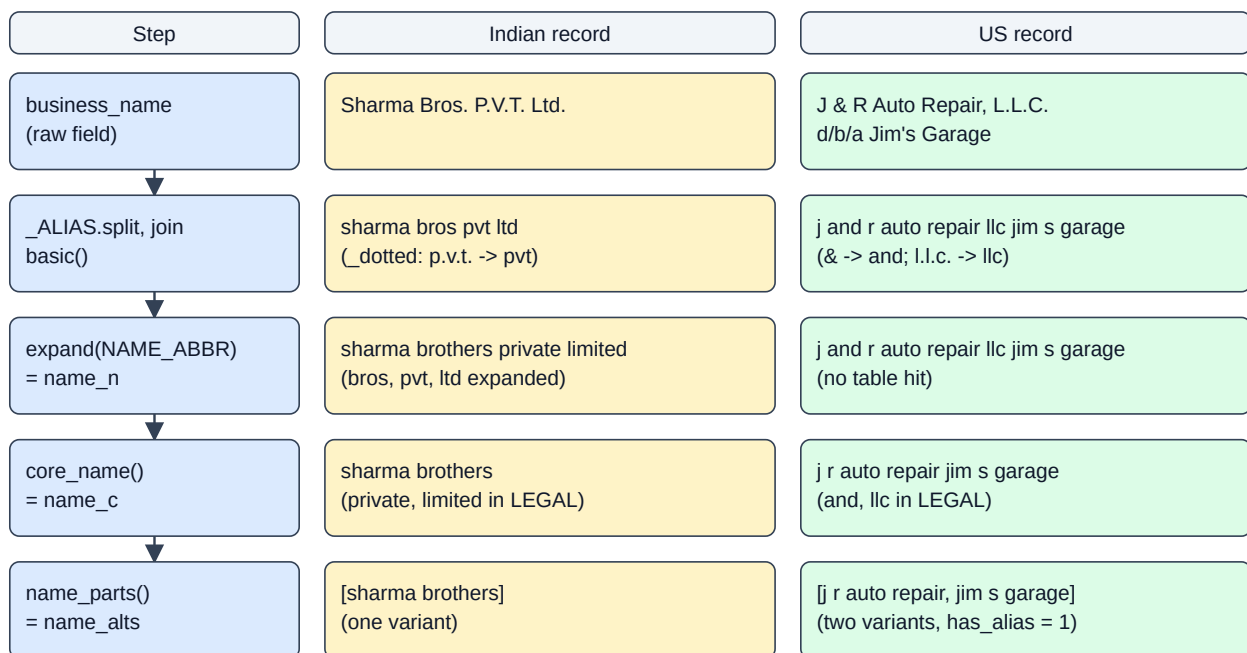


Figure 5.1 — Name normalisation flow: the Indian and US worked records at each step from the raw `business_name` to `name_n`, `name_c` and `name_alts`. The alias split runs on the raw string before `basic()`.

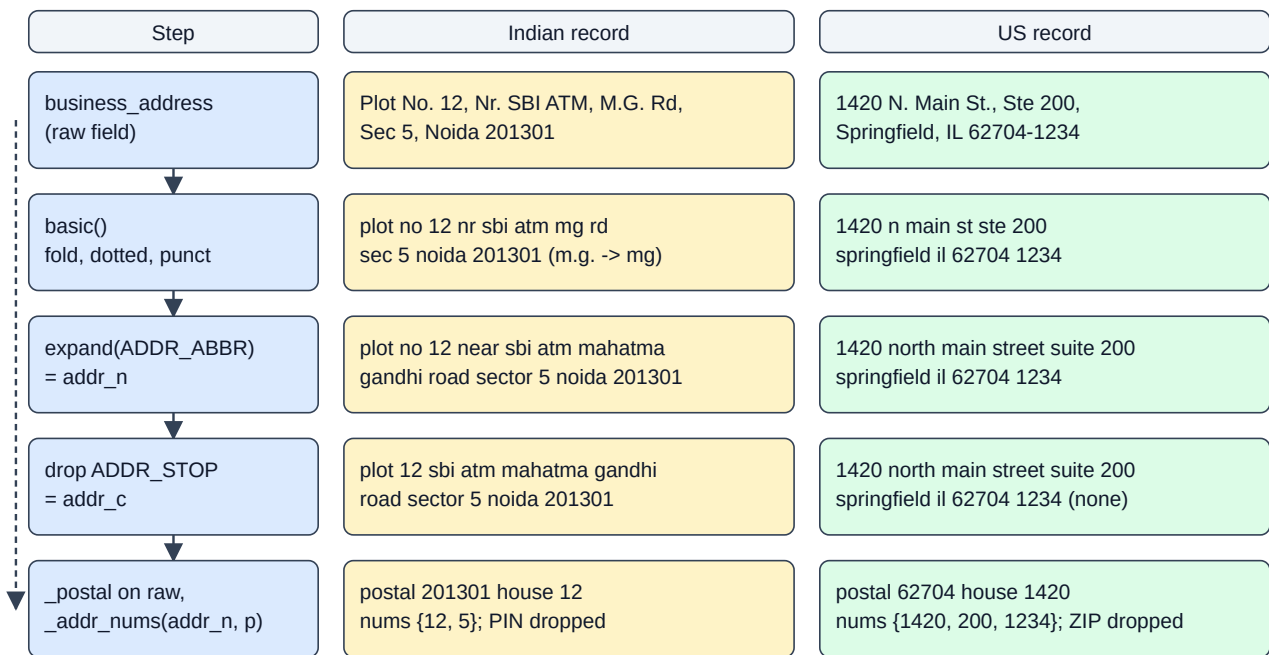


Figure 5.2 — Address normalisation flow for the same two records. The dashed line marks that `_postal` reads the raw address (step 1) while `_addr_nums()` reads `addr_n` (step 3); both feed the final row.

Table 5.6 — `prep()` output for the two worked records

Column	Indian record	US record
name_n	sharma brothers private limited	j and r auto repair llc jim s garage
name_c	sharma brothers	j r auto repair jim s garage
name_alts	["sharma brothers"]	["j r auto repair", "jim s garage"]
addr_n	plot no 12 near sbi atm mahatma gandhi road sector 5 noida 201301	1420 north main street suite 200 springfield il 62704 1234
addr_c	plot 12 sbi atm mahatma gandhi road sector 5 noida 201301	1420 north main street suite 200 springfield il 62704 1234
postal	201301	62704
nums	{12, 5}	{1420, 200, 1234}
house	12	1420
country_n	india	us
src	S1 (from the id prefix)	S1 (from the id prefix)

Both traces are derived by hand from the regexes and tables above; the pipeline has not yet been executed on the challenge data, so corpus-level statistics of this layer (share of records with a postal code, an alias marker or an empty core name) are to be filled from the run log (see Chapter 10). Table 5.5, together with the raw `entity_id` and `country` columns read by `main()` for output and logging, is the complete interface to the rest of the system: any change to a table or regex here alters the feature distribution the LightGBM matcher (see Chapter 8) is trained on and requires retraining and a threshold re-sweep (see Chapter 9).

6 Candidate Generation (Blocking)

This chapter describes the candidate-generation stage implemented by `block()`, `topk_sparse()` and `pair_cos()`. Blocking replaces the infeasible cross product of Source 1 against Source 2 and Source 3 with a short list of plausible partners per Source 1 record and thereby fixes the recall ceiling of the system. It combines three character n-gram TF-IDF views with an exact-key postal view, partitions by country with an open-set fallback, bounds memory by construction, and, together with the training-side checks in `main()`, prints diagnostics that show where to act when the ceiling is too low.

6.1 Why blocking is needed

The matcher of Chapter 8 scores one pair at a time. Without pre-selection it would score every Source 1 (S1) record against every Source 2 or Source 3 (S2/S3) record, $|S1| \times |S2+S3|$ pairs: with, for instance, a few hundred thousand records per side that is 10^{10} to 10^{11} pairs, infeasible for the string-alignment features of Chapter 7 and hopelessly imbalanced for the classifier. Blocking restricts the comparison space to pairs that share cheap evidence of similarity.

Blocking also caps the score. A true partner absent from the candidates of its S1 record can never be predicted; the fraction of ground-truth pairs that survive is the *recall ceiling*. Under the macro-averaged $F_{0.5}$ of Chapter 2 the ceiling bites per entity: an S1 entity with true matches, none of which is blocked, contributes exactly 0.0 whatever the matcher does, while a singleton is unaffected. The stage is therefore designed for recall and leaves precision to the matcher and the decision layer of Chapter 9. The table returned by `block()` is also, unfiltered, the content of `candidate_pairs.tsv`, which must be exactly the set the model runs inference over.

6.2 Representation: character n-gram TF-IDF

Each view is a `TfidfVectorizer` with `analyzer="char_wb"`, `ngram_range=(2, 4)`, `min_df=2`, `sublinear_tf=True` and `dtype=np.float32`. The `char_wb` analyser pads each token with a space and emits its character 2-, 3- and 4-grams without crossing token boundaries, so a typo or transposition changes a handful of n-grams and leaves the rest intact, which tolerates the residual noise the tables of Chapter 5 do not cover. `min_df=2` drops n-grams found in a single document, which can never contribute to a dot product between two records, so the vocabulary shrinks at no cost to recall. `sublinear_tf` replaces the term frequency `tf` by $1 + \log(tf)$, so a long address repeating a common n-gram does not dominate its own vector, and `float32` halves the memory of the matrices and of the dense chunks of 6.5.

The vectoriser is fitted on `pd.concat([a, b])`, the S1 and S2/S3 texts of the split being processed. Rows are L2-normalised (the default), hence the dot product of two rows is their cosine and one sparse product yields all cosines. Refitting on the test split learns French n-grams from the test records themselves, with no external vocabulary (see Chapter 13).

6.3 The four views

`block()` builds three fuzzy views and one exact-key view (Table 6.1). Each fuzzy view vectorises a different text: the core name `name_c`, the concatenation `name_c + " | " + addr_c`, and the cleaned address `addr_c` (Chapter 5). The views are complementary with respect to the noise taxonomy of Chapter 3: the name view finds partners whose address is missing or rewritten; the address view finds partners whose name is a trade name,

acronym or heavy abbreviation; the full view ranks highest the pairs on which both fields agree, and its cosine `cos_full` later drives the rank and gap features of Chapter 7. Depth differs per view: $k = 25$ for name and full, because near-duplicate names (chain branches, common words) are frequent and the true branch must stay in the list; $k = 10$ for address, because addresses are shared by many businesses and an address-only neighbour is weak evidence.

The fourth view is exact-key. Within a country block, S2/S3 records are grouped by their extracted `postal` code; for every S1 postal group with a matching S2/S3 group, the name-view vectors of the two groups are compared and the top $k_{\text{postal}} = 10$ by name cosine are kept (empty codes are skipped). This recovers matches whose name is too noisy for the country-wide top 25: inside one ZIP or PIN group the competition is the postal group rather than the whole country, so the same name cosine is far more discriminative. The view is silent for records whose address carries no 5- or 6-digit code (`postal` is then empty, Chapter 5), so it complements the fuzzy views rather than replacing them.

Table 6.1 — Blocking views and their parameters as set in `block()`.

View	S1 text	S2/S3 text	Search space	k	Purpose
name	<code>name_c</code>	<code>name_c</code>	same country block	25	address missing or rewritten
full	<code>name_c + " " + addr_c</code>	same	same country block	25	both fields agree; source of <code>cos_full</code>
addr	<code>addr_c</code>	<code>addr_c</code>	same country block	10	trade name, acronym, rewritten name
postal	<code>name_c</code> (name vectoriser)	<code>name_c</code>	same country and identical <code>postal</code>	10	noisy name inside one ZIP/PIN

Common settings: `char_wb`, n-grams (2, 4), `min_df=2`, `sublinear_tf=True`, float32, L2 rows; cut `sc > 0.05` on every view; `max_cells=2e7` in `topk_sparse()`.

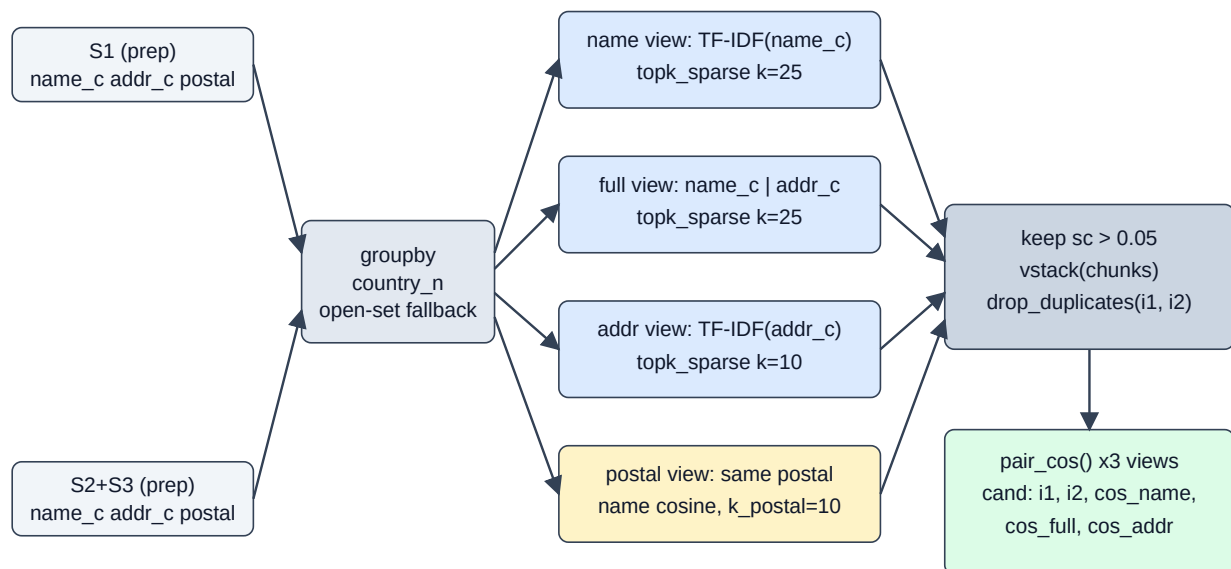


Figure 6.1 — The four blocking views inside `block()`, run per country block, feed one cut-and-union step and then `pair_cos()`, which attaches the three cosines to every surviving pair.

6.4 Partitioning by country and the open-set fallback

The country label is the only field that permits an exact partition. `block()` iterates over `s1.groupby("country_n")`, where `country_n` is `basic(country)`, and restricts the S2/S3 index `i2` to rows with the identical normalised country. Partitioning makes a fixed k more valuable, since the top 25 within one country are likelier to contain the true partner than the top 25 across three, and it reduces the product cost to $\sum_c |S1_c| \times |S2S3_c|$.

The label is an open set, as the problem statement requires. `other_countries` is computed from the S2/S3 side; if an S1 group's label is absent from it, the group is compared against *all* S2/S3 rows rather than an empty block, so no S1 entity is silently dropped when its label is unseen, blank or misspelt on one side. France is unseen in training but present in the test set, so wherever the French label occurs on the S2/S3 side, French S1 records block within that block exactly as US and Indian records do; were the label present only on the S1 side, the fallback would compare those records against every S2/S3 row. Nothing in the code enumerates countries, and the same `block()` call is made for the training and the test split.

Risk. If a blank label occurs on both sides, blank S1 records are compared with blank S2/S3 records only, which is too narrow if blanks are noise rather than a category. The per-country recall diagnostic of 6.8 exposes this; the remedy is to route the blank group through the all-records path.

6.5 Bounded top-k retrieval: `topk_sparse()`

Retrieving the k nearest S2/S3 rows for every S1 row of a view is a sparse product of the S1 matrix A with the transposed S2/S3 matrix B^T , followed by a row-wise selection. To avoid materialising the full $|A| \times |B|$ product, `topk_sparse()` processes A in row chunks sized from $|B|$: `chunk = max(16, int(max_cells // n_b))` with `max_cells = 2e7`. The dense transient `S = (A[i:i+chunk] @ BT).toarray()` then holds about 2×10^7 float32 cells whatever the block size, until $|B|$ exceeds 1.25×10^6 rows: from there the floor of 16 rows keeps the loop efficient at the price of a proportionally larger transient (Table 6.2). Before the loop, k is clipped to $|B|$ and an empty A or B returns empty arrays of width 0, so a tiny country block cannot raise.

Selection uses `np.argpartition(S, S.shape[1] - k, axis=1)`: partitioning at $k_{th} = |B| - k$ places the k largest scores of each row in the last k positions, unordered, at linear expected cost, without a full sort or a negated copy of S . The slice `[:, -k:]` is followed by `.copy()`, and the copy matters: `argpartition` returns a full `chunk \times |B|` int64 array, and a slice of it is a view that keeps the whole array alive; appending views to `idx_out` would retain every full-width array until the function returns, multiplying peak memory by the number of chunks. `np.take_along_axis` gathers the scores, and the stacked arrays are returned.

Because `argpartition` always returns exactly k columns, a row with fewer than k records sharing any n -gram is padded with arbitrary zero-score columns. The cut `keep = sc > 0.05` removes this padding and near-zero neighbours. Each view's survivors are mapped to global indices (`np.repeat(i1, k)` for rows, `i2[idx]` for columns), stacked, and de-duplicated with `drop_duplicates()`, since a good pair is usually found by several views; an S1 record has at most $25 + 25 + 10 + 10 = 70$ candidates before de-duplication.

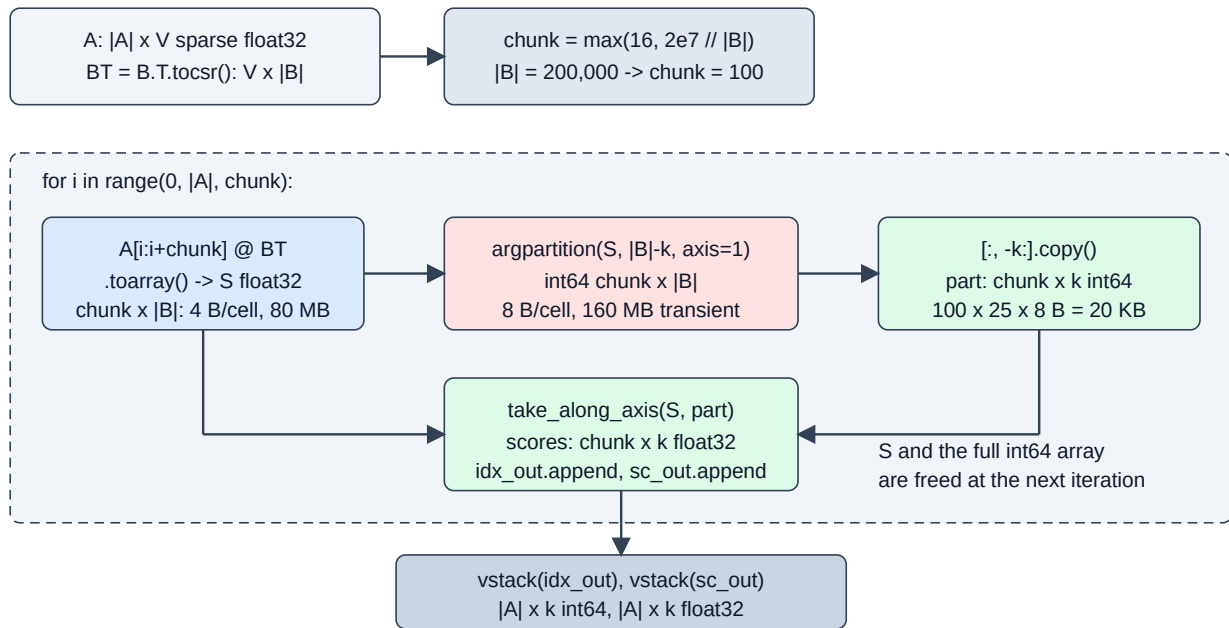


Figure 6.2 — The chunk loop of `topk_sparse()` with memory annotations for $|B| = 200,000$ and $k = 25$; the `.copy()` ensures that neither large transient survives the iteration.

6.6 Memory budget and time complexity

Three transients are allocated per chunk: the dense float32 scores at 4 bytes per cell, the int64 `argpartition` result at 8 bytes per cell, and the sparse product before `toarray()`, at 4 bytes of data plus 4 bytes of column index per non-zero. The sparse product is often close to dense (frequent bigrams occur in most records) and is budgeted at up to 8 bytes per cell. They are not all alive at once: the sparse product is released as soon as `toarray()` returns, before `argpartition` allocates, so the true peak is the larger of (sparse product + scores) and (scores + `argpartition`), and the sum of all three is a conservative bound. Table 6.2 works the budget through for three block sizes; the third row shows the 16-row floor.

Table 6.2 — Memory budget of one `topk_sparse()` chunk for three block sizes ($k = 25$, $\text{max_cells} = 2e7$, $|A| = 100,000$ S1 rows for the iteration count). Values follow from the formulas in 6.5 and 6.6; nothing here is measured.

$ B $ (S2/S3 rows in block)	chunk rows	cells per chunk	scores S (float32)	argpartition (int64)	sparse product (≤ 8 B/nz)	peak transient (bound)	iterations
20,000	1,000	2.0×10^7	80 MB	160 MB	≤ 160 MB	≈ 240 MB (≤ 400 MB)	100
200,000	100	2.0×10^7	80 MB	160 MB	≤ 160 MB	≈ 240 MB (≤ 400 MB)	1,000
2,000,000	16 (floor)	3.2×10^7	128 MB	256 MB	≤ 256 MB	≈ 384 MB (≤ 640 MB)	6,250

Retained per view for $|A| = 100,000$, $k = 25$: indices $100,000 \times 25 \times 8$ B = 20 MB, scores 10 MB. The postal view runs on far smaller B, so one chunk covers a whole group and its transients are negligible.

Time is dominated by the sparse products. The cost of $A_c B_c^T$ is the sum, over S1 rows and their n-grams, of each n-gram's posting-list length; with near-dense overlap it behaves like $O(|A_c| \cdot |B_c|)$ with a small constant, and `argpartition` adds another linear term per row. Over all views and countries the stage is $O(\sum_c |S1_c| \cdot |S2S3_c|)$: the same order as brute force, but with a few floating-point operations per cell instead of several string alignments per pair. Chapter 12 places these terms in the end-to-end budget.

6.7 Cosines for every surviving pair: `pair_cos()`

A pair retrieved only by the address view carries no name score from retrieval, yet the matcher needs all three cosines for every pair. `block()` therefore calls `pair_cos(A, B, i1, i2)` once per fuzzy view after the union. The function gathers `A[i1]` and `B[i2]` in slices of 200,000 pairs, multiplies them element-wise with the sparse `multiply()` and sums each row, which for L2-normalised rows is the cosine; slicing bounds memory regardless of the pair count. The resulting `cos_name`, `cos_full` and `cos_addr` are the first features of Chapter 7, and a uniformly computed `cos_full` is what makes the rank and gap features comparable.

6.8 Diagnostics and how to act on them

On the training split every candidate pair receives a label $y = 1$ if the S2/S3 id appears in the S1 entity's ground-truth list, and `main()` prints the quantities of Table 6.3. The count of *S1 entities with at least one true match blocked* is the number that matters for the macro metric: every matched entity outside it is capped at 0.0. The *blocking oracle* macro $F_{0.5}$, obtained by predicting exactly the true blocked pairs, is the ceiling of the entire system; its gap to the out-of-fold score of Chapter 10 says whether effort belongs in blocking or in the matcher, and the *all-candidates floor* measures how much precision the matcher must add. The measured column is a placeholder until the pipeline has been run on the released data.

Table 6.3 — Blocking diagnostics printed by `block()` and `main()`, their interpretation, and the action each one triggers. Measured values: to be filled from the run log.

Log line	Definition	Measured (train)	Action if unsatisfactory
blocking: N pairs (x per S1)	<code> cand </code> and <code> cand / S1 </code> , printed by <code>block()</code> ; the reduction ratio $1 - cand / (S1 \cdot S2+S3)$ is derived from it by hand, not printed	to be filled from the run log	far above ~50 per S1: lower <code>k_addr</code> or raise the 0.05 cut; far below: check that the country partition is not empty
blocking recall ceiling	$\sum y / \text{total ground-truth pairs}$	to be filled from the run log	raise <code>k_name / k_full</code> ; add a view (e.g. word tokens or house-number key)
S1 entities with ≥ 1 true match blocked	distinct <code>i1</code> with $y = 1$ over S1 entities with non-empty truth	to be filled from the run log	same as above; this is the macro-relevant recall
blocking recall by country	$\sum y / \text{true pairs}$, grouped by S1 <code>country_n</code>	to be filled from the run log	one country low: extend <code>ADDR_ABBR / ADDR_STOP</code> for its idioms (see Chapter 5), or raise <code>k</code> for that block
blocking recall by source	$\sum y / \text{true pairs}$, grouped by S2 / S3 prefix	to be filled from the run log	S3 low: its fields are noisier; raise <code>k_addr</code> or <code>k_postal</code>
macro $F_{0.5}$ if every candidate were predicted (floor)	score of the identity matcher on the candidate lists	to be filled from the run log	informational: the gap to the oracle is the matcher's job
blocking oracle macro $F_{0.5}$ (ceiling)	score of a perfect matcher on blocked pairs	to be filled from the run log	if close to the OOF score, invest in blocking; otherwise in the matcher

Design note. Every `k` in Table 6.1 is a keyword argument of `block()`, so raising a view's depth is a one-token change, and adding a view is one entry in the `fields` dictionary, over which every loop of the stage iterates; the new `cos_*` column enters the feature list automatically.

7 Pair Feature Engineering

This chapter describes `features()`, which turns every candidate pair from blocking (Chapter 6) into the 38-column numeric row scored by the LightGBM matcher of Chapter 8. The features fall into three groups: pairwise similarities of the normalised name and address, agreement states of the structured address components, and context features that place a pair among the competing candidates of the same Source 1 and Source 2/3 record. Each similarity absorbs one noise class from Chapter 3; the context features let a per-pair classifier approximate the per-entity decision that precision-weighted $F_{0.5}$ rewards.

7.1 Role of the feature layer

`features(cand, s1, other)` receives the candidate table (`i1`, `i2`, the three cosines) and the prepared frames of Chapter 5, and aligns the left record `L = s1.loc[cand.i1]` with the right record `R = other.loc[cand.i2]` row by row, so every column below is a function of one (L, R) pair. `main()` takes every column except `i1`, `i2` and the label `y` as model input, so Table 7.1 is the complete feature list. No feature reads the country label, the raw text or the ground truth. Because $F_{0.5}$ weights precision twice as heavily as recall and a singleton scores 1.0 only with an empty list (Chapter 2), the layer supplies evidence *against* a match (conflicting numbers, a dominating competitor) as deliberately as evidence for one.

Table 7.1 — Complete feature catalogue of `features()`; L is the Source 1 record, R the Source 2/3 record

Feature	Definition	Range	Noise addressed / role
<code>cos_name</code>	TF-IDF char_wb 2-4-gram cosine of <code>name_c</code> (from <code>block()</code>)	0-1	Typos, abbreviations, reordering in the core name
<code>cos_full</code>	Cosine of <code>name_c + " " + addr_c</code>	0-1	Joint name and address evidence; basis of rank and gap features
<code>cos_addr</code>	Cosine of <code>addr_c</code>	0-1	Address rewriting, abbreviations
<code>n_ratio</code>	<code>fuzz.ratio(L.name_c, R.name_c)</code>	0-100	Character typos; order-sensitive baseline
<code>n_tsort</code>	<code>fuzz.token_sort_ratio</code> on <code>name_c</code>	0-100	Token reordering
<code>n_tset</code>	<code>fuzz.token_set_ratio</code> on <code>name_c</code>	0-100	Extra tokens on one side (trade names, descriptors)
<code>n_partial</code>	<code>fuzz.partial_ratio</code> on <code>name_c</code>	0-100	Truncation, one name embedded in the other
<code>n_jw</code>	<code>JaroWinkler.similarity</code> on <code>name_c</code>	0-1	Interior typos with intact prefix
<code>n_full_ratio</code>	<code>fuzz.ratio(L.name_n, R.name_n)</code> (legal suffixes kept)	0-100	Legal-form agreement between records with equal core names
<code>n_jacc</code>	<code>jacc()</code> : Jaccard of core-name token sets	0-1	Whole-token overlap independent of spelling
<code>n_first_eq</code>	1 if first tokens of <code>name_c</code> are equal	{0,1}	Brand token agreement
<code>n_init_eq</code>	1 if the strings of token initials are equal	{0,1}	Same initials pattern; strong for multi-token names
<code>n_alias_best</code>	<code>max token_sort_ratio</code> over all pairs of <code>name_alts</code> variants	0-100	Legal name on one side, DBA / trade name on the other

Feature	Definition	Range	Noise addressed / role
has_alias	1 if either record has more than one name variant	{0,1}	Marks pairs where n_alias_best is informative
n_acronym	1 if one core name is a single token equal to the initials of the other multi-token core name	{0,1}	Acronym vs expansion (sbi / state bank india)
a_ratio	fuzz.ratio on addr_c	0-100	Address typos
a_tset	fuzz.token_set_ratio on addr_c	0-100	Partial addresses, extra locality tokens
a_partial	fuzz.partial_ratio on addr_c	0-100	Truncated addresses
a_jacc	Jaccard of address token sets	0-1	Whole-token overlap of addresses
postal_state	0 if either postal empty; 1 equal; -1 different	{-1,0,1}	Missing vs conflicting postal codes
num_overlap	Jaccard of nums digit-run sets; -1 if either empty	-1 or 0-1	Street / plot / floor number agreement
num_conflict	1 if both nums non-empty and disjoint	{0,1}	Explicit evidence against a match
house_state	0 if either house empty; 1 equal; -1 different	{-1,0,1}	Leading (house) number agreement
len_l, len_r	Character length of name_c on each side	integer	Calibrates similarity scores by string length
alen_l, alen_r	Character length of addr_c on each side	integer	Detects empty or stub addresses
is_s3	1 if R.src == "S3"	{0,1}	Source-specific field quality
cos_full_gap1, n_tset_gap1, a_tset_gap1	Value minus the maximum of that column over the same i1	≤ 0	Is this the best candidate of the S1 record
cos_full_gap2, n_tset_gap2, a_tset_gap2	Value minus the maximum over the same i2	≤ 0	Is this S1 the best owner of the S2/S3 record
rank1	Rank of cos_full within the same i1, 1 = best, ties by table order	1..n_cand1	Position among the S1 record's candidates
rank2	Rank of cos_full within the same i2	1..n_cand2	Position among the S2/S3 record's candidates
n_cand1	Number of candidates of the S1 record	≥ 1	Crowding: common names, chains
n_cand2	Number of S1 records competing for the S2/S3 record	≥ 1	Crowding on the other side

7.2 TF-IDF cosines

The three cosines are not recomputed by `features()`; `block()` attaches them through `pair_cos()` for every surviving pair, whichever view retrieved it. Each view has its own `TfidfVectorizer(analyzer="char_wb", ngram_range=(2, 4), min_df=2, sublinear_tf=True)` fitted on the union of both sides, so a cosine is corpus-weighted (the inverse document frequency of the n-grams of "road" or "nagar" is low), which complements the rapidfuzz scorers that weight every character equally. `cos_full` is the basis of the rank and gap features of 7.7 because it is the only score combining name and address evidence for every pair.

7.3 rapidfuzz name similarities

7.3.1 What each scorer computes

All five scorers run on the core name `name_c` (Chapter 5). `fuzz.ratio` is the normalised Indel similarity: with LCS the length of the longest common subsequence, $\text{ratio} = 200 \cdot \text{LCS} / (|a| + |b|)$. It is order-sensitive and treats every character alike; each other scorer relaxes it in one direction.

`fuzz.token_sort_ratio` sorts the whitespace tokens of both strings, re-joins them and applies `ratio`, so any permutation of the same tokens scores 100: it absorbs reordering and nothing else. `fuzz.token_set_ratio` forms the sorted intersection t_0 of the token sets and the strings $t_0 + (\text{tokens only in a})$ and $t_0 + (\text{tokens only in b})$, and returns the best `ratio` among the three pairings; when one token set is a subset of the other the score is 100, so extra tokens on one side are ignored wherever they occur. `fuzz.partial_ratio` returns the best `ratio` of the shorter string against any window of its own length in the longer one; it absorbs truncation but not interleaved extra tokens. `JaroWinkler.similarity` counts characters that match within a window of $\lfloor \max(|a|, |b|)/2 \rfloor - 1$ positions, penalises transpositions and adds a prefix bonus of up to four characters with weight 0.1, $J_w = J + \ell \cdot 0.1 \cdot (1 - J)$; it stays high for an interior typo when the first characters agree and drops fast when they differ.

All five are kept because their disagreement pattern is itself a signal: `n_tset` = 100 with `n_ratio` = 55 is a subset relation, 100 with 100 an exact match. `n_full_ratio` applies `fuzz.ratio` to the full name `name_n`, the only feature that still sees the legal form; it separates records whose core names coincide but whose legal forms differ, often the line between a match and a costly false positive.

7.3.2 Worked comparison

Table 7.2 — rapidfuzz scorers on four example core-name pairs; values are computed by hand from the scorer definitions and are approximate to within about two points

Pair (name_c L / R)	Noise	n_ratio	n_tsort	n_tset	n_partial	n_jw	n_jacc
acme pizza / pizza acme	Token reorder	50	100	100	67	0.52	1.00
bharat petroleum / bharat oil petroleum depot	Interleaved extra tokens	76	76	100	75	0.89	0.50
mahindra logistics / mahindra logistics supply chain solutions	Truncation	61	61	100	100	0.89	0.40
reliance retail / relaince retail	Interior transposition	93	93	93	93	0.98	0.33

The rows show the division of labour. Reordering is invisible to `n_tsort` and `n_tset` but heavily penalised by `n_ratio` and `n_jw`. Interleaved extra tokens are absorbed only by `n_tset`; `n_partial` finds no contiguous window. Truncation is absorbed by both. An interior transposition leaves every scorer above 90 but is best absorbed by Jaro-Winkler through its prefix bonus, while `n_jacc` drops to 0.33 (one shared token out of three distinct ones), which is why whole-token overlap must be complemented by character-level scorers.

7.3.3 Token-level agreement

`n_jacc` is the Jaccard index of the core-name token sets computed by `jacc()`, which returns 0.0 when either set is empty. `n_first_eq` tests whether the first tokens coincide, a cheap proxy for brand agreement because a business name usually leads with its distinctive token. `n_init_eq` compares the strings of token initials on both sides. Both are booleans stored as floats.

7.4 Alias and acronym features

Chapter 5 splits a raw name on the explicit alias markers of the `_ALIAS` regex (`dba`, `d/b/a`, `doing business as`, `trading as`, `t/a`, `a.k.a.`, `also known as`, `formerly known as`; a bare `aka` is excluded because it can be a genuine name token) into the list `name_alts` of core-normalised variants, while `name_c` keeps the concatenation with the markers removed. `n_alias_best` is the maximum `token_sort_ratio` over every variant of L paired with every variant of R, so "Acme Holdings LLC dba Acme Pizza" scores 100 against "Acme Pizza" although the concatenated core name "acme holdings acme pizza" scores lower on `n_ratio` and `n_tsort` (`n_tset` and `n_partial` already reach 100 through the subset relation). `has_alias` flags pairs where either side has more than one variant; on every other pair `n_alias_best` equals `n_tsort`, so the flag tells the model when the alias score carries new information.

`n_acronym` covers the opposite direction: one side is a single token equal to the initials of the other, multi-token side. Because `core_name()` already drops "of", "and" and "the", the initials of "state bank of india" are "sbi", the token a second source is likely to store. The feature is symmetric and, when it fires, carries information no character scorer does: "sbi" against "state bank india" scores about 32 on `n_ratio` and no better than 50 on `n_partial`.

Risk. `n_tset`, `n_partial` and `n_acronym` reward subset relations, which chain branches also share ("acme pizza" against "acme pizza airport"). They buy recall on trade names at a price in precision; the address states of 7.5 and the context features of 7.7 are the counterweight, and Chapter 9 sets the operating point on the measured $F_{0.5}$.

7.5 Address features

Four scorers run on the cleaned address `addr_c`: `a_ratio`, `a_tset`, `a_partial` and `a_jacc`. Addresses carry more benign extra tokens than names (district or state names survive stop-word removal), so `a_tset` matters more here and is the address score used for the gap features.

The structured components yield agreement states rather than similarities. `postal_state` is ternary: 0 when either postal code is missing, 1 when both are present and equal, -1 when both are present and differ. The states are kept apart because a missing code is absence of information whereas a conflict is evidence against a match, which $F_{0.5}$ makes valuable. `house_state` applies the same encoding to the leading digit run `house_num_overlap` is the Jaccard of the digit-run sets `nums` (`_addr_nums()` removes the postal code unless it is also the leading number of the address, the case of a five-digit US house number with the ZIP missing), with the sentinel -1 when either side has no digits, so "nothing to compare" is distinct from "numbers present, none shared"; `num_conflict` makes the latter an explicit boolean.

7.6 Lengths and source flag

`len_l`, `len_r`, `alen_l` and `alen_r` are the character lengths of core name and cleaned address on each side. A `ratio` of 80 on two 6-character names is one typo; on two 40-character names it is several, so the lengths let the model condition every similarity on the amount of text behind it. `is_s3` is 1 when the right record comes from Source 3 (`src` is the two-letter prefix of `entity_id`); it lets the model learn source-specific field quality, and `decode_table()` reuses the same column to group candidates per (S1, source).

7.7 Context features

Every feature so far is a function of the pair alone, but the metric is computed per Source 1 entity, and the question is not "is this pair similar" but "is this the right partner, given the alternatives". The context block gives the pairwise model that view through pandas group operations over the candidate table:

```
for col in ["cos_full", "n_tset", "a_tset"]:
    f[f"{col}_gap1"] = f[col] - f.groupby("i1")[col].transform("max")
    f[f"{col}_gap2"] = f[col] - f.groupby("i2")[col].transform("max")
f["rank1"] = f.groupby("i1")["cos_full"].rank(ascending=False, method="first")
f["rank2"] = f.groupby("i2")["cos_full"].rank(ascending=False, method="first")
f["n_cand1"] = f.groupby("i1")["i2"].transform("size")
f["n_cand2"] = f.groupby("i2")["i1"].transform("size")
```

For each of the three scores, `_gap1` is the pair value minus the best value among all candidates of the same S1 record, and `_gap2` the same relative to all S1 records competing for the same S2/S3 record; both are 0 for the leader and negative otherwise. `rank1` and `rank2` are the positions by `cos_full` on the two sides (1 = best; `method="first"` breaks ties by table order, so ranks are integers), and `n_cand1`, `n_cand2` are the group sizes. Figure 7.1 traces the computation for one S1 record.

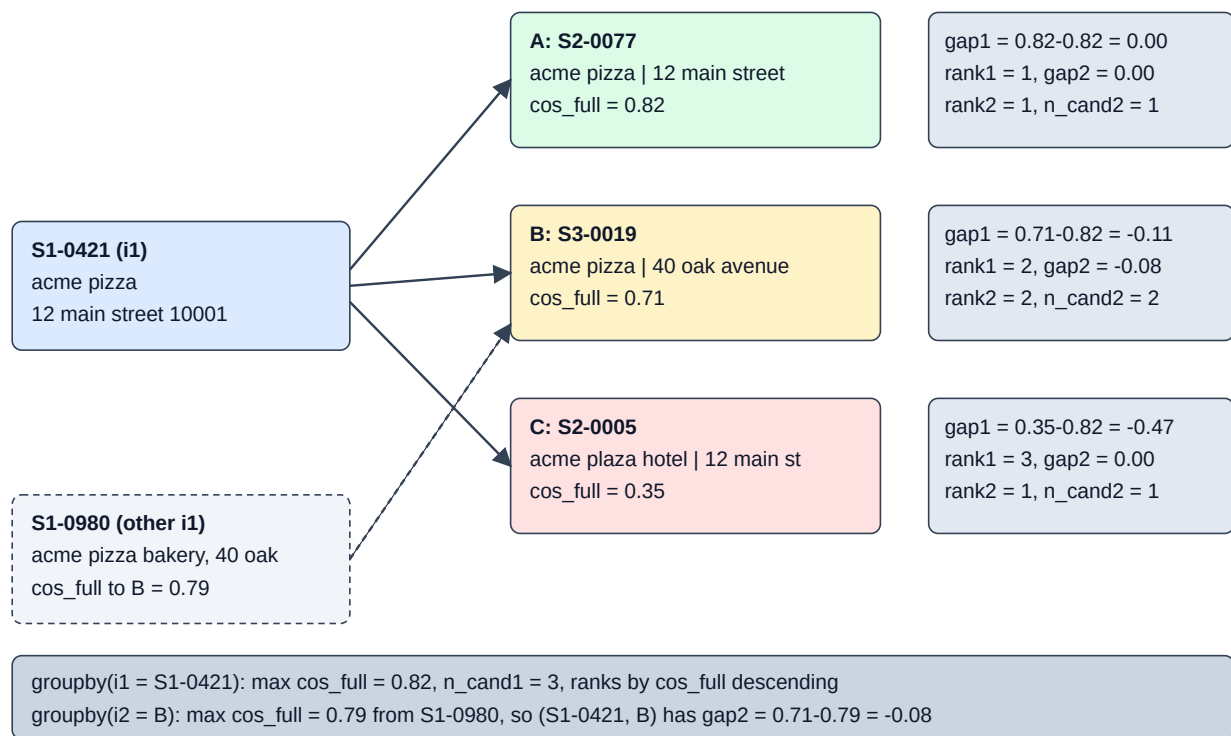


Figure 7.1 — Derivation of the `cos_full` gap, rank and crowding features for one Source 1 record with three candidates. Candidate B is also a candidate of another Source 1 record with a higher `cos_full`, which makes its `gap2` negative and its `rank2` equal to 2.

In the figure, candidate A is the mutual leader: `gap1` = `gap2` = 0, `rank1` = `rank2` = 1. Candidate B has a respectable absolute score and an identical core name but is dominated on both sides, by A for this S1 record and by S1-0980 for the S2/S3 record; candidate C is a different business at the same address. Without context the model would need one absolute threshold for B in every situation; with context it can learn that a dominated name-only match is typically a chain branch, and that a moderate leader is often right when its competitors trail far behind. The hard forms of the same idea are enforced afterwards by `decode_table()` and `decide()` (Chapter 9).

Design note. The three gap bases cover independent axes: `cos_full` for joint evidence, `n_tset` for the most permissive name score, `a_tset` for the most permissive address score. A pair leading on the address but trailing on the name is the signature of a co-located different business. Ranks and counts use `cos_full` only, since one ordering suffices for position.

7.8 Leakage, categorical handling and value encoding

The context features are computed once on the complete candidate table of a split, before the GroupKFold of Chapter 8 is drawn. Groups by `i1` are fold-internal, since every candidate of an S1 record shares its fold; groups by `i2` cross folds, because the competitors of an S2/S3 record may be S1 records held out elsewhere. This is not label leakage: the group statistics use only unsupervised scores and counts, never `y`, and the identical computation is available at test time because `features()` runs on the complete test table. What would be leakage, and is absent, is any group aggregate of the label such as a target encoding of `i2`.

No column is declared categorical. `postal_state` and `house_state` are integers in `{-1, 0, 1}`; two numeric splits (below -0.5, above 0.5) isolate each state, so numeric handling is lossless. The country is intentionally not a feature: the test holds a country absent from training (Chapter 3), and a country column would invite splits that cannot transfer. No feature yields a missing value: `jacc()` returns 0.0 for empty sets and `num_overlap` uses the -1 sentinel. The layer costs nine fixed rapidfuzz calls per pair (six on names, three on addresses) plus `|L.name_alts| × |R.name_alts|` `token_sort_ratio` calls for `n_alias_best`; it is expected to be the largest CPU cost of the pipeline, to be confirmed from the run log timings of Chapter 12. Gain importances are reported in Table 7.3 once the run log is available.

Table 7.3 — Top gain importances of the final LightGBM model as printed by `main()`; to be filled from the run log

Rank	Feature	Gain
1	to be filled from the run log	to be filled from the run log
2	to be filled from the run log	to be filled from the run log
3	to be filled from the run log	to be filled from the run log
4-15	to be filled from the run log	to be filled from the run log

8 Matching Model: LightGBM Classifier

This chapter describes the supervised matcher that turns the 38-column feature row of Chapter 7 into a match probability for every candidate pair: a single LightGBM gradient-boosted tree ensemble with a binary log-loss objective, trained inside `main()` on the blocked training pairs, validated with a five-fold `GroupKFold` grouped by Source 1 record, and refitted on all pairs before scoring the test set. The model never decides anything itself; it emits calibrated probabilities, and every decision the precision-weighted $F_{0.5}$ metric depends on is deferred to the threshold and relative-ratio rule of Chapter 9. The chapter covers the choice of trees, the hyperparameters, label construction, grouped out-of-fold (OOF) validation, early stopping and refit, gain importances, and the interface a replacement model must honour.

8.1 Why gradient-boosted trees for pairwise entity resolution

A pairwise matcher receives one row per (S1, S2/S3) candidate pair and outputs the probability that the two records describe the same business. The feature row of Chapter 7 is heterogeneous by design: rapidfuzz scores on 0-100, TF-IDF cosines on 0-1, Jaccard indices, character lengths, integer ranks and counts, binary flags such as `n_acronym`, and ternary agreement states (`postal_state`, `house_state`) whose values 1, 0 and -1 mean "agree", "unknown" and "conflict". A gradient-boosted decision tree (GBDT) ensemble consumes such a mix without scaling, one-hot encoding or imputation, because every split is a threshold on one raw column.

The decisions that matter for the metric are interactions, not marginal effects. A moderate name score (`n_tset` around 70) is strong evidence when `postal_state = 1` and `num_conflict = 0`, and weak evidence when `postal_state = -1`; a negative `cos_full_gap1` changes the meaning of every absolute similarity, since a pair dominated by a competitor of the same S1 record is far more often a near-duplicate of a different business than a second true record. Trees express such conditional rules natively; a linear model needs hand-built cross terms. Because $F_{0.5}$ weights precision twice as heavily as recall and a singleton scores 1.0 only with an empty prediction (see Chapter 2), the matcher must separate "similar but wrong" from "similar and right", a separation carried by conflicts and context interactions.

The remaining reasons are practical. Positives are a small minority of the blocked pairs (the measured rate is recorded in Table 8.3), so a model that converges in a few hundred trees on CPU (`n_jobs=-1`) is preferable to one needing GPU epochs. LightGBM is MIT-licensed, and at most 2,000 trees of 63 leaves amount to on the order of 10^5 split thresholds and leaf values, negligible against the 8B parameter cap (see Chapter 13). Its per-feature gain also keeps the feature layer auditable when a rule of Chapter 5 changes.

8.2 Hyperparameters

The parameters are set once, in the `params` dictionary of `main()`, and shared by the five cross-validation fits and the final refit. Table 8.1 lists each with its role. They are conservative defaults for a tabular binary classifier of this size, not values tuned on the released data.

Table 8.1 — LightGBM hyperparameters in `params` and the rationale for each value

Parameter	Value	Role and rationale
<code>objective</code>	<code>binary</code>	Log-loss on $y \in \{0, 1\}$; the leaf outputs are log-odds, so <code>predict()</code> returns a calibrated probability that the threshold sweep of Chapter 9 can act on.

Parameter	Value	Role and rationale
learning_rate	0.05	Shrinkage per tree. Small enough that early stopping lands on a smooth loss minimum rather than oscillating between rounds; combined with the 2,000-round cap it leaves ample capacity.
num_leaves	63	Maximum leaves per tree (leaf-wise growth). 63 allows a tree to express a six-way interaction such as name score × postal state × number conflict × gap, which is the depth the data noise demands.
min_child_samples	40	Minimum pairs in a leaf. Prevents leaves that memorise a single S1 record's candidate set; with positives being rare this also stops a leaf from being carved around a handful of positives.
feature_fraction	0.8	Each tree sees a random 80% of the 38 columns, decorrelating trees and forcing the ensemble to use secondary evidence (address, alias, acronym features) instead of leaning only on cos_full.
bagging_fraction / bagging_freq	0.8 / 1	Each tree is fitted on a fresh random 80% of the rows (resampled every round). Reduces variance on the rare positive class.
lambda_l2	1.0	L2 penalty on leaf values; damps extreme log-odds on small leaves so that probabilities near the tuned threshold remain smooth.
verbose / n_jobs	-1 / -1	Silent training; all CPU cores.
rounds / patience	2000 / 100	Passed to <code>lgb.train()</code> and <code>lgb.early_stopping()</code> , not to <code>params</code> ; see Section 8.5.

No `seed` is fixed, so bagging and feature subsampling make consecutive runs differ in the last decimals of the probabilities; adding `seed` (together with `deterministic=True` and a fixed thread count, since `n_jobs=-1` leaves histogram construction order to the scheduler) to `params` makes runs reproducible (see Chapter 11).

8.3 Label construction and class balance

Labels exist only for the training split and come from the ground truth loaded by `load_split()`, which maps each S1 `entity_id` to its set of matched S2/S3 ids. After blocking and feature computation, `main()` sets `y = 1` for a candidate row if and only if the S2/S3 id belongs to the ground-truth set of its S1 id:

```
ftr["y"] = [int(id2[b] in gt.get(id1[a], ()))
            for a, b in zip(ftr.i1.values, ftr.i2.values)]
```

Two consequences follow. The model is trained only on pairs that blocking produced: a true pair no view retrieved is neither positive nor negative, and its loss is charged to the blocking recall ceiling of Chapter 6. And the negatives are the hard negatives blocking found plausible, exactly the distribution met at test time.

The classes are imbalanced: `block()` can attach up to $25 + 25 + 10 + 10 = 70$ candidates to one S1 record before de-duplication (the union of four top-k views, Chapter 6), whereas the ground truth lists zero, one or a few matches per record, so positives are a small minority; the exact positive rate is a run-log quantity (Table 8.3). The pipeline deliberately applies no reweighting, neither `is_unbalance`, `scale_pos_weight` nor sample weights. Reweighting shifts the probability scale, and the decision layer of Chapter 9 never uses 0.5 as a boundary; it sweeps the absolute threshold from 0.20 to 0.95 and a relative ratio directly on the OOF macro $F_{0.5}$. What the sweep needs is a well-ordered, calibrated probability, which unweighted log-loss provides; reweighting would instead inflate probabilities on ambiguous pairs, the opposite of what a precision-weighted metric with an all-or-nothing singleton term rewards.

8.4 Grouped cross-validation and out-of-fold assembly

The held-out probabilities estimate the leaderboard score and are the data on which the threshold and ratio are selected, so validation rows must come from S1 records the model never saw. A plain random split violates this: the candidates of one S1 record would be scattered across folds, so the model would predict one of them having learned the others. The context features of Chapter 7 make the leak worse, since `cos_full_gap1`, `rank1` and `n_cand1` encode the whole candidate set of a record.

`main()` therefore uses `GroupKFold(args.folds)`, five folds by default, with `groups=ftr.i1`: every candidate of an S1 record, and hence every positive and negative competing for it, lands in the same fold. Because the decode of Chapter 9 operates per S1 record, the fold structure also makes the OOF macro $F_{0.5}$ a proper per-entity estimate, including singleton records, which contribute only negatives and are the entities most exposed to a false positive. Figure 8.1 shows the mechanism.

ftr: candidate rows, grouped by i1 (all pairs of one S1 stay together)

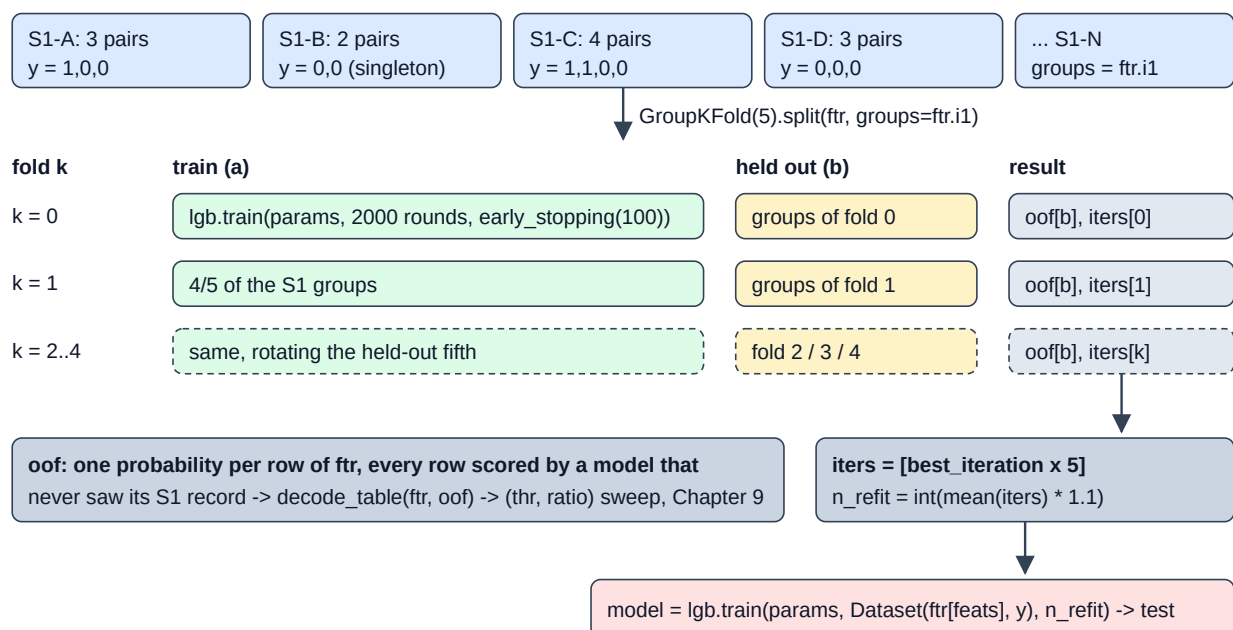


Figure 8.1 — GroupKFold out-of-fold scheme in `main()`. Rows are grouped by `i1`, each fold trains on four fifths of the S1 groups with early stopping on the fifth, the held-out predictions fill `oof`, the five best iterations set the refit length, and the refit model scores the test pairs.

Each fold calls `lgb.train()` with a training `lgb.Dataset` on `ftr.iloc[a][feats]` and a validation set on `ftr.iloc[b][feats]`. The held-out slice is scored with `m.predict(..., num_iteration=m.best_iteration)` and written into `oof[b]`; the five index sets `b` partition the rows, so `oof` ends as a full-length vector aligned with `ftr`, which `decode_table()` consumes. Context features grouped by `i2` (`cos_full_gap2`, `rank2`, `n_cand2`) do span folds, but they derive from unsupervised similarities only and carry no label information.

8.5 Early stopping, best iteration and the refit

Each fold trains for at most 2,000 boosting rounds under `lgb.early_stopping(100, verbose=False)`, which monitors the validation binary log-loss and halts once it has not improved for 100 consecutive rounds; `m.best_iteration` is the round with the lowest validation loss, used by `predict()` for the OOF slice,

appended to `iters` and printed as `fold k: best_iter=...`. Log-loss rather than the competition metric is monitored on purpose: the metric depends on a threshold not yet chosen, whereas log-loss selects the round at which probabilities are best calibrated across the range the sweep explores.

After the sweep, the final model is refitted once on every training pair with a fixed `int(np.mean(iters) * 1.1)` rounds. No held-out data remain, so early stopping is unavailable and the length is transferred from cross-validation: the mean best iteration, scaled by 1.1 because a model fitted on all groups rather than four fifths benefits from slightly more rounds, truncated to an integer. The refit reuses `params`, so the probability scale on which `best_thr` and `best_ratio` were selected is preserved as closely as a refit permits. Test pairs are scored with `model.predict(fte[feats])`, where `fte` comes from the same `block()` and `features()` path (see Chapter 4).

Risk. The threshold and ratio are selected on the same OOF probabilities that report the validation score, over $31 \times 6 = 186$ configurations, so the printed OOF macro $F_{0.5}$ is mildly optimistic; Chapter 10 discusses a nested check. Separately, the refit model has seen no French record and trees cannot extrapolate beyond the training value ranges; the bounded similarities of Chapter 7 limit this exposure, and the per-country match-rate log line for the test split is the only in-pipeline signal for it.

8.6 Reading the gain importances

After the refit, `main()` computes `model.feature_importance("gain")`, indexes it by `feats`, sorts it descending and prints the top 15. Gain is the total reduction of the training objective attributed to splits on a feature, summed over all trees; it has no sign and is not a coefficient. Three readings are useful. High gain for `cos_full`, `n_tset` or `cos_full_gap1` confirms that name similarity and its context dominate. Gain on the ternary and conflict features (`postal_state`, `house_state`, `num_conflict`) shows the model separating near-duplicates by address evidence, where precision is won. Near-zero gain, for instance on `n_acronym` or `has_alias`, means the feature is rare or absorbed by another; it is a removal candidate only once the OOF score is shown not to move. Measured values belong in Table 8.3 (Section 8.8).

8.7 Model interface and how to swap the model

The matcher is coupled to the pipeline through the narrow contract of Table 8.2. Any model that fits on `ftr[feats]` with labels `ftr.y`, respects the grouped fold indices and returns one probability per row can replace LightGBM without touching blocking, features or the decision layer, provided OOF and test predictions share one scale, since `best_thr` and `best_ratio` are selected on the former and applied to the latter.

Table 8.2 — Model interface contract as used by `main()`; a replacement must satisfy every row

Aspect	Contract	Where it is consumed
Training input	<code>ftr[feats]</code> , a DataFrame of 38 numeric columns (all columns except <code>i1</code> , <code>i2</code> , <code>y</code>), no NaN; label <code>ftr.y</code> $\in \{0, 1\}$	<code>lgb.Dataset</code> in the fold loop and the refit
Fold protocol	Fit on index set <code>a</code> , predict index set <code>b</code> , for the five GroupKFold splits grouped by <code>i1</code> ; write predictions into <code>oof[b]</code>	<code>decode_table(ftr, oof)</code> , sweep of Chapter 9
Capacity transfer	Expose a per-fold capacity value (here <code>best_iteration</code>) so the refit can be sized as $1.1 \times \text{mean}$; a model without early stopping may fix its capacity and skip this	<code>iters</code> , refit call
Output	<code>predict(X)</code> returns a float array of length <code>len(X)</code> in $[0, 1]$, row-aligned with <code>X</code> , monotone in match confidence and on one scale across <code>S1</code> groups (the ratio rule compares probabilities within a group)	<code>decode_table()</code> , <code>decide()</code>

Aspect	Contract	Where it is consumed
Test scoring	Same <code>predict()</code> on <code>fte[feats]</code> from the refit model; no test labels, no external data	<code>matches</code> , <code>matching_results.tsv</code>
Introspection (optional)	Per-feature importance for the top-15 log	diagnostics only
Constraints	MIT or Apache-2.0 licence, $\leq 8B$ parameters, CPU-feasible on the candidate count, no lookups	Chapter 13

A swap replaces the two `lgb.train()` calls and the two `predict()` calls; the fold loop, the `oof` assembly, the sweep and the output writers remain. A model emitting unbounded scores must first map them into $[0, 1]$, by a sigmoid or a rank normalisation over all pairs, because the 0.20-0.95 sweep grid assumes that range.

Design note. Alternatives considered. A logistic regression on the same 38 features is the fastest option and is well calibrated, but it is additive: it cannot express that `postal_state = -1` reverses the meaning of a high `n_tset` unless cross terms and one-hot encodings of the ternary states are added by hand, which costs precision at the singleton-heavy operating point. A pairwise transformer cross-encoder, for instance a DistilBERT-class (Apache-2.0) or MiniLM-class (MIT) multilingual encoder with tens to a few hundred million parameters (far below the 8B cap), fine-tuned on the concatenated "name | address" strings of the blocked training pairs, would learn abbreviations and idioms the tables of Chapter 5 miss and would likely transfer better to French text, at the cost of a GPU for fine-tuning and for scoring every blocked pair, an opaque error profile, and the loss of the context features unless re-attached. The pragmatic combination, noted in Chapter 14, keeps LightGBM as the decision model and feeds it a cross-encoder score as one more column, or re-ranks only pairs near the tuned threshold, so the calibrated, auditable tree model remains the interface to Chapter 9.

8.8 Quantities recorded from the run

Table 8.3 collects the training-time quantities the matcher prints. None is estimated here; the pipeline has not yet been executed on the released data, and each cell is filled from the run log (see Chapter 10).

Table 8.3 — Training quantities produced by the matcher; measured values to be filled from the run log (the pipeline has not yet been executed on the released data)

Quantity	Log line / source	Measured
Training pairs and positives	<code>blocking recall ceiling line (pos / true)</code>	to be filled from the run log
Positive rate	<code>pos / ftr </code>	to be filled from the run log
Best iteration per fold ($k = 0 \dots 4$)	<code>fold k: best_iter=</code>	to be filled from the run log
Refit rounds, $\text{int}(1.1 \times \text{mean})$	derived from the five values above	to be filled from the run log
OOF macro $F_{0.5}$, selected thr and ratio	<code>OOF macro F0.5 = ... at thr=... ratio=...</code>	to be filled from the run log
Top-15 features by gain	printed after the refit	to be filled from the run log
Training wall time of the five folds and refit	timestamps of the log lines	to be filled from the run log

9 Decision Layer: From Probabilities to Match Lists

The matcher of Chapter 8 emits one probability per candidate pair; the leaderboard scores one list per Source 1 (S1) entity. This chapter describes the layer between the two: `decode_table()`, which assigns each Source 2 / Source 3 (S2/S3) record to the S1 entity that scores it highest and records the best probability per (S1, source) group; `decide()`, which keeps a pair only if it clears an absolute threshold `thr` and a relative bar `ratio · best`; the 186-point out-of-fold (OOF) grid that selects both parameters by maximising macro $F_{0.5}$ including singletons; and `to_lists()` with the TSV writer. It derives from the metric why a second candidate must be nearly as confident as the first to be worth adding.

9.1 Role of the decision layer

The metric is macro-averaged over S1 entities (see Chapter 2): each contributes one $F_{0.5}$ computed from its predicted and ground-truth lists, and a singleton, an S1 entity with no true match, contributes 1.0 if and only if its predicted list is empty. The matcher is trained with a pairwise log-loss and knows nothing about lists, singletons or the precision-recall asymmetry; all of that is handled after the model, in `decode_table()`, `decide()`, `to_lists()` and the writer loop of `main()`. Because the metric logic sits in a small parameterised layer, the expensive stages run once and the operating point is re-selected cheaply from stored OOF probabilities. Figure 9.1 shows the flow.

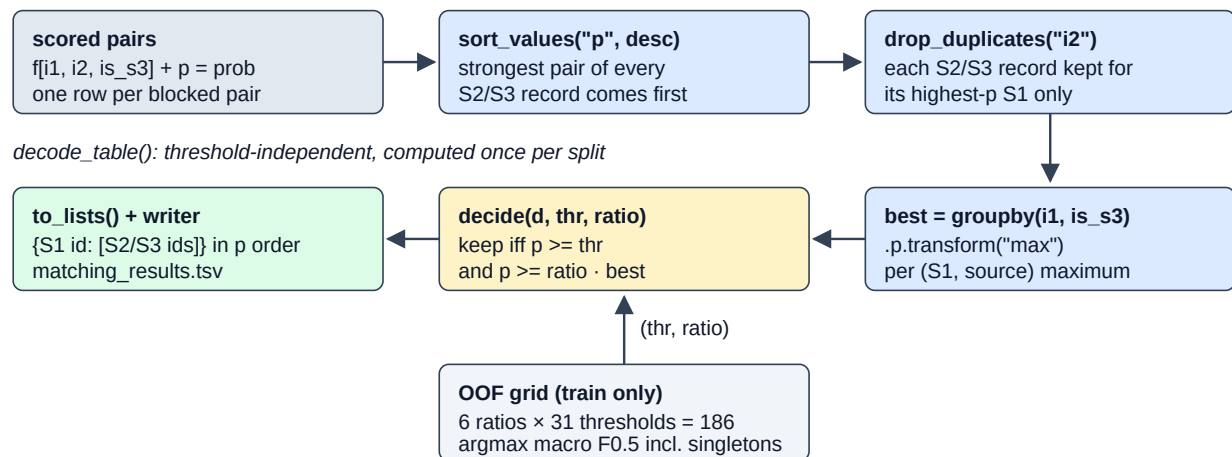


Figure 9.1 — Decode flow: `decode_table()` sorts, de-duplicates per S2/S3 record and computes the per-(S1, source) maximum once; `decide()` applies the two-part rule with parameters from the training OOF grid; `to_lists()` and the writer produce one line per S1 entity.

9.2 `decode_table()` : assignment and group maxima

The function takes the feature table `f` and the probability vector `prob` (OOF probabilities in training, the refitted model's predictions at test time) and returns a reduced table with columns `i1`, `i2`, `is_s3`, `p` and `best`:

```
def decode_table(f, prob):
    d = f[["i1", "i2", "is_s3"]].copy()
    d["p"] = prob
    d = d.sort_values("p", ascending=False).drop_duplicates("i2").reset_index(drop=True)
    d["best"] = d.groupby(["i1", "is_s3"]).p.transform("max")
    return d
```

9.2.1 One S1 entity per S2/S3 record

Sorting by `p` descending and calling `drop_duplicates("i2")` keeps, for every S2/S3 row index `i2`, only the highest-probability pair. The justification is structural: Source 1 is the de-duplicated reference source, so two S1 entities are two different businesses, and one S2/S3 record can belong to at most one of them. Blocking (see Chapter 6) routinely retrieves the same S2/S3 record for several S1 entities (chain branches, franchises, common names); at most one can be right, every other is a false positive, and on a singleton it costs the full 1.0.

The step is a greedy assignment: when the matcher ranks the correct S1 entity highest, the competing false positives vanish at zero recall cost; when it ranks a wrong entity highest, the true owner loses the record to a false positive that a flat threshold would have produced anyway. The net effect is positive whenever the ranking across S1 entities beats chance, which the cross-record features `cos_full_gap2`, `rank2` and `n_cand2` of Chapter 7 support.

9.2.2 The per-(S1, source) maximum

After de-duplication, `best` is the maximum probability among the surviving pairs of the same S1 entity and source (S2 or S3, read from `is_s3`); a record claimed by another S1 entity no longer counts towards its former entity's maximum, so the relative rule compares each pair only against competitors that are actually available. Grouping by source rather than by entity alone reflects the ground truth: one business can legitimately appear once in Source 2 and once in Source 3, so a strong S2 candidate must not suppress a moderately confident S3 candidate. Within one source the problem statement allows several records per business; how common that is, reported by `profile()` as the mean and maximum match count, is what the OOF sweep of Section 9.5 weighs when it sets the strictness of the relative rule.

9.3 `decide()` : an absolute and a relative bar

```
def decide(d, thr, ratio=0.0):
    return d[(d.p >= thr) & (d.p >= ratio * d.best)]
```

A pair survives if its probability is at least `thr` and at least `ratio` times the best probability in its (S1, source) group. The best pair of every group satisfies the second condition trivially, so the relative bar only ever removes secondary candidates. `ratio = 0` makes it vacuous and reproduces a flat per-pair threshold; `ratio = 1.0` keeps at most one pair per (S1, source) group, barring exact ties; the values between interpolate.

A relative rule suits a per-entity metric because the cost of a candidate depends on what the entity already predicts. Take an S1 entity whose best S2 candidate has `p = 0.95` and whose second has `p = 0.55`. A flat threshold of 0.5 keeps both; `ratio = 0.7` drops the second, since $0.55 < 0.665$. Such a candidate may be a genuine second record or a similarly named different business, and Section 9.4 shows that the metric rewards keeping it only when its probability is well above one half. An entity whose sole candidate has `p = 0.55` keeps it under both rules, because there is no stronger competitor to be dominated by. The rule thus prunes weak secondary candidates in crowded groups while sparing lone, moderately confident ones, which one global threshold cannot express.

9.4 Metric arithmetic behind the rule

Let an S1 entity have $n > 0$ true matches. With P predicted ids of which tp are correct, precision is tp / P , recall is tp / n , and $F_{0.5} = 1.25 \cdot P \cdot R / (0.25 \cdot P + R)$, as implemented in `f05()`. Two consequences shape the decision layer.

9.4.1 Singleton economics

For a singleton the score is binary: 1.0 for an empty prediction, 0.0 for any other, so a false merge on a singleton is the largest single loss available, whereas a missed match on a non-singleton loses only a fraction of its score; the singleton share logged by `profile()` sets the price of a lenient threshold. A two-outcome model makes this explicit: if an entity's only candidate is true with probability a (in which case the entity has exactly that one match) and, when false, the entity is a singleton with probability b , predicting it has expected score a and predicting nothing has expected score $(1 - a) \cdot b$. The candidate pays only when $a \geq b / (1 + b)$: 0.50 when a wrong candidate always means a singleton, 0.33 when it does so half the time. The sweep of Section 9.5 replaces these assumptions with the empirical OOF distribution, but the direction is fixed: more singletons, higher optimal `thr`.

9.4.2 Break-even for a second candidate

Suppose the entity already predicts one correct candidate and a second with probability q is under consideration. One correct prediction gives $P = 1$, $R = 1/n$ and $F_{0.5} = 5 / (n + 4)$. Adding the second gives $P = 1$, $R = 2/n$ and $F_{0.5} = 10 / (n + 8)$ if it is true, or $P = 1/2$, $R = 1/n$ and $F_{0.5} = 5 / (n + 8)$ if it is false. The expected score with two predictions, $5 \cdot (1 + q) / (n + 8)$, exceeds $5 / (n + 4)$ if and only if

$$q \geq 4 / (n + 4).$$

Table 9.1 evaluates this for small n . With one true match the second candidate cannot be true and always costs 0.444; with two it must be at least 0.667 likely to pay for itself; the bar reaches 0.5 only at four or more true matches. Since n is unknown at decode time, and to the extent that the mean match count logged by `profile()` shows most non-singleton entities to have few matches, a second candidate should be admitted only when nearly as confident as the first, which `ratio` values of 0.6 to 0.9 enforce against a strong `best`.

Table 9.1 — Expected $F_{0.5}$ when a second candidate of probability q is added to one correct prediction, by number of true matches n (exact values of the formulas in Section 9.4.2)

n	$F_{0.5}$, one correct	Two, second true	Two, second false	Expected with two	Break-even q
1	1.000	impossible	0.556	0.556	none: always -0.444
2	0.833	1.000	0.500	$0.500 + 0.500 \cdot q$	0.667
3	0.714	0.909	0.455	$0.455 + 0.455 \cdot q$	0.571
4	0.625	0.833	0.417	$0.417 + 0.417 \cdot q$	0.500
6	0.500	0.714	0.357	$0.357 + 0.357 \cdot q$	0.400

Risk. The derivation conditions on the first candidate being correct. Entities with many records in one source ($n \geq 4$) are penalised by a high `ratio`, since their second and third records may score well below the first. The sweep balances this against the singleton and one-match entities, whose share `profile()` reports; if many-record entities were more frequent in the test set than in training, the selected `ratio` would be too strict for them.

9.5 Out-of-fold selection of (thr, ratio)

`main()` builds `dtr = decode_table(ftr, oof)` from the grouped-fold OOF probabilities of Chapter 8 and evaluates every combination of `ratio` in `{0.0, 0.6, 0.7, 0.8, 0.9, 1.0}` and `thr` in `np.arange(0.20, 0.96, 0.025)`, 31 thresholds from 0.20 to 0.95, for 186 evaluations:

```
best_thr, best_ratio, best = 0.5, 0.0, -1.0
for ratio in (0.0, 0.6, 0.7, 0.8, 0.9, 1.0):
    for thr in np.arange(0.20, 0.96, 0.025):
        s = macro_f05(to_lists(decide(dtr, thr, ratio), tr1, tr2), gt, tr1.entity_id.values)
        if s > best:
            best_thr, best_ratio, best = float(thr), ratio, s
```

The objective is `macro_f05()` over every training S1 entity: `load_split()` parses a singleton's empty `matched_entity_ids` field into an empty set, and `gt.get(k, set())` covers any S1 entity absent from the ground-truth file, so the all-or-nothing singleton term of Section 9.4.1 is part of what is maximised; tuning on matched entities only would push `thr` down and over-predict on singletons. Because `ratio = 0` is in the grid, the two-parameter rule can never score below the best flat threshold on OOF data. The comparison is strict (`s > best`) and the loops run `ratio` outermost from 0.0 and `thr` ascending, so on an exact tie the lowest `ratio` and, within it, the lowest `thr` win. Each evaluation re-runs only the filter, the list conversion and the scorer.

Illustrative OOF macro F0.5 surface over the (thr, ratio) grid (schematic shading, not measured)

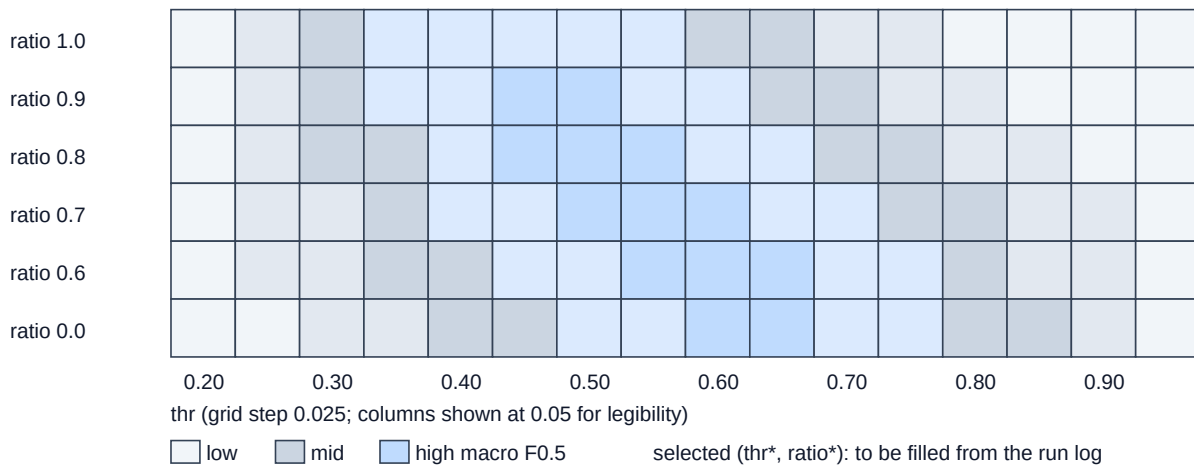


Figure 9.2 — Schematic of the (thr, ratio) grid searched by `main()`: six `ratio` rows and 31 `thr` columns (0.20 to 0.95, step 0.025). Shading is illustrative only: low thresholds lose singletons to false merges, high thresholds lose recall, and a larger `ratio` shifts the ridge left. The actual surface and argmax come from the run log.

Table 9.2 — Selected operating point and decode statistics, to be filled from the run log

Quantity	Source in log	Value
Selected thr	OOF macro F0.5 = ... at thr=...	to be filled from the run log
Selected ratio	same line, ratio=...	to be filled from the run log
OOF macro $F_{0.5}$ at the optimum (singletons included)	same line	to be filled from the run log
Macro $F_{0.5}$ with all candidates predicted (floor)	macro F0.5 if every candidate were predicted	to be filled from the run log

Quantity	Source in log	Value
Blocking-oracle macro $F_{0.5}$ (ceiling)	blocking oracle macro $F_{0.5}$	to be filled from the run log
Test S1 entities with ≥ 1 match, overall and by country	test: .../... S1 entities matched; by country	to be filled from the run log

9.6 `to_lists()`, ordering and the writer

```
def to_lists(d, s1, other, col="i2"):
    out = {e: [] for e in s1.entity_id}
    ids1, ids2 = s1.entity_id.values, other.entity_id.values
    for a, b in zip(d.i1.values, d[col].values):
        out[ids1[a]].append(ids2[b])
    return out
```

`to_lists()` converts a table of row indices into a dictionary from S1 `entity_id` to a list of S2/S3 ids. It is pre-populated with an empty list for every S1 entity, so an entity with no surviving pair is present as a singleton rather than missing, as the one-row-per-entity rule requires. The decoded table is still in descending `p` order after `decide()`, so each match list runs from the most to the least confident id. For the candidate file, `main()` passes the unfiltered `fte[["i1", "i2"]]`, so those lists follow blocking order and matches are a subset of candidates by construction.

The writer in `main()` iterates over `tel.entity_id.values` in source-file order and emits a header and one tab-separated line per entity; `col` is `matched_entity_ids` for `matching_results.tsv` and `candidate_entity_ids` for `candidate_pairs.tsv`, the column names required by the problem statement:

```
fh.write(f"source1_entity_id\t{col}\n")
for e in tel.entity_id.values:
    fh.write(f"{e}\t{' '.join(dict.fromkeys(lists[e]))}\n")
```

`dict.fromkeys()` removes duplicate ids while preserving order, which a `set` would not. Duplicates cannot in fact arise, since `drop_duplicates("i2")` leaves each S2/S3 id at most once in the whole matching file and `block()` de-duplicates (`i1`, `i2`) pairs before features are computed; the call guards the rejection rule "no duplicate entity IDs within a single ID list" (see Chapter 2) against upstream changes. An empty list renders as the id, a tab and nothing else, the required singleton format; `newline=""` keeps line endings at exactly `\n`. Chapter 11 covers packaging.

Design note. A single global (`thr`, `ratio`) is selected on US and Indian OOF data and applied unchanged to France, whose probability distribution the model has never seen. Two extensions follow. Per-source parameters would let S2 and S3 differ in strictness, today reflected only in the `is_s3` matcher feature and the per-source grouping of `decode_table()`: the grid becomes (`thrS2`, `thrS3`, `ratio`) inside the same OOF loop. Per-country parameters would be selected for US and India separately and, for an unseen country, replaced by the more conservative of the two, since a false merge costs more than a missed link. Both add conditions to `decide()` only; the per-country match rate logged by `main()` is the check for France (see Chapter 14).

10 Validation Strategy and Tooling

The test set carries no labels, so every number that guides a design decision comes from the training split. This chapter describes the grouped out-of-fold (OOF) protocol inside `main()` and its diagnostics, the stand-alone scorer `utils/score.py`, the addition needed to feed it a train-side prediction file, the validator `utils/validate_submission.py` and every rule it enforces, an error-analysis playbook, and the pre-upload checklist. Throughout, macro $F_{0.5}$ with all-or-nothing singletons is measured exactly as the organisers compute it, on entities the model has never seen.

10.1 Validation protocol

10.1.1 Grouped out-of-fold predictions

The problem statement asks participants to hold out part of the training data and score it with the $F_{0.5}$ formula (see Chapter 2). The pipeline uses K-fold cross-validation instead of a single hold-out, so that every training pair is scored by a model that never saw it. `main()` splits the labelled candidate table `ftr` with `GroupKFold(args.folds)`, five folds by default, grouped by `ftr.i1`, the S1 row index. Grouping by S1 entity is essential: the contextual features of Chapter 7 (`cos_full_gap1`, `rank1`, `n_cand1` and their siblings) and the decision rule of Chapter 9 both act on the whole candidate set of one S1 entity. Spreading that set across folds would overstate what happens on the test set, where every S1 entity is new.

Within each fold, LightGBM trains for at most 2,000 rounds with early stopping after 100 rounds without improvement of the held-out log-loss (see Chapter 8). The held-out predictions at `best_iteration` fill the vector `oof`; the final refit uses `int(np.mean(iters) * 1.1)` rounds, so the OOF run also fixes the capacity of the deployed model.

10.1.2 Scoring the OOF predictions with the official metric

The OOF probabilities are decoded exactly as test probabilities are: `decode_table(ftr, oof)` assigns each S2/S3 record to its highest-probability S1 and records the per-(S1, source) maximum, and `decide()` applies the absolute threshold `thr` and the relative bar `ratio · best`. For each of the 186 grid points (six ratios in {0.0, 0.6, 0.7, 0.8, 0.9, 1.0}, thirty-one thresholds from 0.20 to 0.95 in steps of 0.025), `to_lists()` builds per-entity id lists and `macro_f05()` averages `f05()` over `trl.entity_id.values`. The key set is every training S1 entity, not only those with candidates, so an entity whose true matches were never blocked contributes 0.0 and singletons contribute 1.0 or 0.0: blocking loss and the singleton penalty are inside the estimate. `f05()` implements the challenge rule literally: empty truth scores 1.0 if and only if the prediction is empty, an empty prediction against non-empty truth scores 0.0, zero true positives score 0.0, otherwise $F_{0.5} = 1.25 \cdot P \cdot R / (0.25 \cdot P + R)$. The best grid point is logged as `OOF macro F0.5 = ... at thr=... ratio=...`, the primary estimate of the leaderboard score.

Design note. The OOF number is a tuned score, not an untouched hold-out: (`thr`, `ratio`) is chosen by maximising it over 186 points. With only two free parameters selected against the full set of training S1 entities the optimistic bias is expected to be small, but the public leaderboard should be expected to sit slightly lower even for US and India, before any effect of the unseen French records.

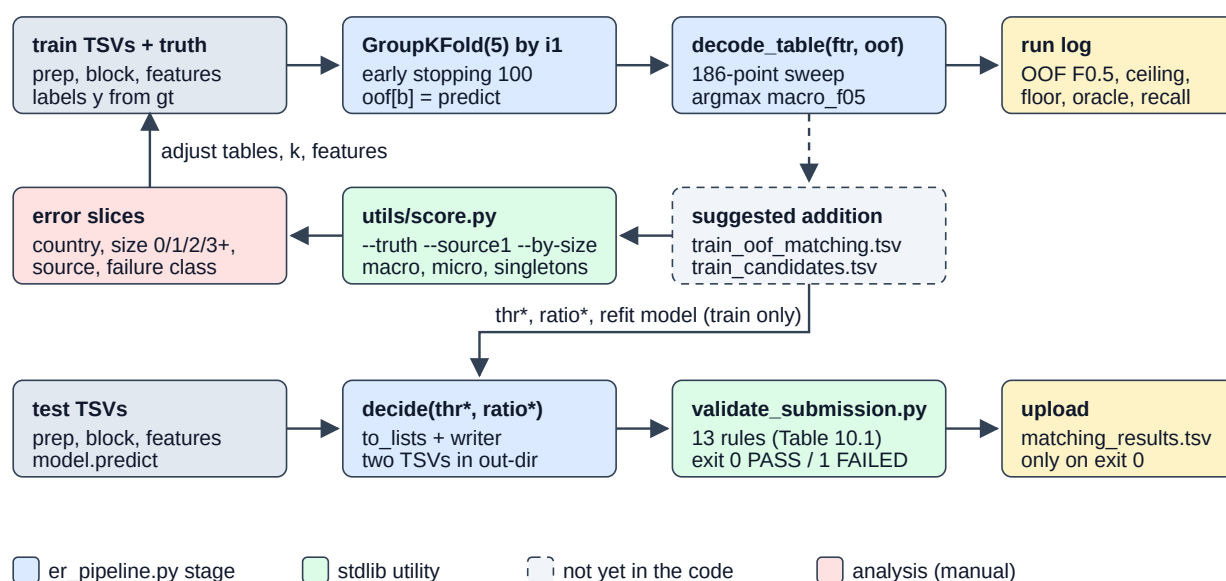
Two things lie outside this protocol. The TF-IDF vectorisers and the contextual features are computed on the full training split before folding; they use no labels, and `block()` refits the vectorisers on test text in the same way, so this mirrors inference rather than leaking. France has no labels, so the only check for the unseen country is the

per-country fraction of test S1 entities that received a match, printed as the last line of `main()` (see Chapter 14).

10.2 Diagnostics printed by `main()`

Every quantity a design decision depends on is written by `log()` with a wall-clock prefix (only the feature-importance table and the data previews of `profile()` go through a plain `print()`), so one run log is a complete validation record. Figure 10.1 places these lines in the loop that connects them to the two utilities; Table 10.4 lists the values to be recorded once the pipeline has run.

- `profile()`: record counts, country distributions and, for train, the singleton fraction and the mean and maximum matches per entity. With singleton fraction s , merging one in every hundred singletons removes $s/100$ from the macro average.
- `block()`: candidate pairs and pairs per S1 entity, from which the reduction ratio of Chapter 6 follows.
- The *blocking recall ceiling*, the number of non-singleton S1 entities with at least one true match blocked (the macro metric charges 0.0 for every such entity with none), and recall by country and by source.
- One `fold k: best_iter=...` line per fold; widely differing values indicate an unstable fit.
- The OOF macro $F_{0.5}$ line, the *floor* (every candidate predicted) and the *oracle ceiling* (exactly the true candidate pairs). OOF-to-oracle is what the matcher and decision layer leave on the table; oracle-to-1.0 is what only better blocking can recover.
- The fifteen highest gain importances of the refit model, and the test-side match count with the per-country match rate.



Test labels are never read; only `thr*`, `ratio*` and the refit model cross from the top row to the bottom row.

Figure 10.1 — The validation loop. Top row: the training-side estimate produced by `main()`; dashed box: the suggested train-side prediction file that lets `utils/score.py` break the OOF score down; bottom row: the test-side path that ends in `utils/validate_submission.py` and, only on exit code 0, an upload.

Table 10.4 — Validation quantities to be filled from the run log (the pipeline has not yet been executed on the challenge data; no value below is measured).

Log line	Quantity	Value
----------	----------	-------

Log line	Quantity	Value
<code>== train:</code>	S1 / S2+S3 counts, singleton fraction, mean and max matches	to be filled from the run log
<code>blocking:</code>	candidate pairs, pairs per S1 (train and test)	to be filled from the run log
<code>blocking recall ceiling:</code>	true pairs blocked / total true pairs	to be filled from the run log
<code>S1 entities with >=1 true match blocked:</code>	entities not charged 0.0 by blocking	to be filled from the run log
<code>blocking recall by country / by source:</code>	per-country and S2/S3 recall	to be filled from the run log
<code>fold k: best_iter=</code>	five early-stopping iterations	to be filled from the run log
<code>00F macro F0.5 =</code>	primary estimate, thr, ratio	to be filled from the run log
<code>floor / ceiling lines</code>	all-candidates macro $F_{0.5}$; blocking-oracle macro $F_{0.5}$	to be filled from the run log
<code>test: ... matched; by country:</code>	match rate per test country incl. France	to be filled from the run log

10.3 The scorer: `utils/score.py`

`utils/score.py` is a standard-library-only re-implementation of the metric, kept separate from the pipeline so that the two implementations of `f05()` can be checked against each other. It takes `--pred` and `--truth`, both two-column TSVs in the submission format, an optional `--source1` file that supplies each S1 entity's country, and the flag `--by-size`. `read_lists()` skips the header, strips whitespace and treats a blank or absent second column as an empty set; `read_country()` maps a blank country to `UNKNOWN` rather than dropping the entity, consistent with the open-set treatment of Chapter 6.

The scorer iterates over the entities of the `truth` file: an entity missing from the prediction file is scored as empty, and a predicted entity absent from the truth is ignored; both cases print a warning on `stderr`. The pipeline's `macro_f05()` iterates over `tr1.entity_id` instead, so the two agree only when the ground-truth file has a row for every training S1 entity, singletons included. Table 10.2 lists the scorer's output.

Table 10.2 — Outputs of `utils/score.py` and their meaning.

Line	Computation	How to read it
Entities scored	number of truth rows	must equal the S1 count of the split; otherwise the truth file is incomplete
Macro $F_{0.5}$	mean of per-entity <code>f05()</code>	the leaderboard quantity; must reproduce the log's OOF line for the train-OOF file
Micro precision	$tp / (tp + fp)$ over all predicted ids	pair-level view; an entity with many matches weighs many times more than in the macro score
Micro recall	$tp / (tp + fn)$	with micro precision, shows whether loss is precision- or recall-driven
Singleton accuracy	singletons predicted empty / singletons	each miss costs a full 1.0 for that entity; the most expensive error class under the metric
False merges (singl.)	singletons with a non-empty prediction	direct count of the errors the threshold should suppress first

Line	Computation	How to read it
Per-country $F_{0.5}$	macro within each country of -- source1, sorted by size	reveals whether one country dominates the loss; UNKNOWN collects blank labels
$F_{0.5}$ by truth list size	buckets 0, 1, 2, 3+ of the truth list length	bucket 0 is singleton accuracy; a weak 3+ bucket implicates the ratio rule or drop_duplicates("i2")

The micro figures are reported on purpose: under macro averaging an entity with twelve true matches and one with a single match count equally, so micro precision can look healthy while many small entities each lose one pair.

10.4 Producing a train-OOF prediction file (suggested addition)

The pipeline currently writes only the two test-side files. Everything needed for a train-side file already exists in `main()` after the sweep: `dtr` holds the decoded OOF table, `best_thr` and `best_ratio` the operating point, and `cand_lists` the training candidate lists. The addition below, which is **not yet part of `er_pipeline.py`**, writes both under names that cannot be confused with the submission files, reusing the writer loop of the test path.

```
oof_lists = to_lists(decide(dtr, best_thr, best_ratio), tr1, tr2)
for fname, col, lists in [("train_oof_matching.tsv", "matched_entity_ids", oof_lists),
                          ("train_candidates.tsv", "candidate_entity_ids", cand_lists)]:
    path = os.path.join(args.out_dir, fname)
    with open(path, "w", newline="") as fh:
        fh.write(f"source1_entity_id\t{col}\n")
        for e in tr1.entity_id.values:
            fh.write(f"{e}\t{' '.join(dict.fromkeys(lists[e]))}\n")
    log(f"wrote {path}")
# then, from the repository root:
# python3 utils/score.py --pred output/train_oof_matching.tsv \
# --truth dataset/train/train_ground_truth.tsv \
# --source1 dataset/train/train_source1.tsv --by-size
```

Scoring `train_oof_matching.tsv` must reproduce the log's OOF macro $F_{0.5}$ to its four printed digits, provided the truth file has a row for every training S1 entity; any other discrepancy means one of the two `f05()` implementations has drifted. The candidate file lets Section 10.6 separate blocking misses from matcher misses. Neither file belongs in the submission package, and the validator is never pointed at them, because their ids are training ids.

10.5 The validator: `utils/validate_submission.py`

A submission that violates any format rule is rejected and consumes a submission slot, so `utils/validate_submission.py` checks both output files against the test source files before anything is uploaded. `load_test_ids()` reads the three test TSVs, requires each to begin with an `entity_id` header, and builds the set of S1 ids and the set of S2/S3 ids. `validate_file()` parses one submission file on the raw text rather than through a CSV reader, so that quoting characters and padded tokens are detected instead of silently absorbed. Table 10.1 lists every rule.

Table 10.1 — Rules enforced by `utils/validate_submission.py`, in the order they are checked.

#	Rule	Scope	Outcome
1	each test source file starts with an <code>entity_id</code> header	test dir	immediate exit with message (exit 1)
2	submission file is not empty	file	issue

#	Rule	Scope	Outcome
3	header is exactly source1_entity_id TAB matched_entity_ids (or candidate_entity_ids)	file	issue
4	every non-empty row has exactly two tab-separated columns (empty lines are skipped)	row	issue; row skipped for further checks
5	no " or ' anywhere in the line	row	issue
6	no duplicate source1_entity_id	row	issue
7	source1_entity_id exists in test_source1.tsv	row	issue
8	no empty or whitespace-padded token in the id list	token	issue; token skipped
9	every id starts with S2- or S3- (rejects S1 self-matches)	token	issue
10	every id exists in test_source2.tsv or test_source3.tsv	token	issue
11	no id repeated within one list	token	issue
12	every test S1 id has a row (completeness)	file	one issue per missing id
13	matched ids are a subset of the candidate ids of the same S1 entity	cross-check	issue when --candidate is given; warning that the rule was not checked otherwise

After parsing, `print_stats()` writes one line per file with the row count, the entities with at least one id, the total number of ids and the average list length. These are the first sanity check on a new run: a matching file whose average list length exceeds the training mean matches, or whose share of matched entities is far from the training non-singleton fraction, is suspect before any score is known. The program prints `PASS` with exit code 0 when no issue was recorded, or `FAILED: n issue(s) found` with a numbered list capped at `MAX_SHOWN = 50` and exit code 1, so the notebook of Chapter 11 can gate the upload on the exit code alone.

10.5.1 Difference to the official validator

The official helper, as described in the problem statement, treats a matched id that never appeared as a candidate as a *warning*; this validator treats it as an *error*. The pipeline satisfies the subset property by construction, since `decide()` only selects rows of the blocked candidate table and both files are written from the same `fte` frame, so a violation can only come from a packaging bug, files from two different runs or a hand edit, which the problem statement itself calls a pipeline bug. Promoting the check to a failure ensures it cannot be scrolled past. When the candidate file is omitted, the validator prints a `WARNING` rather than a clean pass, because the subset rule is part of what the organisers audit (see Chapter 13).

Risk. A stale or partial copy of the test set under `--test-dir` makes rule 10 fail for every id the copy lacks or, worse, lets rule 12 pass on an incomplete submission. Table 10.3 therefore requires that the validator and the pipeline read the same `dataset/test` directory.

10.6 Error-analysis playbook

With `train_oof_matching.tsv`, `train_candidates.tsv`, the ground truth and `train_source1.tsv` in hand, failures are attributed in a fixed order.

1. **By country.** Run the scorer with `--source1`. If the weaker country's blocking recall in the log also trails, the cause lies in normalisation or blocking (the abbreviation tables, the postal regex `_postal`, the top-k of the address view), not in the model.

2. **By truth size.** Run with `--by-size`. A weak bucket 0 means false merges: raise `thr` or inspect the features that let a singleton's best candidate pass. A weak bucket 1 with healthy blocking recall means the matcher ranks the true record below the threshold. A weak 3+ bucket implicates the relative rule ($p < \text{ratio} \cdot \text{best}$) or the assignment in `decode_table()`.
3. **By source.** The scorer has no source flag; restrict both truth and prediction lists to ids beginning with `S2-` (then `S3-`) and score each restriction. Read with the `blocking recall by source` log line, this shows whether S3's field quality (feature `is_s3`) warrants its own operating point (see Chapter 14).
4. **By failure class.** For each entity scoring below 1.0, classify each missing true id as a *blocking miss* (absent from `train_candidates.tsv`), a *threshold miss* (below `thr`), a *ratio suppression* (below $\text{ratio} \cdot \text{best}$) or an *assignment loss* (given to another S1 by `drop_duplicates("i2")`); classify each spurious id as a *singleton merge* or an *over-inclusion*. The last two are the classes $F_{0.5}$ penalises most and are read next to the raw strings to find the noise pattern responsible (see Chapter 3).

Every change to a table, a top-k or a feature is followed by a full rerun of the OOF loop, because the operating point is re-selected on the new probabilities and a gain in one bucket can move `thr` and cost singletons elsewhere.

10.7 Pre-submission checklist

Table 10.3 — Pre-submission checklist; every item must hold before `matching_results.tsv` is uploaded or the package is zipped.

#	Check	Evidence
1	The run log shows all diagnostics of Table 10.4 and no Python traceback	log file kept with the outputs
2	OOF macro $F_{0.5}$ is not lower than the previous accepted run	OOF macro $F_{0.5}$ = line compared with the run history
3	Scorer on <code>train_oof_matching.tsv</code> reproduces the log's OOF value	<code>utils/score.py</code> output
4	Singleton accuracy and per-country $F_{0.5}$ show no collapse in any slice	scorer breakdowns
5	Test match rate per country, including France, is plausible relative to the training non-singleton fraction	last test: log line
6	<code>validate_submission.py</code> run with <i>both</i> files and the same dataset/test the pipeline read prints PASS (exit 0)	terminal output
7	Statistics lines: rows equal the test S1 count; matching avg list length does not exceed the candidate avg list length	validator [matching] and [candidate] lines
8	Both TSVs come from the same run and were not hand-edited	file mtimes, wrote ... log lines
9	No training-side files (<code>train_oof_matching.tsv</code> , <code>train_candidates.tsv</code>) are inside output/ of the package	zip listing (see Chapter 11)
10	No external data source was read; the only inputs are the train and test TSVs	review of <code>load_split()</code> and <code>find_data_dir()</code> (see Chapter 13)

A submission that fails any item is not uploaded; the OOF estimate read through the breakdowns above, not the leaderboard, is the instrument that guides development.

11 Outputs, Packaging and Reproducibility

The pipeline ends where the organisers' tooling begins: two tab-separated files, a zip archive with a fixed layout, and the claim that anyone can regenerate the files from the training and test data alone. This chapter documents the writer loop in `main()`, the invariants that make both files pass validation by construction, the repository and package layout, the Kaggle notebook cell by cell with its dataset mirrors, and the sources of non-determinism a bit-exact re-run must control. Numbers that come from an actual run are marked as placeholders; nothing measured is reported.

11.1 The output files

11.1.1 The writer code path

Both files are written by one loop at the end of `main()`, after `decide()` has selected the test pairs (see Chapter 9). Its inputs are two dictionaries built by `to_lists()`: `matches`, from the decoded pairs, and `cands`, from `fte[["i1", "i2"]]`, the full test feature table. `to_lists()` starts from `{e: [] for e in s1.entity_id}`, so every test S1 entity owns a key before any pair is appended; an entity with no surviving pair, or no candidate at all, keeps an empty list instead of vanishing from the output.

```
for fname, col, lists in [("matching_results.tsv", "matched_entity_ids", matches),
                          ("candidate_pairs.tsv", "candidate_entity_ids", cands)]:
    path = os.path.join(args.out_dir, fname)
    with open(path, "w", newline="") as fh:
        fh.write(f"source1_entity_id\t{col}\n")
        for e in tel.entity_id.values:
            fh.write(f"{e}\t{''.join(dict.fromkeys(lists[e]))}\n")
        log(f"wrote {path}")
```

Four details carry the format guarantees. The header is a literal f-string with one tab, so the column names are exactly those of the specification; nothing is delegated to `DataFrame.to_csv()`, whose index, quoting and line-terminator defaults would each need overriding. Rows follow `tel.entity_id.values`, the S1 test frame after `prep()`, which calls `reset_index(drop=True)` but never reorders, so output order and row count equal those of `test_source1.tsv`. Each list passes through `dict.fromkeys()`, which keeps insertion order and collapses repeated keys, before the comma join. The file is opened with `newline=""`, which disables platform newline translation, so the explicit `\n` is written verbatim and a Windows run cannot glue a carriage return to the last id of a row.

Duplicates inside a list are already impossible upstream: `block()` ends with `drop_duplicates()` on `(i1, i2)` and `decode_table()` keeps each S2/S3 row for one S1 only, so `dict.fromkeys()` is a guard, not a repair. An empty list joins to the empty string, so a singleton row is the id, a tab and a newline, exactly the third example row of the problem statement.

11.1.2 Row invariants and schema

Table 11.1 lists the rules the official validator enforces (see Chapter 10 for `validate_submission.py`, which mirrors them) and the construct that satisfies each. Entity ids are copied unchanged from frames that `read_tsv()` loads with `dtype=str`, `keep_default_na=False` and `quoting=3`, so an id is never coerced to a number, never becomes `NaN`, and never has quotation characters added or stripped.

Table 11.1 — Output file schema and format rules, with the construct that guarantees each.

Rule (problem statement)	matching_results.tsv	candidate_pairs.tsv	Guaranteed by
Header, tab-separated	source1_entity_id matched_entity_ids	source1_entity_id candidate_entity_ids	literal header f-string in the writer loop
Exactly one row per test S1 entity	rows follow te1.entity_id.values, in test_source1.tsv order		iteration over the S1 frame; to_lists() pre-seeds every id
Empty list for singletons / no candidates	S1-00003	S1-00003	", ".join([]) is the empty string
No duplicate ids within a list	comma-joined, insertion order kept		dict.fromkeys(); upstream drop_duplicates()
Only S2-/S3- ids that exist in the test set	ids come from other.entity_id.values, the concatenation of test_source2.tsv and test_source3.tsv		index-to-id mapping in to_lists()
No quoting characters	ids written verbatim; commas and tabs never appear inside an id		quoting=3 on read, manual write
Matches are a subset of candidates	decoded pairs	every scored pair	decide() filters rows of decode_table(fte, pte)
Line terminator	LF only, on every platform		open(..., newline="") and explicit \n

11.2 candidate_pairs.tsv: the set the model scores

The problem statement defines `candidate_pairs.tsv` as the exact set of records fed to the matching model for inference, not the output of an early blocking pass that is later filtered. The pipeline meets this definition literally. `cte = block(te1, te2)` is the only candidate-generation stage; `fte = features(cte, te1, te2)` starts from `cand.copy()` and only assigns columns, so it neither adds nor removes rows; `pte = model.predict(fte[feats])` scores every row; and `cands = to_lists(fte[["i1", "i2"]], te1, te2)` converts the same rows to id lists. The organisers' recall-ceiling and reduction-ratio analysis of this file therefore describes precisely what the LightGBM model saw.

The subset property follows from the same flow. `decode_table(fte, pte)` takes `fte[["i1", "i2", "is_s3"]]`, attaches probabilities and drops rows via `drop_duplicates("i2")`, but never creates any; `decide()` is a boolean mask over that table. Every row of `matches` is thus a row of `fte`, and the validator's cross-check can fail only when the two files come from different invocations.

Design note. The candidate file is much larger than the match file: each S1 entity carries up to $25 + 25 + 10 + 10 = 70$ candidates before the per-view $sc > 0.05$ cosine cut and the de-duplication of the union (see Chapter 6), against the few that survive `decide()`. Because $F_{0.5}$ is precision-weighted and a singleton scores 1.0 only when predicted empty, the gap between the files is the visible record of how much the matcher and decision layer remove.

11.3 Repository layout and the submission package

The repository keeps the pipeline in the layout the challenge prescribes for the zip, so packaging is a copy rather than a restructuring. `code/business_entity_resolution/` holds `src/er_pipeline.py`, the pipeline `README.md` and `requirements.txt`; `Documentation_template.md` at the root is the methodology write-

up; `utils/` contains the `stdlib`-only validator and scorer plus a synthetic-data generator; `notebooks/kaggle_run.ipynb` is the runner of Section 11.4; `docs/` carries this document; `output/` receives the generated TSVs. The dataset, zip archives and `output/*.tsv` are excluded by `.gitignore`.

The archive is `<team_name>_submission.zip` with the structure of Figure 11.1: the two output files, the self-contained code folder and the methodology document, nothing else. The organisers use it to reproduce results, audit blocking and check fair play and licensing (see Chapter 13), so the code folder must run alone: `er_pipeline.py` imports, beyond the standard library, only NumPy, pandas, scikit-learn, LightGBM and rapidfuzz (SciPy is used indirectly through the sparse TF-IDF matrices), and installs rapidfuzz through `pip` when its import fails.

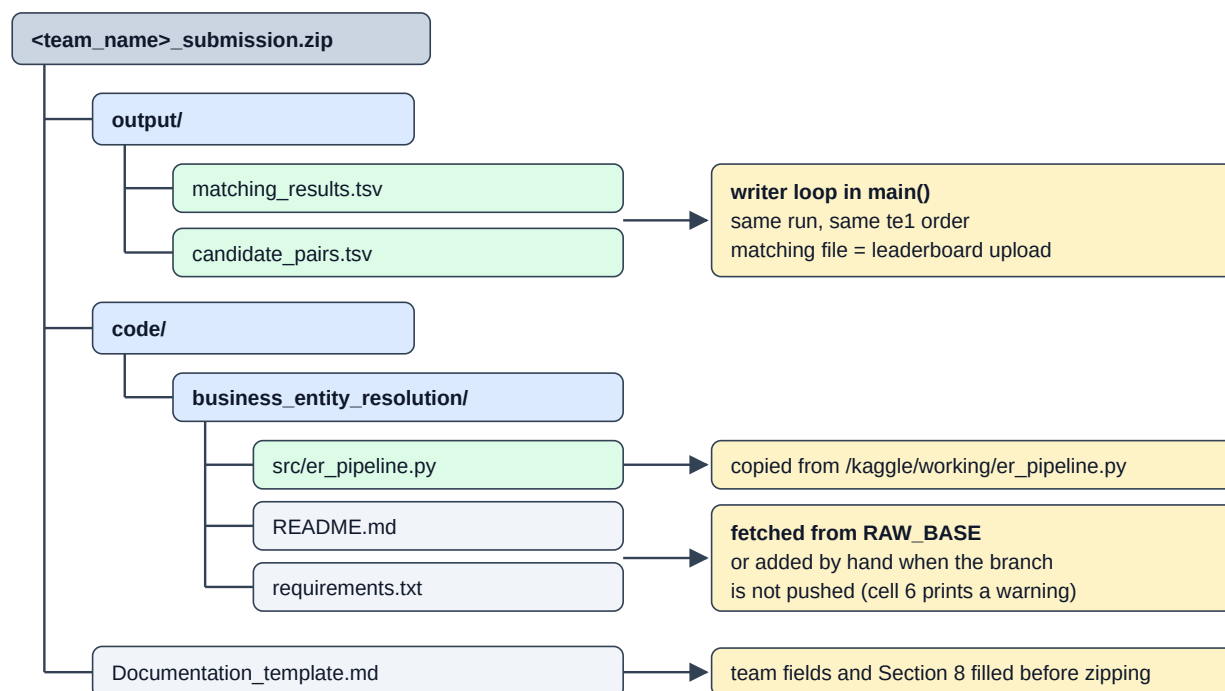


Figure 11.1 — Submission package tree as assembled by cell 6 of `kaggle_run.ipynb` (`shutil.make_archive(PKG, "zip", root_dir=PKG)`), with the origin of each file.

11.4 The Kaggle notebook

`notebooks/kaggle_run.ipynb` is the reference way to produce the package on a machine that has the dataset, the dependencies and enough memory, without cloning the repository. It targets a Kaggle kernel with internet enabled and uses `/kaggle/working` as its only scratch area. Table 11.2 walks through the cells.

Table 11.2 — Cells of `kaggle_run.ipynb` and their purpose.

Cell	Kind	What it does	Failure mode and fallback
0	markdown	Describes the six steps and states that no external data lookup is performed.	—
1	code	<code>pip install -q gdown rapidfuzz lightgbm.</code>	Needs internet; on a kernel without it, rapidfuzz and lightgbm are usually pre-installed.

Cell	Kind	What it does	Failure mode and fallback
2	code	Locates the data: <code>find_train_source1("/kaggle/input")</code> globs recursively for <code>train_source1.tsv</code> (shortest path wins); otherwise <code>gdown.download(id=GDRIVE_ID)</code> fetches <code>/kaggle/working/data.zip</code> (skipped when the zip is already present), <code>unzip -q -o</code> extracts it under <code>/kaggle/working/data</code> , and the glob is repeated. <code>DATA_DIR</code> is the grandparent of the hit, <code>TEST_DIR = DATA_DIR/test</code> .	Asserts when no <code>train_source1.tsv</code> is found; attaching the dataset as a Kaggle input avoids any download.
3	code	Fetches <code>er_pipeline.py</code> , <code>validate_submission.py</code> and <code>score.py</code> from <code>RAW_BASE</code> (<code>raw.githubusercontent.com</code> , branch <code>claudio/wizardly-cori-snj0nx</code>) with <code>urllib.request.urlretrieve()</code> .	A failed fetch prints manual-upload instructions; only <code>er_pipeline.py</code> is asserted.
4	code	Runs <code>python er_pipeline.py --data-dir {DATA_DIR} --out-dir /kaggle/working/output</code> , then lists the output directory.	The run log (see Chapter 10) records the blocking ceiling, fold iterations and the selected (<code>thr</code> , <code>ratio</code>).
5	code	Runs <code>validate_submission.py</code> on both files against <code>TEST_DIR</code> ; a commented line shows how <code>score.py</code> scores a prediction file in ground-truth format.	Non-zero exit lists the issues; the package must not be built until it prints <code>PASS</code> .
6	code	Builds <code>{TEAM}_submission/</code> : copies the two TSVs and <code>er_pipeline.py</code> , fetches <code>README.md</code> , <code>requirements.txt</code> and <code>Documentation_template.md</code> from <code>RAW_BASE</code> , zips with <code>shutil.make_archive()</code> , lists the archive.	A failed fetch prints a warning and leaves that file out; <code>TEAM = "team"</code> must be edited first.
7	markdown	Where the files live under <code>/kaggle/working</code> , how to download them from the Output tab, and what to upload to the portal.	—

11.4.1 Dataset acquisition and mirrors

Cell 2 prefers an attached Kaggle input because it needs no transfer and is version-pinned by Kaggle. The Google Drive path uses file id `1bukugde70Drs9bHw8nr5oSr72ZPHDHzc`, one of the two links in the repository `README`. A second mirror is the challenge portal's own download, https://cdn.unstop.com/files/6ab10eb3b23ba_student_resource.zip, a plain HTTPS URL that needs no `gdown`; replacing `gdown.download()` with `urllib.request.urlretrieve(URL, ZIP_PATH)` is the only change. Both archives are searched with the same recursive glob, so the nesting inside the zip (for example a `student_resource/` folder wrapping `dataset/` and `utils/`, as the problem statement implies) is irrelevant; `find_data_dir()` applies the same recursive glob and grandparent rule when `--data-dir` is omitted, searching `/kaggle/input`, the working directory, its parent and `dataset/` in that order and taking the first hit rather than the shortest path.

11.4.2 Code acquisition

Cell 3 assumes that the branch named in `RAW_BASE` has been pushed; a raw URL for a local-only branch returns HTTP 404. When the repository is not pushed, or the kernel has no internet, the fallback is to upload the three files into `/kaggle/working` by hand (Kaggle sidebar, Add data, Upload) and re-run the cell; the same applies to the files fetched in cell 6, whose warning names the missing file to add to the package folder before zipping. Attaching a repository snapshot as a Kaggle dataset and replacing `fetch()` with a `shutil.copy()` from `/kaggle/input` for each entry of `FILES` removes the dependency on the branch name.

Risk. The zip must be built from the output directory the validator checked, in the same session. Re-running cell 4 regenerates both TSVs, and because LightGBM is not seeded (Section 11.5) the new `matching_results.tsv` can differ in a few borderline pairs from the one uploaded to the leaderboard. Run cells 4, 5 and 6 once, in order, and upload the leaderboard file from the same `/kaggle/working/output`.

11.5 Determinism

Most of the pipeline is a pure function of the data and library versions: `TfidfVectorizer` vocabularies, the sparse products in `topk_sparse()`, `np.argmaxpartition`, `rapidfuzz` scores, `GroupKFold` (which never shuffles) and the `(thr, ratio)` grid. The stochastic component is LightGBM. The `params` dictionary in `main()` sets `feature_fraction=0.8`, `bagging_fraction=0.8` and `bagging_freq=1`, so every tree is grown on a random 80 % of rows and columns, but sets no `seed`. LightGBM then uses its built-in default seeds, which keeps a single-threaded run repeatable, while `n_jobs=-1` lets histogram construction be split across threads in an order that varies between machines and builds. The OOF probabilities, the fold best iterations, the refit round count `int(np.mean(iters) * 1.1)` and a few borderline decisions can therefore differ between environments.

The recommended follow-up is to add `seed=42`, `deterministic=True` and `force_row_wise=True` to `params`, so that LightGBM neither draws unseeded samples nor picks its histogram strategy by timing; this costs some speed and changes nothing in the OOF sweep or the decision layer. A minor second source is tie-breaking: `decode_table()` sorts by probability with the default quicksort, which is not stable, so bit-identical probabilities could be ordered differently across NumPy versions; `kind="stable"` removes this. `Documentation_template.md` already calls for a seed for bit-exact reproduction.

11.6 Dependency pinning

`requirements.txt` currently states lower bounds (`numpy>=1.24`, `pandas>=2.0`, `scipy>=1.10`, `scikit-learn>=1.3`, `lightgbm>=4.0`, `rapidfuzz>=3.0`). Lower bounds document the API assumptions, such as the callback form `lgb.early_stopping(100, verbose=False)` that LightGBM 4 makes the only early-stopping interface, but neither satisfy the package specification's request for pinned versions nor fix the floating-point behaviour on which Section 11.5 depends. Before the final zip, the versions of the environment that produced the submitted files should be captured with `pip freeze` restricted to these six packages, written as exact `==` pins with the Python version in a comment, and re-installed in a clean environment to confirm that the validator passes and the output files are identical.

11.7 Reproducibility checklist

Table 11.3 collects the conditions under which a reviewer can regenerate both output files from the package alone and obtain the same leaderboard score. Each row names the artefact that evidences it.

Table 11.3 — Reproducibility checklist for the final package.

#	Check	Evidence / where it lives	Status
1	Only the provided train/test TSVs are read; no network call other than <code>pip</code> .	<code>load_split()</code> , <code>find_data_dir()</code> ; <code>import fallback</code> in <code>er_pipeline.py</code>	satisfied by design
2	Single entry point regenerates both files: <code>python src/er_pipeline.py --data-dir ... --out-dir ...</code>	<code>pipeline README.md</code> , cell 4	satisfied
3	Both files come from one invocation and pass <code>validate_submission.py</code> (PASS, exit 0).	cell 5 output, kept with the run log	to be confirmed on the real data

#	Check	Evidence / where it lives	Status
4	Uploaded <code>matching_results.tsv</code> is byte-identical to <code>output/matching_results.tsv</code> in the zip.	checksum recorded in <code>Documentation_template.md</code> Section 8	to be recorded
5	Exact dependency pins and Python version.	<code>requirements.txt</code> (<code>==</code> pins from <code>pip freeze</code>)	follow-up (currently lower bounds)
6	LightGBM seeded and deterministic (<code>seed=42</code> , <code>deterministic=True</code> , <code>force_row_wise=True</code>); stable sort in <code>decode_table()</code> .	params in <code>main()</code>	follow-up
7	Run log archived: blocking ceiling, fold iterations, OOF macro $F_{0.5}$, selected (<code>thr</code> , <code>ratio</code>), per-country match rate.	cell 4 output; Section 8 of the methodology document	to be filled from the run log
8	Package layout matches Figure 11.1; TEAM set; team fields of the methodology document filled.	<code>unzip -l</code> listing in cell 6	to be confirmed at packaging time
9	Licences and model size within limits (MIT/Apache-2.0-compatible libraries, tree ensemble far below 8B parameters).	see Chapter 13	satisfied
10	Smoke test passes on the synthetic set (<code>make_synthetic_dataset.py</code> , pipeline, validator, scorer).	repository README, <i>Smoke test</i> section	satisfied per README

Items 5 and 6 are the only ones that require a code change: three keys added to `params`, one `kind="stable"` argument in `decode_table()`, and rewritten pins in `requirements.txt`. Until they are applied, a re-run reproduces the candidates, features and grid exactly but not necessarily every borderline match, so the leaderboard file is the artefact of one specific run and is archived with its log.

11.8 Run-time expectations

The stages scale differently. Blocking is bounded by the chunked sparse products of `topk_sparse()`, about $2 \cdot 10^7$ dense cells per chunk (see Chapter 12). Pair features in `features()` are Python-level list comprehensions over the candidate table, at least one `rapidfuzz` call per string feature per pair (`n_alias_best` makes one per alias combination), and are expected to dominate wall-clock time; their cost is linear in the pair count the log reports as *pairs per S1*. LightGBM runs five folds with early stopping plus one refit on all cores. For the synthetic smoke test of 3,000 training and 1,500 test S1 entities generated by `make_synthetic_dataset.py`, the repository README reports about 30 s end to end; this figure is quoted from the README, not measured for this document, and says nothing about the real data. Table 11.4 reserves the slots for the real data; the elapsed-seconds prefix that `log()` prints at each stage boundary supplies them.

Table 11.4 — Run-time and size figures to be taken from the run log (not yet measured).

Quantity	Log line	Value
Train candidate pairs, pairs per S1	blocking: N pairs (x per S1) (train)	to be filled from the run log
Test candidate pairs, pairs per S1	blocking: N pairs (x per S1) (test)	to be filled from the run log
Fold best iterations, refit rounds	fold k: best_iter=...	to be filled from the run log
Wall-clock: load + prep, blocking, features, training, decode + write	elapsed-seconds prefix of each <code>log()</code> line	to be filled from the run log

Quantity	Log line	Value
Peak memory (Kaggle kernel)	kernel resource panel	to be filled from the run log
Test S1 entities matched, by country	test: n/N S1 entities matched; by country: ...	to be filled from the run log
Size of the two TSVs and of the zip	ls -la {OUT_DIR}, unzip -l in cells 4 and 6	to be filled from the run log

12 Performance, Complexity and Scalability

This chapter gives the cost model of `er_pipeline.py`: the asymptotic complexity of every stage, the memory bounds that `topk_sparse()` and `pair_cos()` enforce by construction, time and memory estimates for three hypothetical dataset sizes, the knobs that trade recall, precision and run time, how to locate hot spots from the timestamps that `log()` prints, and the parallelism the code does and does not exploit. The pipeline has not yet been run on the competition data; every number a run would measure is marked as an estimate or left as a placeholder.

12.1 Notation and cost model

$|S1|$ is the number of Source 1 records of a split, $|B| = |S2| + |S3|$ the number of Source 2 and Source 3 records that `load_split()` concatenates, and $|S1_c|$, $|B_c|$ the per-country blocks that `block()` forms by grouping on `country_n`. P is the number of candidate pairs, $F = 38$ the number of feature columns produced by `features()` (every column except `i1`, `i2`, `y`), R the number of boosting rounds trained, and `nnz` the number of non-zeros of a TF-IDF row. Train and test both pass through `prep()` → `block()` → `features()`, so per-split costs are paid twice; cross-validation, refit and the decode grid are paid once.

Two quantities dominate: the dense cell count of retrieval, $\sum_c |S1_c| \cdot |B_c|$ per view, quadratic within a country because `topk_sparse()` materialises the score matrix with `toarray()`; and P , which bounds the feature stage, the model and the output, and is at most $(k_{\text{name}} + k_{\text{full}} + k_{\text{addr}} + k_{\text{postal}}) \cdot |S1| = 70 \cdot |S1|$ before the union removes duplicates and the cosine floor of 0.05 drops empty overlaps.

12.2 Asymptotic complexity per stage

12.2.1 Normalisation and TF-IDF

`prep()` applies `basic()`, `expand()`, `core_name()`, `name_parts()` and the regular expressions `_dotted`, `_punct` and `_space` (inside `basic()`), `_postal`, `_digits` and `_ALIAS` once per record, $O((|S1| + |B|) \cdot L)$ for average field length L , in pure Python. `block()` then fits three `TfidfVectorizer` instances (`char_wb`, `ngram_range=(2, 4)`, `min_df=2`) on the concatenated texts of each view and transforms both sides. The analyser is Python code emitting about three n-grams per character, so fitting plus transforming is $O(3 \cdot (|S1| + |B|) \cdot L)$, and each row holds roughly $3L$ non-zeros at 8 bytes each (float32 value, int32 index); the `full` view, `name_c + " | " + addr_c`, has the longest rows.

12.2.2 Top-k retrieval

For every country and fuzzy view, `topk_sparse(A[i1], B[i2], k)` multiplies a chunk of $S1$ rows by the transposed block BT , densifies, and runs `np.argmaxpartition` per row. The sparse product costs $\sum_g df_A(g) \cdot df_B(g)$ over n-grams g , close to the dense bound because almost any two records share a frequent bigram; `toarray()` and `argpartition` are exactly $\Theta(|S1_c| \cdot |B_c|)$. The stage is $O(3 \cdot \sum_c |S1_c| \cdot |B_c| \cdot nnz)$, and country partitioning is what makes it affordable: two equal countries halve the cell count, while the open-set fallback for an unseen label costs a full $|S1_{\text{unseen}}| \cdot |B|$ product (see Chapter 6). The postal view adds $\sum_p |S1_p| \cdot |B_p|$ cells, normally negligible, plus one Python iteration and one `B.T.tocsr()` per postal code, so its cost is call overhead rather than arithmetic.

12.2.3 Union, pair cosines and features

The chunks are stacked with `np.vstack` and reduced by `drop_duplicates()`, $O(P_{\text{raw}})$ with $P_{\text{raw}} \leq 70 \cdot |S1|$; `pair_cos()` then computes the three cosines in slices of 200,000 pairs, $O(P \cdot \text{nnz})$. `features()` is a sequence of list comprehensions: ten `rapidfuzz` calls per pair (`fuzz.ratio`, `token_sort_ratio`, `token_set_ratio`, `partial_ratio`, `JaroWinkler.similarity`, `fuzz.ratio` on the full name, at least one `token_sort_ratio` for `n_alias_best`, three address scorers) plus the Python-level `jacc()`, initials, acronym and number-set checks. Each scorer is $O(|a| \cdot |b|)$ worst case but runs in C++ on short strings, so the Python-to-C boundary dominates and the stage behaves as $O(P)$ at tens of microseconds per pair. The context features (`_gap1`, `_gap2`, `rank1`, `rank2`, `n_cand1`, `n_cand2`) are vectorised group-by transforms, $O(P \log P)$ at most.

12.2.4 Matcher and decode grid

With `GroupKFold(args.folds)` each fold trains on $(1 - 1/\text{folds}) \cdot P$ rows for up to 2,000 rounds with early stopping at patience 100; histogram boosting costs $O(\text{rows} \cdot F \cdot R)$ per fit, scaled by `feature_fraction=0.8` and `bagging_fraction=0.8`, and the refit runs $1.1 \cdot \text{mean}(\text{best_iteration})$ rounds on all P rows. Prediction is $O(P \cdot R)$. `decode_table()` sorts once, $O(P \log P)$, and `drop_duplicates("i2")` leaves $|d| \leq \min(P, |B|)$ rows, which keeps the grid off the P axis. The sweep has 6 ratios $\times$ 31 thresholds (`np.arange(0.20, 0.96, 0.025)`) = 186 points, each a vectorised mask in `decide()`, a Python loop over kept rows in `to_lists()` and a loop over all $S1$ entities in `macro_f05()`: $O(|d| + |S1|)$ per point at a few microseconds per entity.

Table 12.1 — Asymptotic complexity per stage (per split unless marked train-only).

Stage	Function(s)	Time	Peak transient memory	Executes in
Load + normalise	<code>load_split()</code> , <code>prep()</code>	$O((S1 + B) \cdot L)$	Derived object columns	pandas parser; Python, 1 core
TF-IDF fit + transform (3 views)	<code>block()</code>	$O(3 \cdot (S1 + B) \cdot L)$	8 B per non-zero, $\approx 3L$ non-zeros per row per view	Python analyser, 1 core
Top-k retrieval (3 views)	<code>topk_sparse()</code>	$\Theta(3 \cdot \sum_c S1_c \cdot B_c)$ cells $\times$ nnz constant	Bounded: 2×10^7 cells $\times$ (4 + 8) B + sparse product ≈ 0.4 GB	SciPy / NumPy, 1 core
Postal view	<code>topk_sparse()</code> per postal code	$O(\sum_p S1_p \cdot B_p) + O(\#\text{postal codes})$ calls	Small blocks	Python loop + NumPy
Union + pair cosines	<code>drop_duplicates()</code> , <code>pair_cos()</code>	$O(P_{\text{raw}}) + O(P \cdot \text{nnz})$	Bounded: 200,000 rows $\times$ 2 matrices per slice	NumPy / SciPy, 1 core
Pair features	<code>features()</code>	$O(P)$ rapidfuzz + $O(P \log P)$ ranks	≈ 1 KB per pair (L, R frames, tuple lists, F columns)	Python + rapidfuzz, 1 core
Matcher CV + refit (train-only)	<code>lgb.train()</code>	$O(P \cdot F \cdot R \cdot (\text{folds} + 0.1))$	float64 copies of fold slices + binned Dataset	LightGBM, all cores
Decode grid (train-only)	<code>decode_table()</code> , <code>decide()</code> , <code>to_lists()</code> , <code>macro_f05()</code>	$O(P \log P) + 186 \cdot O(d + S1)$	One filtered copy of d per point	Python, 1 core
Predict + decode + write (test)	<code>model.predict()</code> , <code>decide()</code> , file loop	$O(P \cdot R) + O(S1 + d)$	Two id-list dictionaries	LightGBM (cores) + Python

12.3 Memory model

The two stages that could allocate proportionally to $|S1| \cdot |B|$ are bounded by construction. In `topk_sparse()` the chunk is `max(16, int(max_cells // n_b))` with `max_cells = 2e7`, so the dense float32 score block never exceeds about 80 MB (the floor of 16 rows only matters beyond 1.25 million rows in a block), the full-width int64 output of `np.argmaxpartition` about 160 MB, and the nearly dense sparse product a further 160 MB; the top-k slice is taken with `.copy()` so the full-width array is released every iteration rather than pinned by a view (Figure 12.2 in 12.6). `pair_cos()` bounds its transient through slices of 200,000 pairs. Neither bound depends on block size, so an unseen country falling back to all of B costs time, not memory.

What grows is per-pair state. The feature frame holds about 40 columns of 8 bytes, 320 bytes per pair; inside `features()` the row-aligned copies L and R and the lists of string tuples roughly triple that. During cross-validation `ftr.iloc[a][feats]` and the validation slice make float64 copies of about 300 bytes per row beside the binned LightGBM Dataset, and `ftr` is still referenced while `fte` is built, so both frames are resident at test time. The TF-IDF matrices, about 3.2 KB per record for three views at 400 non-zeros per record, are local to `block()`. Table 12.2 combines these unit costs for three hypothetical sizes; the values are estimates, not measurements.

Table 12.2 — Estimated memory and wall-clock for three hypothetical sizes (ESTIMATES from the cost model; assumptions: $|B| = 2 \cdot |S1|$, two equal countries, test split the same size as train, ≈ 50 pairs per S1 after union, 5×10^7 dense cells/s in retrieval, 40 μ s per pair in `features()`, ≈ 500 rounds per fold, 4 cores).

Quantity	Small: 20k S1	Medium: 100k S1	Large: 500k S1
$ B = S2 + S3 $	40k	200k	1M
Candidate pairs P (per split)	$\approx 1M$	$\approx 5M$	$\approx 25M$
Dense retrieval cells (3 views, per split)	1.2×10^9	3×10^{10}	7.5×10^{11}
TF-IDF matrices (3 views)	≈ 0.2 GB	≈ 1 GB	≈ 5 GB
Feature frame (one split)	≈ 0.3 GB	≈ 1.6 GB	≈ 8 GB
Peak during <code>features()</code>	≈ 1 GB	≈ 5 GB	≈ 25 GB
Estimated peak RSS of the run	≈ 1.5 GB	$\approx 6\text{--}8$ GB	$\approx 28\text{--}30$ GB
TF-IDF fit + transform (both splits)	≈ 0.5 min	≈ 2.5 min	≈ 12 min
Top-k retrieval (both splits)	≈ 1 min	≈ 20 min	≈ 8.5 h
<code>features()</code> (both splits)	≈ 1.5 min	≈ 7 min	≈ 35 min
LightGBM 5-fold CV + refit	≈ 4 min	≈ 20 min	≈ 100 min
Decode grid (186 points)	≈ 15 s	≈ 75 s	≈ 6 min
Total wall-clock (estimate)	$\approx 8\text{--}10$ min	$\approx 50\text{--}60$ min	$\approx 11\text{--}12$ h

Risk. The quadratic retrieval term is invisible at the small size and dominant at the large one. If one country holds most records, the retrieval estimate must be re-derived from the actual $\sum_c |S1_c| \cdot |B_c|$ given by the `profile()` country counts before a Kaggle session is committed.

12.4 Scaling knobs

Every knob in Table 12.3 moves at least two of recall ceiling, precision and run time, and the metric of Chapter 2 decides the direction. Because $F_{0.5}$ weights precision twice and a singleton scores 1.0 only for an empty prediction, extra candidates are not free even with a good matcher: each is a chance for a false positive the

threshold must absorb. Raising a top-k pays only when the blocking recall by country or by source lines show true pairs being missed; lowering it is the first response when the run exceeds the budget.

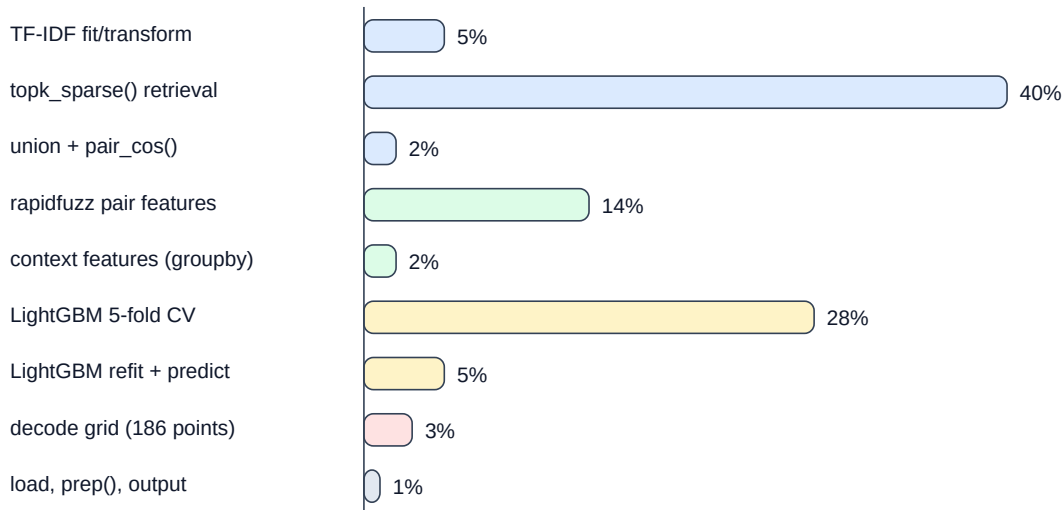
Table 12.3 — Tuning knobs and their effect on the recall ceiling, precision and run time.

Knob (location)	Default	Recall ceiling	Precision / $F_{0.5}$	Time and memory
k_name, k_full (block())	25, 25	Up with k, diminishing beyond a few tens	More low-similarity candidates to reject; harder singletons	P, hence features(), LightGBM and frame memory, $\approx$ linear; retrieval unchanged
k_addr, k_postal (block())	10, 10	Recovers name-rewritten matches	Address-only pairs are the noisiest candidates	As above, smaller share of P
Cosine floor sc > 0.05 (block())	0.05	0.05 removes only near-empty n-gram overlaps; loss grows with the floor	Removes pairs sharing almost nothing	Reduces P at the union only
max_cells (topk_sparse())	2×10^7	None	None	Transient $\propto$ max_cells; larger chunks amortise Python overhead
chunk(pair_cos())	200,000	None	None	Transient memory only
ngram_range, min_df (block())	(2, 4), 2	Narrower range: coarser cosines	Indirect, through cosine features	nnz per row and TF-IDF memory $\propto$ number of n-gram sizes
-- folds (CLI)	5	None	Fewer folds: noisier OOF and (thr, ratio)	CV time $\propto$ folds; grid unchanged
num_leaves, learning_rate (params)	63, 0.05	None	Capacity vs. over-fitting, checked by early stopping	Rounds $\propto 1/\text{learning_rate}$; cost per round $\propto$ leaves
Early-stopping patience, max rounds	100, 2,000	None	Marginal	Bounds R
Grid: ratio set $\times$ thr step	$6 \times 31 = 186$	None	Finer step: better operating point, more OOF over-fitting	$\propto$ points $\times (d + S1)$
n_jobs (params)	-1	None	None	LightGBM uses every core

12.5 Where time goes and how to profile it

Figure 12.1 shows the expected split of wall-clock at the medium size: retrieval and cross-validation take two thirds, the rapidfuzz loop is the third block, and TF-IDF fitting, the grid and I/O are minor. At the small size LightGBM dominates because the quadratic blocks are tiny; at the large size retrieval swamps everything.

Illustrative share of wall-clock at the medium size (train + test), estimate from Table 12.2



Blue: blocking (1 core) · Green: features (1 core) · Amber: LightGBM (all cores) · Red: decode (1 core)

Figure 12.1 — Where time goes (illustrative estimate at the medium size; to be replaced by measured intervals from the run log).

The pipeline carries its own coarse profiler: `log()` prefixes every message with the seconds elapsed since module import (`T0`), so the difference between consecutive lines is the duration of the code between them, and Table 12.4 maps each interval to the stage it measures. Two intervals are composite: the last `profile()` line to the first `blocking:` line covers `prep()` of all four frames plus `block(tr1, tr2)`, and the `blocking oracle` line to the second `blocking:` line covers the refit, the importance print and `block(te1, te2)`; a `log()` call after `prep()` and before the refit would separate them. For finer resolution the script runs unchanged under `python -m cProfile -o run.prof er_pipeline.py`, which attributes time to `topk_sparse()`, each rapidfuzz scorer and `lgb.train()` individually.

Table 12.4 — Log-line intervals and the stage each one measures.

From log line	To log line	Stage measured
data dir:	first == train:	load_split() for both splits
last == test: block	first blocking:	prep() × 4 + block(tr1, tr2)
first blocking:	blocking recall ceiling:	features(ctr, ...) + labelling
blocking recall by source:	fold 0:	First fold of lgb.train()
fold k:	fold k+1:	Fold k+1 training + prediction
fold 4:	00F macro F0.5	decode_table() + 186-point grid
00F macro F0.5	blocking oracle	Floor and ceiling evaluations (2 × macro_f05())
blocking oracle	second blocking:	Refit + importances + block(te1, te2)
second blocking:	wrote .../matching_results.tsv	features(cte, ...) + predict + decode + write

Table 12.5 — Measured stage timings and peak memory (to be filled from the run log).

Interval (Table 12.4)	Train split	Test split	Notes
Load + prep	to be filled from the run log	to be filled from the run log	

Interval (Table 12.4)	Train split	Test split	Notes
<code>block()</code>	to be filled from the run log	to be filled from the run log	pairs per S1: to be filled from the run log
<code>features()</code>	to be filled from the run log	to be filled from the run log	
LightGBM folds 0–4 (<code>best_iter</code>)	to be filled from the run log	—	
Decode grid	to be filled from the run log	—	
Refit + test predict + write	—	to be filled from the run log	
Total wall-clock / peak RSS	to be filled from the run log		Kaggle session monitor

12.6 Parallelism and optimisation opportunities

Only LightGBM is multi-threaded. The `char_wb` analyser, the SciPy sparse product, `np.argpartition`, the rapidfuzz comprehensions, the pandas group-bys and the decode grid run on one core, so on a four-core machine three cores idle for roughly two thirds of the run at the medium size. Three changes, none altering an output value, would recover most of that:

- 1. Vectorised string scoring.** Recent rapidfuzz releases provide `rapidfuzz.process.cpdist(queries, choices, scorer=..., workers=-1)`, which scores the *i*-th query against the *i*-th choice in C++ threads and returns a NumPy array; one call per feature replaces each comprehension in `features()` and uses every core. `process.cdist` is the all-pairs variant, appropriate only inside one small block. `requirements.txt` pins only `rapidfuzz>=3.0`, and `cpdist` is absent from the earliest 3.x releases, so the installed version must be checked.
- 2. Process pools for retrieval.** The per-country, per-view loop in `block()` is embarrassingly parallel, since each `topk_sparse()` call touches only its own row slices; a `concurrent.futures.ProcessPoolExecutor` over (country, view) tasks multiplies throughput by the core count at the price of one bounded transient per worker (Figure 12.2).
- 3. Vectorised grid.** Replacing `to_lists()` and `macro_f05()` by group-by counts of true positives per S1 entity makes each grid point $O(|d|)$ NumPy work, so a finer threshold step becomes free.

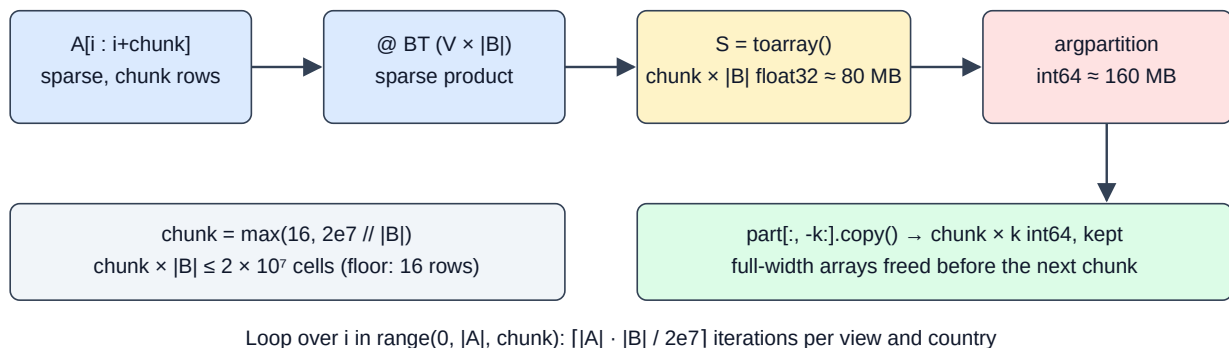


Figure 12.2 — Memory bound of `topk_sparse()`: the chunk width is derived from $|B|$, so the dense transient is constant regardless of block size and one such transient per worker is affordable.

Design note. The single-threaded loops are kept because they make the feature definitions readable and auditable for the fair-play review of Chapter 13; the optimisations above preserve every feature value and head the roadmap of Chapter 14.

12.7 Kaggle limits and capacity

The reference environment is a Kaggle CPU notebook, which at the time of writing offers 4 virtual cores, about 30 GB of RAM, 20 GB of scratch under `/kaggle/working` and a 12-hour wall-clock limit per session; these limits change and must be re-checked before a run is scheduled. The cost model places the small and medium sizes inside that envelope and the large size outside it on both axes (Table 12.6). Out-of-memory is the failure mode to guard against first: it ends the session abruptly, typically inside `features()` or a fold, whereas a run that is merely slow can be stopped after the train-side log lines needed to calibrate the estimates have appeared, since `log()` flushes every line.

Table 12.6 — Capacity envelope on a 4-core, 30 GB, 12-hour Kaggle CPU session (estimates).

Size	Peak RSS vs. 30 GB	Wall-clock vs. 12 h	Verdict as shipped	Changes that restore headroom
Small (20k S1)	≈ 5%	≈ 1.5%	Fits; LightGBM dominates	None needed; a finer grid step is affordable
Medium (100k S1)	≈ 25%	≈ 8%	Fits with wide margin	None needed; <code>cpdist</code> halves <code>features()</code>
Large (500k S1)	≈ 95–100%	≈ 95–100%	Does not fit reliably	Halve <code>k_name/k_full</code> ; delete <code>ftr</code> after the refit; process-pool retrieval; or block each country in a separate session
Actual competition split	to be filled from the run log	to be filled from the run log	to be filled from the run log	

The verdicts are read against the metric as well as the budget. Lowering a top-k lowers the recall ceiling, but the singleton term rewards the cleaner candidate lists that result, so a modest reduction is often neutral on OOF $F_{0.5}$; the `blocking oracle` and `floor` lines of the log (see Chapter 10) show whether a given k has moved the ceiling by more than the matcher can recover, and they are the first numbers to read once Table 12.5 is filled.

13 Fair Play, Licensing and Constraint Compliance

The problem statement attaches five hard constraints and a fair-play rule to every submission, and the top packages are reviewed before rankings are confirmed. This chapter is written for that review: it maps each rule to the construct in `er_pipeline.py` that satisfies it, inventories every file-system and network touch, lists dependency licences, bounds the parameter count, and shows how `candidate_pairs.tsv` makes blocking auditable.

13.1 Compliance matrix

Table 13.1 states the conclusion; the remaining sections supply the evidence. Every hard constraint is met by construction; the two recommendations concern the strength of the reproducibility claim, not eligibility.

Table 13.1 — Constraint-to-evidence compliance matrix

Constraint	Evidence in code	Status
No external data lookup (fair play)	Only <code>read_tsv()</code> reads of the seven provided TSVs; no network-capable imports (Section 13.2)	Compliant
No network activity at run time	One conditional <code>os.system(pip install rapidfuzz)</code> , software only, skipped when the package is installed	Compliant; remove fallback and pre-install (Section 13.2.2)
No use of test labels	<code>load_split()</code> returns <code>gt = None</code> for test; <code>thr</code> , <code>ratio</code> and rounds come from <code>GroupKFold</code> OOF	Compliant
Model licence MIT / Apache-2.0	LightGBM (MIT) booster trained in-run; no pretrained weights (Table 13.2)	Compliant
Model $\leq 8B$ parameters	$\leq 2,200$ trees $\times$ 63 leaves, about 4×10^5 numbers (Section 13.3.2)	Compliant
Exactly one row per test S1 entity	<code>to_lists()</code> keyed on <code>s1.entity_id</code> ; writer iterates <code>tel.entity_id.values</code>	Compliant
Empty list for singletons	Empty list joins to <code>""</code> ; row is <code>id\t\n</code>	Compliant
No duplicate ids within a list	<code>drop_duplicates()</code> in <code>block()</code> and <code>decode_table()</code> ; <code>dict.fromkeys()</code> in writer	Compliant
S2-/S3- ids only, present in the test set	<code>i2</code> indexes <code>pd.concat([s2, s3])</code> loaded from the test files; S1 self-matches impossible	Compliant
Matches $\subseteq$ candidates; candidates = inference set	<code>decide()</code> and <code>cands</code> both derive from <code>fte</code> ; <code>model.predict(fte[feats])</code>	Compliant
Exact header, tab-separated, no quoting	Literal <code>f"source1_entity_id\t{col}\n"</code> ; ids loaded with <code>quoting=3</code>	Compliant; checked by <code>validate_submission.py</code>
Country as an open set; France included	<code>groupby("country_n")</code> with all-records fallback; no country feature (Section 13.5)	Compliant
Package layout (output/, code/.../src, README, requirements, methodology)	Single script under <code>src/</code> , <code>requirements.txt</code> , README run steps (see Chapter 11)	Compliant

Constraint	Evidence in code	Status
Bit-exact reproducibility	Row and column subsampling in params with no explicit seed or deterministic key and <code>n_jobs=-1</code> ; lower-bound-only pins in <code>requirements.txt</code>	Partial; set seed and <code>deterministic=True</code> , pin exact versions (see Chapter 11)

13.2 No external lookup

13.2.1 What the script can open

The fair-play rule forbids external databases, APIs, geocoders and internet-sourced augmentation. The strongest evidence of compliance is an exhaustive list of the process's input and output channels. The imports are seven standard-library modules and five third-party packages (NumPy, pandas, scikit-learn, rapidfuzz, LightGBM); none is a network client, and no `socket`, `urllib`, `requests` or cloud SDK appears in the file. Figure 13.1 draws the boundary; the touches are:

`find_data_dir()`

`glob()` for `train_source1.tsv` under `/kaggle/input`, `.`, `..` and `dataset`: local metadata only.

`load_split()`

Three `read_tsv()` calls per split plus an `os.path.exists()`-guarded read of the ground truth; on the test split the guard fails and `gt = None`.

`main()`

`os.makedirs()` and two `open(path, "w")` calls for the output files (see Chapter 11).

module import

`os.system(pip install -q rapidfuzz)`, only when the import raises `ImportError` (Section 13.2.2).

The vectorisers in `block()` are fitted on the texts of the split being processed, on the test split the provided test records: transductive use of the challenge's own unlabelled inputs, which lets French character n-grams be learned without a French resource (see Chapter 6).

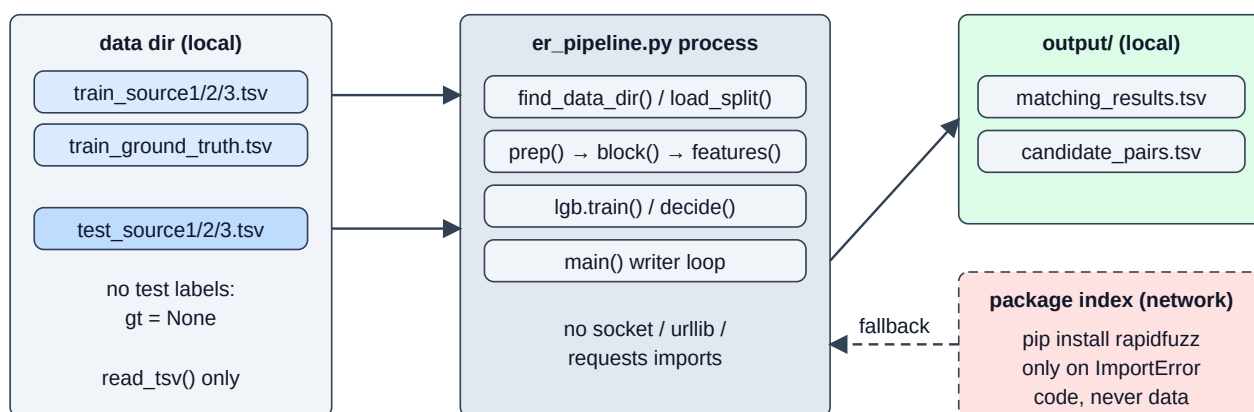


Figure 13.1 — The complete I/O boundary of the pipeline: seven local TSV reads, two local TSV writes, and one conditional network action that fetches software, not data.

13.2.2 The rapidfuzz install fallback

The one statement that can reach the network sits in the import block: if importing `fuzz` and `JaroWinkler` from `rapidfuzz` raises `ImportError`, the script runs `os.system(f"{sys.executable} -m pip install -q rapidfuzz")` and retries both imports. This is compatible with the fair-play rule, which targets lookups of business identities and address-normalisation services: installing an MIT-licensed string-similarity library fetches no record, gazetteer or model weights, and no input data leaves the process. It is nonetheless the one line a reviewer scanning for network activity will stop at.

Risk. The `pip install` fallback is unpinned and environment-dependent. Offline, it fails silently inside `os.system()`, the second import raises `ImportError`, and the run aborts before reading any data. Online, it may resolve a rapidfuzz release newer than the one behind the submitted outputs, shifting the similarity scores at the last decimal and, through the tuned threshold, a few near-boundary decisions. Neither breaks a rule, but both weaken the "anyone can regenerate both output files" promise. The remedy is to pin rapidfuzz to an exact version in `requirements.txt`, install it before the run as the README instructs, and delete the `except ImportError` branch so a missing dependency fails loudly. On Kaggle the package is pre-installed and the branch never executes.

13.3 Licence and model-size constraints

13.3.1 Licence inventory

Constraint 5 requires an MIT- or Apache-2.0-licensed model of at most 8 billion parameters. The final model is a LightGBM booster trained from scratch on the provided training pairs; no pretrained weights, embeddings or language models are loaded, so the only licence attached to the model is that of the software that produced it. Table 13.2 lists every dependency in `requirements.txt`; the BSD-3-Clause packages are tooling rather than models, and BSD-3-Clause is at least as permissive as the two licences the rule names.

Table 13.2 — Licence inventory of the runtime dependencies

Package	Role in the pipeline	Licence	Carries model weights?
LightGBM (lightgbm>=4.0)	Gradient-boosted matcher, <code>lgb.train()</code>	MIT	No; trained in-run
rapidfuzz (rapidfuzz>=3.0)	fuzz ratios and JaroWinkler in <code>features()</code>	MIT	No
scikit-learn (scikit-learn>=1.3)	TfidfVectorizer, GroupKFold	BSD-3-Clause	No; vectorisers fitted in-run
NumPy (numpy>=1.24)	Arrays, <code>argpartition</code> in <code>topk_sparse()</code>	BSD-3-Clause	No
SciPy (scipy>=1.10)	Sparse matrices behind TF-IDF (not imported directly; a scikit-learn dependency)	BSD-3-Clause	No
pandas (pandas>=2.0)	Frames, <code>read_csv</code> , <code>groupby</code> context features	BSD-3-Clause	No
Python standard library	<code>re</code> , <code>unicodedata</code> , <code>glob</code> , ...	PSF	No

13.3.2 Parameter count

The size bound follows from `params` and the training loop alone. Each fold trains up to 2,000 rounds with `lgb.early_stopping(100)`, `iters` collects each fold's `best_iteration`, and the final refit in `main()` uses `int(np.mean(iters) * 1.1)` rounds, so even if every fold ran to the cap the deployed forest holds at

most 2,200 trees. With `num_leaves=63`, a binary tree stores 62 internal nodes (a feature index and a threshold each) and 63 leaf values, about 187 numbers, so the worst case is roughly $2,200 \times 187 \approx 4.1 \times 10^5$ numbers, a factor of about 2×10^4 below the 8×10^9 ceiling. The actual tree count is a run-time quantity and is recorded in the placeholder table of Chapter 11 rather than estimated here. The three TF-IDF vocabularies are preprocessing artefacts, not model parameters, bounded by the distinct 2–4-character n-grams with `min_df=2` in the provided texts. The 38 input columns of `feats` (every column of the feature frame except `i1`, `i2` and `y`) are listed in Chapter 7.

13.4 Output-format constraints by construction

Constraints 1 to 4 concern the two TSV files, and each is satisfied by a specific construct. `to_lists()` starts from `{e: [] for e in s1.entity_id}`, so every test S1 entity owns a row before any pair is appended, and the writer iterates `te1.entity_id.values` once; missing and duplicate rows are both impossible. Self-matches are excluded because `load_split()` returns `pd.concat([s2, s3])` as the only "other" frame, `block()` draws `i2` from it alone, and `to_lists()` maps `i2` through `other.entity_id.values`. Unknown ids are excluded because every emitted id string was loaded by `read_tsv()` from the test S2/S3 files with `dtype=str` and `quoting=3`. Duplicates within a list are removed at up to three points: `drop_duplicates()` on the `(i1, i2)` table in `block()`, which already makes `candidate_pairs.tsv` duplicate-free; `drop_duplicates("i2")` in `decode_table()`, which additionally assigns each S2/S3 record to a single S1 entity in `matching_results.tsv`; and `dict.fromkeys()` in the writer, an order-preserving safety net for both files. Matches are a subset of candidates, and `candidate_pairs.tsv` is the exact inference set, because `decide()`, `cands` and `model.predict(fte[feats])` all operate on the same `fte` frame. The stdlib validator of Chapter 10 re-checks all of this from the files alone.

13.5 Country as an open set

The problem statement forbids hard-coding, filtering or one-hot encoding the pipeline to {US, India}. The country label enters the code at two points: `prep()` normalises it through `basic()` into `country_n`, a free string, and `block()` partitions the S1 frame by `groupby("country_n")`, whatever values occur, comparing a group whose label has no S2/S3 counterpart (or a blank label) against every S2/S3 record instead of an empty block; the `continue` that guards this loop fires only when the whole S2/S3 frame is empty. Country is not a model feature: `features()` emits no country column and no column derived from one, so the booster cannot learn a US- or India-specific branch; the only source indicator is `is_s3`, taken from the entity-id prefix. French legal forms and address idioms sit in `NAME_ABBR` (`cie`, `ste`), `LEGAL` (`sarl`, `sas`, `eurl`, ...), `ADDR_ABBR` (`fbg`, `chem`, `imp`) and `ADDR_STOP` (`de`, `la`, `des`, ...) as generic normalisation entries applied to every record, not as a country branch, so a French test entity takes the same path as a US one and is written to both files.

Design note. `best_thr` and `best_ratio` are tuned on US and Indian out-of-fold predictions, the only labelled entities. Under a precision-weighted $F_{0.5}$ with all-or-nothing singleton credit, an operating point too permissive for France costs more than one too strict; the per-country match-rate line logged at the end of `main()` checks that France is neither starved nor over-matched (see Chapter 14).

13.6 Auditability of blocking

The organisers use `candidate_pairs.tsv` to measure the recall ceiling and reduction ratio of blocking and to verify the pipeline. The file supports that audit directly: it is the full `fte[["i1", "i2"]]` table converted to id lists, with no filtering between it and `model.predict()`, so the reduction ratio $1 - |\text{pairs}| / (|S1| \cdot |S2 + S3|)$ can

be computed from the file and the test sources alone. On the training split the same `block()` call is scored against the labels and logged: the pair-level recall ceiling, the count of S1 entities with at least one true match blocked (the quantity the macro metric actually depends on), recall by country and by source, the all-candidates floor and the blocking-oracle ceiling. A reviewer can therefore confirm that the submitted candidate set came from a stage whose behaviour was measured on labelled data, not tuned on hidden information; the values themselves are to be filled from the run log (Chapter 11). The remaining sources of run-to-run variation, unseeded bagging and dependency drift, are catalogued in Chapter 11 and carried into the risk register of Chapter 14.

14 Limitations, Risks and Roadmap

The pipeline of Chapters 4 to 11 is a complete baseline, but it has not yet been executed on the real data, and several design choices are deliberate simplifications. This chapter states those limitations plainly, quantifies where possible how each bounds the macro $F_{0.5}$ score, records the operational risks with likelihood, impact and mitigation, and lays out a three-horizon roadmap in which every item is gated on the out-of-fold (OOF) validation score of Chapter 10.

14.1 Status of the evidence

Every quantitative claim in this document is a property of the code, a bound derived from the metric, or a placeholder marked "to be filled from the run log". `er_pipeline.py` has been smoke-tested only on the synthetic set produced by `utils/make_synthetic_dataset.py`: every stage of `main()` runs and the outputs pass `utils/validate_submission.py`. No blocking recall ceiling, OOF macro $F_{0.5}$ or per-country test match rate has been observed on the challenge data. The values the code carries (`k_name=25`, `k_full=25`, `k_addr=10`, `k_postal=10`, the `0.05` cosine floor, `num_leaves=63`, the sweep grid) are informed defaults, not tuned values.

Risk. Until the first full run completes, the expected score is unknown. The log lines that `main()` prints (recall ceiling overall, by country and by source; all-candidates floor; blocking oracle ceiling; OOF macro $F_{0.5}$ with the selected `thr` and `ratio`) are the first evidence to collect; every roadmap item in Section 14.4 is judged against them.

14.2 Limitations

Table 14.1 lists each limitation, the mechanism by which it affects the macro $F_{0.5}$ of Chapter 2, and the mitigation the roadmap assigns to it. The subsections explain the mechanisms in order of expected effect on the score.

Table 14.1 — Limitations, their effect on macro $F_{0.5}$, and planned mitigation

Limitation	Where in the code	Effect on $F_{0.5}$	Mitigation (horizon)
Not yet run on real data	whole of <code>main()</code>	Score unknown; defaults untuned	First run; fill the run-log placeholders against Table 15.2 (H1)
Recall bounded by fixed top-k blocking	<code>block()</code> , <code>topk_sparse()</code>	A missed true pair caps the entity at $F_{0.5} = 1.25R/(0.25+R)$; a one-match entity scores 0	Phonetic keys, embedding view (H1, H2)
Hand-built abbreviation tables	<code>NAME_ABBR</code> , <code>ADDR_ABBR</code> , <code>LEGAL</code>	Uncovered variants lower the string features; false expansions (<code>st</code> , <code>pl</code> , <code>ste</code>) add noise	Cross-encoder re-ranker; country-adaptive normalisation (H2, H3)
No transliteration model; non-ASCII dropped	<code>basic()</code> NFKD + ASCII ignore	Latin-script variants rely on n-gram overlap only; fully non-Latin strings become empty and are never blocked	Phonetic keys; multilingual encoder (H1, H2)
France unseen in training	<code>gt</code> covers US and India only	<code>thr</code> , <code>ratio</code> and tree splits calibrated on two countries; French error rate unmeasured	Pseudo-unseen validation; per-country (<code>thr</code> , <code>ratio</code>) (H1, H2)

Limitation	Where in the code	Effect on $F_{0.5}$	Mitigation (horizon)
Context features depend on candidate-set composition	<code>features()</code> <code>groupby("i1")("i2")</code>	gap, rank, <code>n_cand</code> shift when block sizes or <code>k</code> change between train and test	Refit whenever <code>k</code> changes; monitor <code>n_cand1</code> distribution (H1)
Single global (<code>thr</code> , <code>ratio</code>)	sweep in <code>main()</code> , <code>decide()</code>	Optimal operating point differs by country and source; global value is a compromise	Per-country and per-source sweep (H1)
No transitive consistency	<code>decode_table()</code> , <code>decide()</code>	Two accepted S2/S3 records of one S1 may contradict each other; false positives on multi-match entities	Graph decoding per connected component (H3)
Python-loop feature computation	<code>features()</code> list comprehensions	No effect on score; bounds the candidate volume that fits the time budget, hence bounds <code>k</code>	<code>rapidfuzz.process.cdist</code> (H1)
Non-deterministic training	params without seed	Last-decimal variation between runs; reproducibility claim weakened	Fix seed, <code>bagging_seed</code> , <code>feature_fraction_seed</code> (H1)

14.2.1 The blocking ceiling

The matcher only sees pairs that `block()` produces, and the union of four views (core name `k=25`, name-plus-address `k=25`, address `k=10`, same-postal name `k=10`, each cut at cosine 0.05) is a finite budget. A true match outside all four lists is unrecoverable, and because the metric is macro-averaged per Source-1 entity the loss is concentrated: an entity whose single true match is missed scores exactly 0. For an entity with m true matches of which the blocker retrieves $m-1$ and the matcher accepts exactly those, precision is 1 and recall is $(m-1)/m$, so $F_{0.5} = 1.25 \cdot R / (0.25 + R)$ gives 0.833 for $m=2$ and 0.909 for $m=3$. Records whose name and address are both heavily rewritten, or whose name is in a non-Latin script, are the most likely to fall outside every view; the log line "S1 entities with ≥ 1 true match blocked" measures this on the training split.

14.2.2 Coverage of the normalisation tables

`NAME_ABBR` holds 18 entries and `ADDR_ABBR` 41, written from the problem statement's noise list, not learned from the data. Any abbreviation absent from the tables is left to the fuzzy features, which penalise `mfrs` against `manufacturers` as heavily as a typo. Several expansions are ambiguous and applied unconditionally: `st` becomes `street` even in `st louis`, `ste` is `suite` in addresses but `societe` in names, and in addresses the single letters `n`, `s`, `e`, `w` become compass points even when they are initials of a landmark or a person. Hindi-transliteration coverage is limited to `marg`, `salai`, `ngr`, `clny`, `sec`, `chowk` and `mg`; alternations such as `sri/shri` are handled only implicitly by n-gram overlap, and the French list covers legal forms and four street types. Because `basic()` encodes to ASCII with `errors="ignore"`, a string written entirely in Devanagari collapses to the empty string, its TF-IDF row in the name view is all zeros, and the record can be blocked only through the address view or the address half of the name-plus-address view; the same-postal view ranks by name cosine and therefore never returns it either.

14.2.3 The unseen country and feature drift

No French pair contributes to the LightGBM training set or to the `(thr, ratio)` sweep, so behaviour on France is an extrapolation. The design mitigates this (Chapters 5 to 7): normalisation is language-agnostic, the vectorisers are refitted on the test corpus inside `block()`, and the context features (`cos_full_gap1`, `rank1`, `n_cand1` and their `i2` counterparts) are relative rather than absolute. Those context features are also the second concern: computed by `groupby` over the candidate table, their distribution depends on how many candidates each record

collects, hence on block size, view overlap and country. If French blocks differ in size from US and Indian ones, `n_cand1` and the gap features shift and tree splits may be systematically off. The only test-time signal is the per-country match rate that `main()` logs: a French rate far from the seen countries' is a warning, not a diagnosis.

14.2.4 Decision-layer simplifications

The decision rule (Chapter 9) is where precision is bought, and it is deliberately simple. One `thr` and one `ratio` serve all countries and both sources; if the OOF-optimal threshold for India differs from the US one, the global value under-matches one and over-matches the other, and France inherits a compromise it was never measured on. Second, `decode_table()` assigns each S2/S3 record to at most one S1, and `decide()` compares each pair with the best of its `(i1, is_s3)` group, but the S2 and S3 matches accepted for the same S1 are never compared with each other. An S1 entity may accept an S2 record at one address and an S3 record at another; whichever is wrong is a false positive. On a two-match entity that accepts one correct and one wrong record, $P = R = 0.5$ and the score falls from 1.0 to 0.5; on a one-match entity that accepts its true record plus a wrong one, $P = 0.5$, $R = 1$ and the score is 0.556.

14.2.5 Throughput

`features()` evaluates about twenty rapidfuzz and Python-level expressions per pair inside list comprehensions, and the sweep calls `to_lists()` and `macro_f05()`, both Python loops, $6 \times 31 = 186$ times. Chapter 12 estimates the rapidfuzz loop as the third-largest block of wall-clock after retrieval and cross-validation, and the grid as minor; none of these has been timed on the real data. The effect on the score is indirect: the candidate budget k is chosen with this cost in mind, so a faster feature stage permits a larger k and a higher recall ceiling at the same wall-clock.

14.3 Risk register

Table 14.2 records the operational risks for the first real run and the submission, with qualitative likelihood and impact (Low, Medium, High) and the mitigation that exists in the code or is scheduled. Impact is judged on score or eligibility, not on engineering effort.

Table 14.2 — Risk register

Id	Risk	Likelihood	Impact	Mitigation
R1	First full run exceeds Kaggle memory or time	Medium	High (no submission)	<code>topk_sparse()</code> chunks at <code>max_cells=2e7</code> ; <code>pair_cos()</code> slices 200k pairs; reduce k as fallback
R2	Blocking recall ceiling below expectation (for example under 0.9)	Medium	High	Per-country and per-source recall logs isolate the weak view; raise k or add a view (H1, H2)
R3	France over- or under-matched because <code>(thr, ratio)</code> is calibrated on US and India	High	High	Per-country test match-rate log; pseudo-unseen-country validation (H1)
R4	Singletons over-predicted, each costing a full 1.0	Medium	High	Sweep maximises macro $F_{0.5}$ including singletons; <code>ratio</code> suppresses weak seconds
R5	Non-Latin-script names reduced to empty strings by <code>basic()</code>	Low to Medium (data dependent)	High for affected entities	Add a count of empty <code>name_c</code> after <code>prep()</code> ; address and name-plus-address views still apply; phonetic or encoder view (H1, H2)
R6	Vectorisers refitted on test shift cosine-feature scale	Medium	Medium	Context features are relative; monitor <code>cos_full</code> quantiles train vs test

Id	Risk	Likelihood	Impact	Mitigation
R7	Result not bit-reproducible (no LightGBM seed)	High	Low	Fix seeds (H1); document tolerance
R8	Public/private leaderboard split variance	Medium	Medium	Select parameters on OOF, never on public score
R9	_postal captures a 5- or 6-digit non-postal number	Low	Low	_addr_nums() keeps the leading number; postal_state is ternary, so the model can discount it
R10	Feature stage too slow at real candidate volume	Medium	Medium	rapidfuzz.process.cdists vectorisation (H1)
R11	Roadmap re-ranker violates licence or 8B limit, or loads weights via network	Low	High (disqualification)	Restrict to Table 14.3 families; attach weights offline; see Chapter 13

14.4 Roadmap

The roadmap has three horizons ordered by effort and by confidence that the change improves macro $F_{0.5}$. Every item has one acceptance rule: it is merged only if the OOF macro $F_{0.5}$ from the `GroupKFold` loop in `main()` does not fall, and preferred if the blocking recall ceiling rises. Figure 14.1 shows the sequencing; the horizons are cumulative.

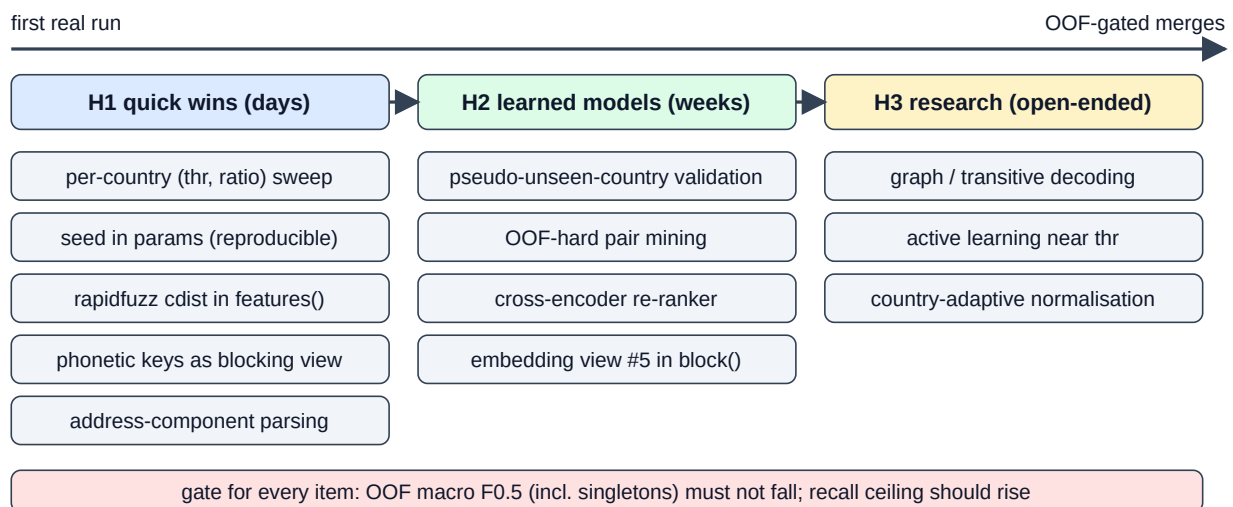


Figure 14.1 — Roadmap timeline: three cumulative horizons, each item gated on the OOF macro $F_{0.5}$ reported by `main()`.

14.4.1 Horizon 1: quick wins

These changes touch a few lines each and carry no licence or resource risk.

- **Per-country and per-source (thr, ratio).** The sweep in `main()` is repeated with `macro_f05()` restricted to one country's S1 entities. France, having no labels, receives the higher of the two thresholds, because a false positive on a singleton costs 1.0 whereas a missed match costs a fraction.
- **Seed fixing.** Adding `seed`, `bagging_seed` and `feature_fraction_seed` to `params` makes the fold models and the refit deterministic, so the reproducibility claim of Chapter 11 becomes exact.
- **Vectorised string features.** Replacing the list comprehensions in `features()` with `rapidfuzz.process.cdists` on aligned arrays removes the per-pair Python overhead; the freed budget goes to a larger `k_addr` or `k_postal`, the two views with the smallest `k` (10 each).

- **Phonetic keys.** A fifth blocking key from a phonetic encoding of core-name tokens (Double Metaphone or an in-repo consonant-skeleton code, no external data) groups `vishwakarma` with `bishwakarma` and `sri` with `shri`; it joins the union in `block()` like the same-postal view.
- **Address-component parsing.** A regex split of `addr_n` into house number, street, locality, city and postal code enables component-wise features, notably a street-name ratio not diluted by landmarks such as `near sbi atm`.

14.4.2 Horizon 2: learned components

Horizon 2 replaces the two hand-built parts that limit generalisation, the abbreviation tables and the n-gram representation, with learned models, subject to the licence and size rules of Chapter 13.

Pairwise cross-encoder re-ranker. A cross-encoder takes the concatenation `name_n | addr_n [SEP] name_n | addr_n` of a pair and outputs one match probability, attending across both records, which suits abbreviation, transliteration and word-order noise. It is trained on the OOF-hard pairs (those with `oof` probability in an illustrative band of roughly 0.2 to 0.8, to be set from the run) plus all positives, so it learns the cases the feature model cannot separate. Its output becomes one more LightGBM feature or, more conservatively, a second-stage re-ranker applied only to pairs within a margin of `thr`, leaving confident decisions and singletons untouched. Table 14.3 lists candidate backbones, all far below 8 billion parameters; weights must be attached to the Kaggle environment offline.

Table 14.3 — Candidate backbones for the pairwise re-ranker and the embedding view

Model family	Approx. parameters	Licence	Multilingual	Role
multilingual-e5-small / base / large	118M / 278M / 560M	MIT	Yes (100+ languages)	Embedding view; fine-tune as bi- or cross-encoder
paraphrase-multilingual-MiniLM-L12-v2	118M	Apache-2.0	Yes (50+ languages)	Embedding view (fastest option)
xlm-roberta-base / large	278M / 560M	MIT	Yes (100 languages incl. Hindi, French)	Cross-encoder backbone
mdeberta-v3-base	about 280M	MIT	Yes	Cross-encoder backbone (strong pair classification)
bert-base-multilingual-cased	178M	Apache-2.0	Yes (104 languages)	Cross-encoder backbone
distilbert-base-multilingual-cased	134M	Apache-2.0	Yes	Cross-encoder backbone (CPU-friendly)
bge-m3	568M	MIT	Yes	Embedding view
LaBSE	471M	Apache-2.0	Yes (109 languages)	Embedding view; robust to transliteration
gte-multilingual-base	305M	Apache-2.0 (verify licence for the exact checkpoint)	Yes	Embedding view
Qwen2.5-0.5B / 1.5B / 7B	0.5B / 1.5B / 7B	Apache-2.0 (verify licence per size; the 3B checkpoint is not Apache-2.0)	Yes	Decoder scored as pair classifier; only if encoders prove insufficient

Embedding-based blocking view. A multilingual sentence embedding of `name_c + " | " + addr_c`, from any bi-encoder in Table 14.3, indexed by exact inner product per country block, provides a fifth top-k view whose similarity is semantic rather than orthographic, retrieving pairs that share few character n-grams. Its cosine joins `cos_name`, `cos_full` and `cos_addr` as a feature; the union logic of `block()` is unchanged.

Pseudo-unseen-country validation. The OOF loop is run with the US as training country and India as held-out validation, and vice versa; the gap between in-country and cross-country OOF macro $F_{0.5}$ is the expected penalty for France and sets the threshold margin chosen in horizon 1.

14.4.3 Horizon 3: research directions

- **Graph and transitive decoding.** Score S2–S3 pairs with the same feature set, build the graph of accepted and candidate edges, and decode per connected component so that an S1 entity's accepted S2 and S3 records are consistent with each other. This removes the contradiction of Section 14.2.4; the solver rejects, never adds, when evidence conflicts, so the step stays precision-first.
- **Active learning on uncertain pairs.** Pairs with OOF probability near `thr` carry the most information; reweighting them, or training the re-ranker on them first, spends capacity where the $F_{0.5}$ decision is made.
- **Country-adaptive normalisation.** Learn abbreviation expansions from the training pairs by aligning tokens between matched names and addresses, and apply per-country tables in `norm_name()` and `norm_addr()`; a new country is covered by unsupervised alignment among its own candidate pairs.

Design note. The order of the horizons follows the metric. Horizon 1 raises precision where it is cheapest (per-country thresholds, a singleton-safe France margin) and recall where it is safest (a larger candidate budget behind an unchanged matcher). Horizon 2 attacks residual errors no string feature can express. Horizon 3 changes the decision structure itself, the step with the highest potential and the highest risk of new false positives, so it comes last.

15 Appendices

This chapter is the reference layer of the document. It collects, in tabular form, every knob that governs the behaviour of `er_pipeline.py` (command-line flags, function parameters, lookup tables, model parameters and search grids), every line that `main()` writes to the run log together with the range in which each value indicates a healthy run, the exact byte-level format of the seven input files and the two output files, a glossary of the vocabulary used throughout the document, and a one-line index of every function in the script. Nothing here introduces new behaviour; each entry is traceable to a name in the source, and the narrative chapters (see Chapters 5 to 11) explain why the values are what they are.

15.1 Appendix A: CLI and configuration reference

The pipeline exposes three command-line flags and no configuration file; every other setting is a Python default or a module-level constant. This is deliberate: a single script whose behaviour is fixed by its source is easier to audit under the reproducibility review described in Chapter 13 than one whose behaviour depends on an external configuration. Table 15.1 lists each setting with its owner, its default and the effect of changing it. Values marked *const* are module-level constants (regular expressions and lookup tables) that cannot be changed without editing the file.

Two conventions in the argument parser matter for operation. First, `parse_known_args()` is used instead of `parse_args()`, so the `-f kernel.json` argument that Jupyter injects when the script is run inside a notebook cell is ignored rather than rejected. Second, `--data-dir` is optional: when it is absent, `find_data_dir()` searches `/kaggle/input`, the working directory, its parent and `dataset/` recursively for `train_source1.tsv` and returns the grandparent of the first hit, which is the directory that contains both `train/` and `test/`.

Table 15.1 — Configuration reference: every flag, parameter, constant and grid in `er_pipeline.py`

Setting	Owner	Default / value	Effect and notes
<code>--data-dir</code>	<code>main()</code> <code>argparse</code>	none (auto-detect)	Directory containing <code>train/</code> and <code>test/</code> . Absent: <code>find_data_dir()</code> searches <code>/kaggle/input</code> , <code>.</code> , <code>...</code> , <code>dataset</code> ; raises <code>FileNotFoundError</code> when no <code>train_source1.tsv</code> is found.
<code>--out-dir</code>	<code>main()</code> <code>argparse</code>	output	Destination of <code>matching_results.tsv</code> and <code>candidate_pairs.tsv</code> ; created with <code>os.makedirs(exist_ok=True)</code> .
<code>--folds</code>	<code>main()</code> <code>argparse</code>	5 (int)	Number of GroupKFold splits grouped by S1 row index i1. Fewer folds: faster, noisier OOF estimate; more folds: closer to the full-data model.
<code>read_tsv()</code> options	<code>read_tsv()</code>	<code>sep="\t"</code> , <code>dtype=str</code> , <code>keep_default_na=False</code> , <code>quoting=3</code>	Tab separation, no NA parsing (empty cells stay <code>" "</code>), <code>QUOTE_NONE</code> so quotation marks inside names and addresses are data, not delimiters.

Setting	Owner	Default / value	Effect and notes
NAME_ABBR	const	18 entries	Name abbreviation expansion applied by <code>norm_name()</code> token by token (corp, co, inc, ltd, pvt, pvtltd, intl, mfg, svc, svcs, bros, assoc, natl, tech, grp, ent, cie, ste).
LEGAL	const	26 tokens	Tokens removed by <code>core_name()</code> : 16 English/Indian legal and function words plus 10 French forms (sarl, sas, sasus, sa, eurl, sci, snc, societe, compagnie, et). Falls back to the full name when nothing remains.
ADDR_ABBR	const	41 entries	Address abbreviation expansion applied by <code>norm_addr()</code> ; covers US street types, compass points, Indian idioms (nr, opp, mg, ngr, clny, sec, marg, salai) and French idioms (bd, fbg, chem, imp).
ADDR_STOP	const	16 tokens	Tokens dropped from <code>addr_n</code> to form <code>addr_c</code> : landmark connectors (near, opposite, behind, beside, next, to), function words and French articles (de, la, le, du, des).
_postal	const regex	<code>\b(\d{5,6})(?:-\d{4})?\b</code>	Postal code = last match on the raw address; 5 digits (US ZIP, French code postal) or 6 digits (Indian PIN); optional ZIP+4 suffix is consumed but not captured.
_dotted	const regex	<code>(?<![a-z])(?:[a-z]\.){2,}[a-z]?(?:![a-z])</code>	Dotted acronyms (p.v.t., u.s.a) collapsed before punctuation removal.
_ALIAS	const regex	d/b/a, d.b.a., dba, doing business as, trading as, t/a, a/k/a, a.k.a., also known as, formerly (known as); case-insensitive, spaces around the slashes tolerated	Applied to the raw name in <code>prep()</code> and <code>name_parts()</code> , before <code>basic()</code> destroys the slashes and dots the markers depend on. A bare aka is deliberately excluded because it can be a legitimate name token.
_punct, _space, _digits	const regex	<code>[^a-z0-9]+, \s+, \d+</code>	Punctuation-to-space, whitespace collapse, digit-run extraction for nums and house.
k_name	<code>block()</code>	25	Top-k neighbours per S1 row in the core-name view. Must exceed the maximum number of true matches per S1 entity (logged by <code>profile()</code>).
k_full	<code>block()</code>	25	Top-k in the <code>name_c + " " + addr_c</code> view.
k_addr	<code>block()</code>	10	Top-k in the address-only view; smaller because addresses are shared by unrelated businesses.
k_postal	<code>block()</code>	10	Top-k by name cosine among the S2/S3 records of the same country block that share the S1 row's postal value; rows with an empty postal code skip this view.
cosine floor	<code>block()</code>	<code>sc > 0.05</code>	Neighbours at or below 0.05 cosine are discarded in all four views.

Setting	Owner	Default / value	Effect and notes
TfidfVectorizer	block()	analyzer="char_wb", ngram_range=(2, 4), min_df=2, sublinear_tf=True, dtype=float32	One vectoriser per view, fitted on the union of the S1 and S2/S3 texts of the split being processed.
max_cells	topk_sparse()	2e7	Budget for the dense chunk $\times B $ score block; chunk rows = $\max(16, \text{int}(\text{max_cells} // \text{n_b}))$.
k clamp	topk_sparse()	$\min(k, \text{n_b})$	Prevents argpartition from requesting more columns than exist in a small country block.
chunk	pair_cos()	200_000	Pairs per slice when recomputing cos_name, cos_full, cos_addr; bounds peak memory.
params	main()	objective="binary", learning_rate=0.05, num_leaves=63, min_child_samples=40, feature_fraction=0.8, bagging_fraction=0.8, bagging_freq=1, lambda_l2=1.0, verbose=-1, n_jobs=-1	LightGBM parameters shared by the fold models and the final refit. No seed is set, so bagging is nondeterministic at the last decimals.
max rounds / patience	main()	2000 / early_stopping(100)	Per fold: up to 2,000 boosting rounds, stop after 100 rounds without validation-loss improvement.
refit rounds	main()	$\text{int}(\text{mean}(\text{best_iteration}) * 1.1)$	Final model on all training pairs; 10% more rounds than the fold average compensates for the larger training set.
ratio grid	main()	(0.0, 0.6, 0.7, 0.8, 0.9, 1.0)	Relative-ratio candidates for decide(); 0.0 reproduces a flat threshold.
thr grid	main()	$\text{np.arange}(0.20, 0.96, 0.025) = 31$ values, 0.200 to 0.950	Absolute probability threshold; $6 \times 31 = 186$ decodes of the OOF table are scored with macro_f05().
sweep initial state	main()	best_thr=0.5, best_ratio=0.0, best=-1.0	Any scored grid point replaces the initial state; the fallback 0.5 is never used in practice.
decide(ratio)	decide()	0.0	Function default; main() always passes the swept value.
to_lists(col)	to_lists()	"i2"	Column of the decoded table holding the S2/S3 row index.
importance display	main()	feature_importance("gain"), head(15)	Printed after the refit; not written to a file.
profile sample	profile()	head(3)	Three raw rows of S1 and of S2+S3 are printed for a visual sanity check of the parse.
log() format	log()	[{elapsed:7.1f}s] {msg}	Elapsed seconds since module import T0, right-aligned in 7 characters, one decimal.

15.2 Appendix B: Log-line reference

Every diagnostic the pipeline produces goes through `log()`, which prefixes the message with the elapsed wall-clock time, so the run log doubles as a coarse profiler (see Chapter 12). Figure 15.1 places each log line on the `main()` timeline; Table 15.2 lists the lines in the order they appear, with the quantity each one reports and the range in which the value indicates a healthy run. The healthy ranges are design expectations derived from the metric and the algorithm, not measured results: the pipeline has not yet been run on the released data, and the measured values are to be filled from the run log in the results table of Chapter 10.

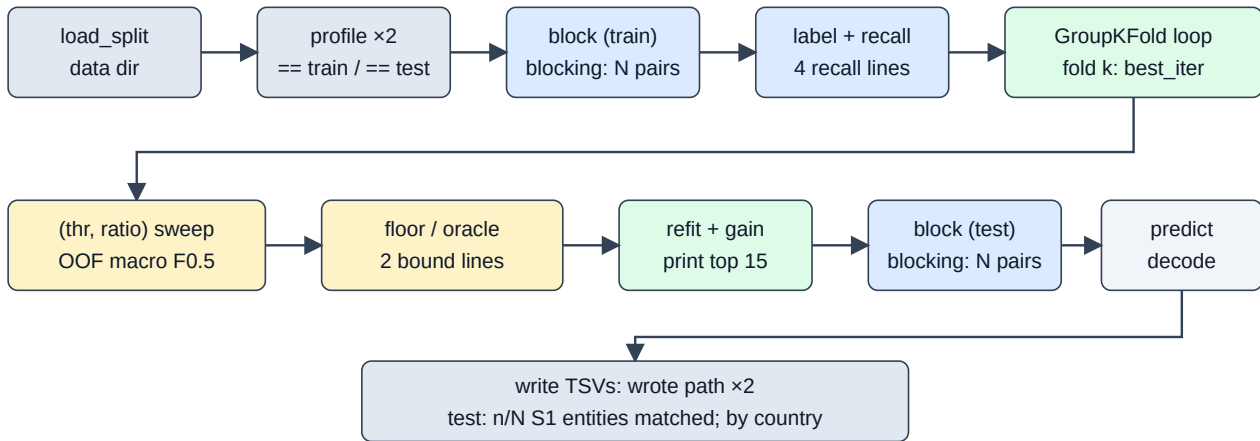


Figure 15.1 — Position of every `log()` call on the `main()` timeline; box colour marks the stage (grey: I/O, blue: blocking, green: model, yellow: decision tuning).

Table 15.2 — Log-line reference in order of appearance, with meaning and healthy range

#	Log line (format)	Emitted by	Meaning	Healthy range / what a deviation means
1	data dir: {path}	<code>main()</code>	Resolved data directory.	Path must contain <code>train/</code> and <code>test/</code> . A path under <code>/kaggle/input</code> when running locally means a stale copy was picked up.
2	<code>== train: S1={n:,}</code> <code>S2+S3={m:,}</code>	<code>profile()</code>	Record counts per split.	Must equal the line counts of the TSVs minus headers; a shortfall means a tab or quote parse fault.
3	<code>S1 countries: {dict}</code>	<code>profile()</code>	Raw country value counts in S1.	Train: US and India only. Test: additionally France. An S1 label absent from S2/S3 after <code>basic()</code> folding (including a blank) falls back to all S2/S3 records in <code>block()</code> ; a spelling variant present on both sides forms its own partition and should be noted.
4	<code>S2/S3 countries: {dict}</code>	<code>profile()</code>	Raw country counts in S2+S3.	Same keys as line 3; a key present in S1 but absent here triggers the all-records fallback for that group.
5	<code>singletons: {f:.3f}</code> <code>mean matches: {x:.2f}</code> <code>max: {n}</code>	<code>profile()</code> (train only)	Fraction of S1 entities with an empty ground-truth list; mean and maximum list length.	<code>max</code> should not exceed <code>k_name</code> (25): above it, no single view can hold every true match of the largest entity and only the union of the four views can reach them. The singleton fraction sets the floor a trivial empty submission would score (see Chapter 2).
6	(raw head(3) of S1 and S2+S3)	<code>profile()</code> via print	Visual parse check.	Four named columns with plausible content; a single wide column means the file was read without a tab separator.

#	Log line (format)	Emitted by	Meaning	Healthy range / what a deviation means
7	lines 2 to 4 and 6 repeated for == test	profile()	Test split profile; no singleton line because gt is None.	As above.
8	blocking: {n:,} pairs ({x:.1f} per S1)	block() (train)	Size of the de-duplicated candidate table and its mean per S1 entity.	Upper bound per S1 is k_name + k_full + k_addr + k_postal = 70 before de-duplication; typical values are well below because the views agree. Near 0 means the country partition or the 0.05 floor removed everything; near 70 means the views disagree entirely.
9	blocking recall ceiling: {r:.4f} (pos={p:,} / true={t:,})	main()	Fraction of ground-truth pairs present in the candidate table.	As close to 1.0 as possible; every missed pair is unrecoverable downstream. A value below about 0.9 makes blocking, not the matcher, the bottleneck.
10	S1 entities with >=1 true match blocked: {a:,}/{b:,}	main()	Entity-level recall of blocking (the metric is macro over entities).	Ratio should track line 9; if it is markedly lower, misses concentrate on entities with a single true match, which cost a full point each.
11	blocking recall by country: {dict}	main()	Line 9 broken down by normalised S1 country.	The two seen countries should be close; a gap of several points indicates a country-specific normalisation deficit (see Chapter 5).
12	blocking recall by source: {dict}	main()	Line 9 broken down by S2 / S3.	Similar values; the weaker source reveals which field quality the views fail on.
13	fold {k}: best_iter={n} (x --folds)	main()	Early-stopped round count per fold.	Well below 2,000 and similar across folds. 2,000 means no convergence (raise the cap); a very small value suggests a trivially separable or leaking feature.
14	00F macro F0.5 = {s:.4f} at thr={t:.3f} ratio={r}	main()	The validation score and the selected operating point.	This is the number reported as the expected leaderboard score for seen countries. thr should be interior to [0.20, 0.95]; a boundary value means the grid should be widened. thr above 0.5 is expected because singletons and F _{0.5} both reward precision.
15	macro F0.5 if every candidate were predicted (no model; floor): {s:.4f}	main()	Score of predicting all candidates.	Always at or below line 14; the gap is the value the matcher adds over blocking alone.
16	blocking oracle macro F0.5 (predict exactly the true candidate pairs; ceiling): {s:.4f}	main()	Score of a perfect matcher on the blocked pairs.	Always at or above line 14. Gap to line 14 = matcher headroom; gap to 1.0 = blocking headroom.
17	(gain importances, 15 rows)	main() via print	Top features of the refit model.	Context features (*_gap1, rank1) and cos_full are expected near the top; a length or count feature dominating suggests a distribution artefact.
18	blocking: {n:,} pairs ({x:.1f} per S1)	block() (test)	Test candidate table size.	Pairs per S1 comparable to line 8; the test value is what candidate_pairs.tsv contains.
19	wrote {path} (x 2)	main()	Output file paths.	Two lines, matching_results.tsv then candidate_pairs.tsv.

#	Log line (format)	Emitted by	Meaning	Healthy range / what a deviation means
20	test: {n:},{N:},{ S1 entities matched; by country: {dict}}	main()	Fraction of test S1 entities with a non-empty match list, overall and per raw country.	Seen countries should approximate 1 minus the train singleton fraction (line 5). France should be in the same range: far lower means the unseen country is starved by the threshold, far higher means it is over-matched (see Chapter 14).

Risk. Lines 14 to 16 are computed on training entities of the seen countries only. A healthy OOF score does not certify French performance; line 20 is the only France-specific signal the log provides.

15.3 Appendix C: File format reference

All nine files are tab-separated text with a single header row and no quoting. The pipeline reads the seven dataset files with `read_tsv()`; the problem statement's rule that tabs are the only delimiter matters because both addresses and identifier lists contain commas. The examples below use illustrative identifiers and values in the format the problem statement specifies; they are not rows of the released data. Tab characters are shown as literal tabs inside the code blocks.

train/train_source1.tsv, train/train_source2.tsv, train/train_source3.tsv, test/test_source1.tsv, test/test_source2.tsv, test/test_source3.tsv

Four columns: `entity_id` (prefix S1-, S2- or S3- identifies the source), `business_name`, `business_address`, `country`. Any field other than `entity_id` may be empty. The problem statement designates Source 1 as the deduplicated reference source and gives no such guarantee for Sources 2 and 3; there is no separate source column, the prefix and the file are the only source indicators.

```
entity_id      business_name  business_address      country
S1-00001      Acme Holdings LLC dba Acme Pizza      12 Main St, Suite 4, Springfield,
62704 US
S2-00047      ACME PIZZA      12 Main Street Ste 4 Springfield IL 62704-1234 US
S3-00812      Sharma Bros. Pvt. Ltd. Nr SBI ATM, MG Rd, Sector 5, Noida 201301
India
```

train/train_ground_truth.tsv

Two columns: `source1_entity_id` and `matched_entity_ids`, the latter a comma-separated list of S2/S3 ids, empty for a singleton. `load_split()` parses it into a dictionary of sets, dropping empty tokens. There is no test ground truth.

```
source1_entity_id      matched_entity_ids
S1-00001      S2-00047,S3-00812
S1-00002      S3-00004
S1-00003
```

output/matching_results.tsv

Same shape as the ground truth: one row per test S1 entity in file order, `matched_entity_ids` de-duplicated by `dict.fromkeys()` and empty for predicted singletons. The singleton row is written as the id, a tab and a newline. This is the only file scored on the leaderboard.

output/candidate_pairs.tsv

Header `source1_entity_id`, `candidate_entity_ids`; one row per test S1 entity listing every S2/S3 record in the candidate table `fte`, i.e. exactly the rows scored by `model.predict()`. Matches are a subset by construction.

```

source1_entity_id      candidate_entity_ids
S1-00001               S2-00047,S2-00193,S3-00812,S3-00999
S1-00002               S3-00004
S1-00003

```

Design note. `validate_submission.py` (standard library only) enforces the rules the writer relies on: exact header, exactly two tab-separated columns per row, no quote characters, every test S1 id exactly once and no unknown S1 id, no empty or whitespace-padded token, S2-/S3- prefixes that exist in the test set, no duplicate token within a list, and matches $\subseteq$ candidates when `--candidate` is given. A PASS (exit code 0) from the validator is the packaging gate (see Chapter 11); it does not compute the score.

15.4 Appendix D: Glossary

Table 15.3 defines the vocabulary used in this document. Where a term names a specific construct in the code, the construct is given.

Table 15.3 — Glossary of terms

Term	Definition
Entity resolution (ER)	Determining which records, from one or more sources without shared keys, refer to the same real-world entity; here, linking S1 businesses to S2/S3 records.
Deduplicated reference source	Source 1, in which each business appears once; justifies assigning every S2/S3 record to at most one S1 entity in <code>decode_table()</code> .
Blocking	Cheap candidate generation that restricts expensive pairwise scoring to a small set of plausible pairs; implemented by <code>block()</code> .
Candidate pair	An (S1 row, S2/S3 row) pair that survives blocking; one row of the table produced by <code>block()</code> and one entry of <code>candidate_pairs.tsv</code> .
Recall ceiling	Fraction of ground-truth pairs present among the candidates; the highest recall any matcher can reach. Logged as <i>blocking recall ceiling</i> .
Reduction ratio	$1 - (\text{candidate pairs}) / (S1 \times S2+S3)$; the fraction of the full cross product that blocking discards.
Blocking oracle	Macro $F_{0.5}$ obtained by predicting exactly the true pairs among the candidates; the score of a perfect matcher.
Singleton	An S1 entity with no true S2/S3 match. Scores 1.0 when the predicted list is empty and 0.0 otherwise.
Precision / Recall	Per entity: true positives divided by predicted ids / by true ids.
F_β , $F_{0.5}$	Weighted harmonic mean of precision and recall; $F_{0.5} = 1.25 \cdot P \cdot R / (0.25 \cdot P + R)$ weights precision twice as much as recall. Implemented by <code>f05()</code> .
Macro average	The metric is computed per S1 entity and averaged with equal weight, singletons included; <code>macro_f05()</code> .
Open-set label	A categorical field whose test values may not appear in training (country: France). The pipeline never enumerates countries; unseen S1 labels fall back to all S2/S3 records.
NFKD folding	Unicode compatibility decomposition followed by dropping non-ASCII code points; turns <i>Société</i> into <i>Societe</i> . First step of <code>basic()</code> .
Legal suffix	A corporate-form token (Ltd, LLC, SARL) carrying no identity information; stripped by <code>core_name()</code> using the LEGAL set.
Core name	<code>name_c</code> : the normalised name with legal suffixes removed; the primary text for blocking and name features.
DBA / alias marker	"Doing business as" and equivalents (t/a, a.k.a., formerly) separating a legal name from a trade name inside one field; split by <code>_ALIAS</code> in <code>name_parts()</code> .

Term	Definition
ZIP, PIN, code postal	US 5-digit (+4) postal code, Indian 6-digit Postal Index Number, French 5-digit postal code; all captured by <code>_postal</code> .
House number	The leading digit run of the normalised address (house); compared as a ternary state.
TF-IDF	Term frequency $\times$ inverse document frequency weighting of a text vector; here over character n-grams with <code>sublinear_tf</code> ($1 + \log tf$) and <code>min_df=2</code> .
<code>char_wb</code>	scikit-learn analyser producing character n-grams only inside word boundaries (words padded with spaces), so n-grams never span two tokens.
n-gram	A contiguous sequence of n characters; the vectorisers use $n = 2, 3, 4$.
Cosine similarity	Dot product of two L2-normalised vectors; equals the sparse matrix product in <code>topk_sparse()</code> and the row-wise product in <code>pair_cos()</code> .
L2 normalisation	Scaling a vector to unit Euclidean length; <code>TfidfVectorizer</code> does this by default.
Top-k / <code>argpartition</code>	Selecting the k largest scores per row without a full sort; <code>np.argpartition</code> is $O(n)$ per row and returns the k best indices unordered.
Mutual nearest neighbour	A pair that is the best candidate for both its S1 entity and its S2/S3 record on <code>cos_full</code> (<code>cos_full_gap1 = cos_full_gap2 = 0</code> , <code>rank1 = rank2 = 1</code>).
Gap feature	A pair's feature value minus the maximum over its competitors on one side (<code>*_gap1</code> , <code>*_gap2</code>); zero for the best candidate.
Indel ratio (<code>fuzz.ratio</code>)	Normalised similarity based on insertions and deletions, 0 to 100.
<code>token_sort_ratio</code>	<code>fuzz.ratio</code> after sorting the tokens of both strings; order-insensitive.
<code>token_set_ratio</code>	Ratio over the intersection and differences of the token sets; tolerant of extra tokens on one side.
<code>partial_ratio</code>	Best <code>fuzz.ratio</code> of the shorter string against any equal-length window of the longer one; substring matching.
Jaro-Winkler	Character-transposition similarity with a bonus for a common prefix; <code>n_jw</code> .
Jaccard	$ A \cap B / A \cup B $ over token sets (<code>jacc()</code>) or digit-run sets (<code>num_overlap</code>).
Gradient boosting / LightGBM	An ensemble of decision trees fitted sequentially to the gradient of the loss; LightGBM is the MIT-licensed implementation used by <code>lgb.train()</code> .
GroupKFold	Cross-validation that keeps all rows sharing a group key (<code>i1</code>) in one fold, so an S1 entity's candidates are never split between training and validation.
OOF (out-of-fold)	Predictions for each row made by the fold model that did not train on it; the basis of the (<code>thr</code> , <code>ratio</code>) sweep.
Early stopping	Halting boosting when the validation loss has not improved for 100 rounds; the mean of the folds' best rounds, scaled by 1.1, sets the <code>refit</code> round count.
Threshold (<code>thr</code>) / relative ratio (<code>ratio</code>)	The two decision parameters of <code>decide()</code> : absolute probability floor and minimum fraction of the best probability within the (S1, source) group.
Transitive closure	Propagating matches along chains (if $A \sim B$ and $B \sim C$ then $A \sim C$); not applied here because the task is bipartite S1-to-S2/S3, but relevant to the roadmap (see Chapter 14).
Cross-encoder	A transformer that reads both records jointly and outputs a match score; a roadmap option constrained to MIT/Apache-2.0 models of at most 8B parameters.
Feature importance (gain)	Total loss reduction attributed to splits on a feature across all trees; printed for the top 15.

15.5 Appendix E: Function index

Table 15.4 lists every function defined in `er_pipeline.py` in source order, with its inputs, outputs and a one-line description. Names beginning with an underscore are private helpers; `_acr()` is a closure defined inside `features()`. The stage each function belongs to follows the architecture in Chapter 4.

Table 15.4 — Function index of `er_pipeline.py`

Function	Inputs → outputs	Description
<code>log(msg)</code>	<code>str</code> → prints	Writes [elapsed s] msg to stdout with <code>flush=True</code> .
<code>find_data_dir(explicit=None)</code>	<code>str</code> or <code>None</code> → <code>str</code>	Returns the explicit path or searches four roots recursively for <code>train_source1.tsv</code> ; raises <code>FileNotFoundError</code> .
<code>read_tsv(path)</code>	<code>str</code> → <code>DataFrame</code>	Tab-separated read, all columns as <code>str</code> , no NA parsing, <code>QUOTE_NONE</code> .
<code>load_split(data_dir, split)</code>	<code>str</code> , "train"/"test" → (<code>S1 df</code> , <code>S2+S3 df</code> , <code>gt dict</code> or <code>None</code>)	Loads the three source files, concatenates <code>S2</code> and <code>S3</code> , and parses the ground truth into <code>{s1_id: set(ids)}</code> when present.
<code>profile(name, s1, other, gt)</code>	<code>str</code> , <code>df</code> , <code>df</code> , <code>dict</code> or <code>None</code> → prints	Logs counts, country distributions, singleton fraction and match-list statistics; prints three sample rows.
<code>basic(s)</code>	<code>str</code> → <code>str</code>	NFKD ASCII fold, lowercase, &→and, @→at, dotted-acronym collapse, punctuation to space, whitespace collapse.
<code>expand(s, table)</code>	<code>str</code> , <code>dict</code> → <code>str</code>	Replaces each whitespace token by its table entry when present.
<code>norm_name(s)</code>	<code>str</code> → <code>str</code>	<code>expand(basic(s), NAME_ABBR)</code> .
<code>core_name(n)</code>	<code>str</code> → <code>str</code>	Drops tokens in <code>LEGAL</code> ; returns the input unchanged when every token is legal.
<code>name_parts(raw)</code>	<code>str</code> → <code>list[str]</code>	Splits the raw name on <code>_ALIAS</code> , core-normalises each part, drops empties; falls back to one element.
<code>norm_addr(s)</code>	<code>str</code> → <code>str</code>	<code>expand(basic(s), ADDR_ABBR)</code> .
<code>_addr_nums(a, p)</code>	normalised address, postal → (<code>frozenset</code> of digit runs, house <code>str</code>)	Digit runs minus the last occurrence of the postal code, unless that code is the leading run and the address string does not end with it (a 5-digit US house number with no ZIP); the first remaining run is returned as the house number.
<code>prep(df)</code>	<code>DataFrame</code> → <code>DataFrame</code>	Adds <code>name_n</code> , <code>name_c</code> , <code>name_alts</code> , <code>addr_n</code> , <code>addr_c</code> , <code>postal</code> , <code>nums</code> , <code>house</code> , <code>country_n</code> , <code>src</code> ; resets the index so positions are row ids.
<code>topk_sparse(A, B, k, max_cells=2e7)</code>	<code>CSR</code> , <code>CSR</code> , <code>int</code> → (<code>int64 [n_A, k]</code> indices, <code>float32 [n_A, k]</code> scores)	Chunked sparse product against <code>B.T</code> , densified, <code>argpartition</code> top-k per row; chunk size derived from <code> B </code> .
<code>pair_cos(A, B, i1, i2, chunk=200_000)</code>	<code>CSR</code> , <code>CSR</code> , <code>index arrays</code> → <code>float32 [n_pairs]</code>	Row-wise cosine of paired rows in slices of 200k pairs.
<code>block(s1, other, k_name=25, k_full=25, k_addr=10, k_postal=10)</code>	prepped <code>dfs</code> → <code>DataFrame [i1, i2, cos_name, cos_full, cos_addr]</code>	Fits three TF-IDF views, retrieves top-k per country (fallback to all records for unseen labels) plus a same-postal name view, unions, de-duplicates, recomputes the three cosines; logs pair count.
<code>jacc(a, b)</code>	<code>str</code> , <code>str</code> → <code>float</code>	Jaccard index of whitespace token sets; 0.0 if either is empty.

Function	Inputs → outputs	Description
<code>features(cand, s1, other)</code>	candidate df, prepped dfs → DataFrame	Adds string, token, alias, acronym, postal, number, length, source and context features (gap, rank, count) to the candidate table.
<code>_acr(n)</code> (inside <code>features()</code>)	str → str	Initials of the tokens of a name, for the <code>n_acronym</code> flag.
<code>f05(pred, truth)</code>	set, set → float	Per-entity $F_{0.5}$ with the singleton rule (empty truth: 1.0 iff empty prediction).
<code>decode_table(f, prob)</code>	feature df, probabilities → DataFrame [i1, i2, is_s3, p, best]	Assigns each S2/S3 row to its highest-probability S1 and records the group maximum per (S1, source).
<code>decide(d, thr, ratio=0.0)</code>	decoded df, float, float → DataFrame	Keeps rows with $p \geq thr$ and $p \geq ratio * best$.
<code>to_lists(d, s1, other, col="i2")</code>	df with i1 and col, prepped dfs → dict {s1_id: [ids]}	Converts row indices to entity-id lists, with an entry (possibly empty) for every S1 entity.
<code>macro_f05(lists, gt, keys)</code>	dict, dict, iterable → float	Mean of <code>f05()</code> over the given S1 ids.
<code>main()</code>	argv → files and log	Orchestrates load, profile, prep, blocking, labelling, cross-validation, sweep, refit, test inference, decoding and TSV writing.